

問題1.電飾



解説

城下 慎也 (IOI2011 タイ大会)

はじめに



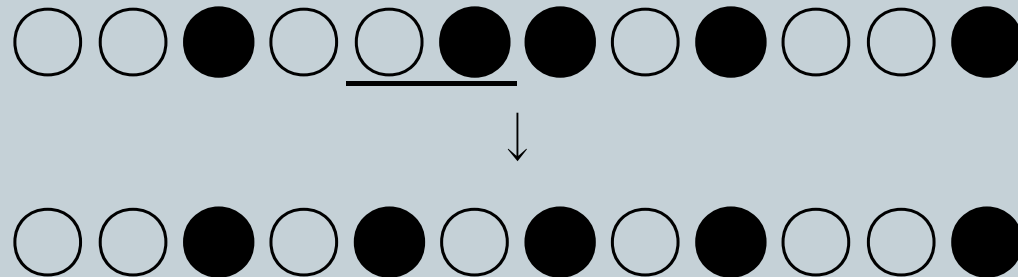
本選競技お疲れ様でした!

概要



- 電球の列が与えられるので、電球の列の一部を反転させてできる交互な区間の最大値を求めます。

例

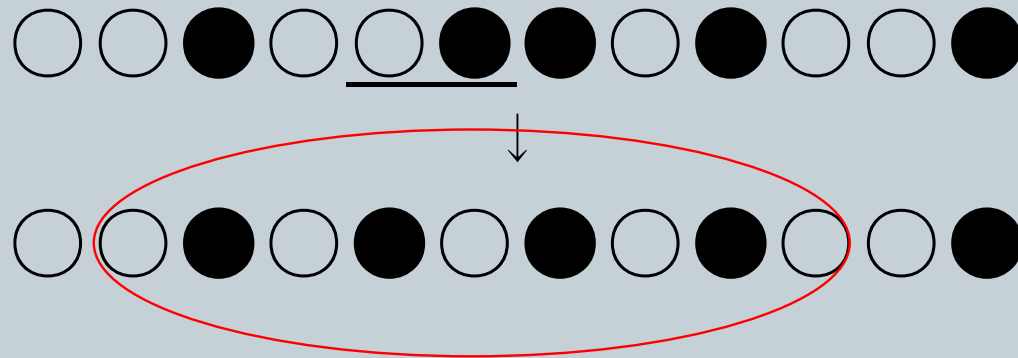


概要



- 電球の列が与えられるので、電球の列の一部を反転させてできる交互な区間の最大値を求めます。

例



長さ 9 の交互な区間

計算量について



- アルゴリズムを考える際、そのアルゴリズムの速度の目安として計算量(オーダー)について考えます。

制限

$2 \leq N \leq 100\,000$ 電飾を構成する電球の個数

- この場合は $O(N)$ や $O(N\log N)$ などの解法が満点解法となります。

部分点



- プログラミングコンテストの問題では満点解法ほど効率が良いアルゴリズムにも部分点が設けられていることがあります。

採点基準

採点用データのうち、配点の 20%分については、 $N \leq 500$ を満たす。

採点用データのうち、配点の 40%分については、 $N \leq 2000$ を満たす。

- 今回の例では、上は $O(N^3)$ で下は $O(N^2)$ で得点がもらえます。

部分点



- 部分点解法は満点解法と比べてアイデアや実装が簡単なことが多いです。
- 満点解法が思いつかなくても積極的に部分点を狙って行きましょう。
- 満点解法を思いつくヒントになる場合もあります。

部分点解法(1)



- 考えられるひっくり返す操作全てについて、ひっくり返した後に交互に並んでいる区間を探します。

操作後



部分点解法(1)



- 考えられるひっくり返す操作全てについて、ひっくり返した後に交互に並んでいる区間を探します。
- 純粹に全て調べると $O(N^2)$ の操作候補に対し $O(N^2)$ の区間それぞれ調べるので、交互か調べるときの $O(N)$ をあわせて $O(N^5)$

操作後



部分点解法(1)



- 考えられるひっくり返す操作全てについて、ひっくり返した後に交互に並んでいる区間を探します。
- 純粹に全て調べると $O(N^2)$ の操作候補に対し $O(N^2)$ の区間それぞれ調べるので、交互か調べるときの $O(N)$ をあわせて $O(N^5)$
- しかし左から順に見れば $O(N)$ で最長区間が分かる。
→全体で $O(N^3)$ となり、20%の得点が得られる。
($N \leq 500$)

操作後



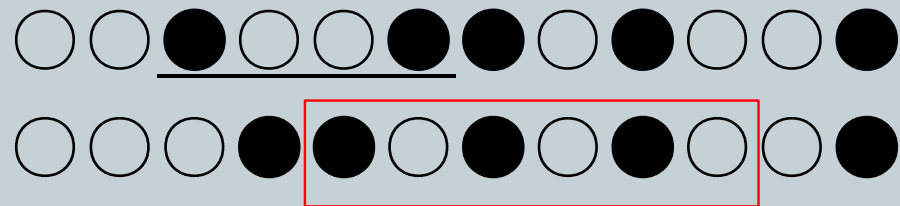
部分点解法(2)



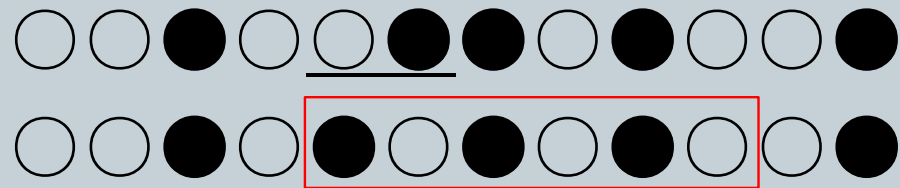
- 反転動作について考えると、反転させた領域全体を使用していない場合は考えなくても良い。

(例えば(A)の入れ替えの場合、(B)で提示されたものが(A)の解以上になる。)

(A)



(B)

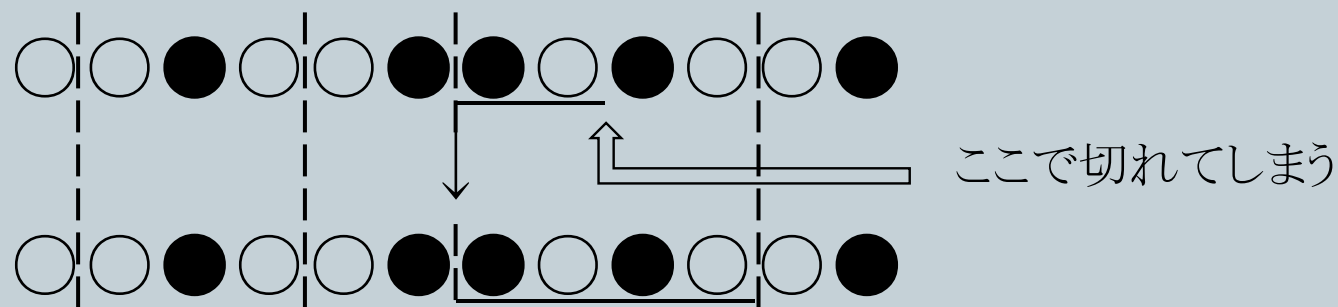


部分点解法(2)



- よって、反転させる区間内で○同士、●同士が連続すると反転させる区間全体を使用することができない。
(逆に交互な区間ならば全て使用出来る。)
- さらに、反転させる候補は交互になっている列全体に引き伸ばすことができる。

(交互になっている場所を切れ目にするるとそちら側に伸ばせなくなる。)



部分点解法(2)



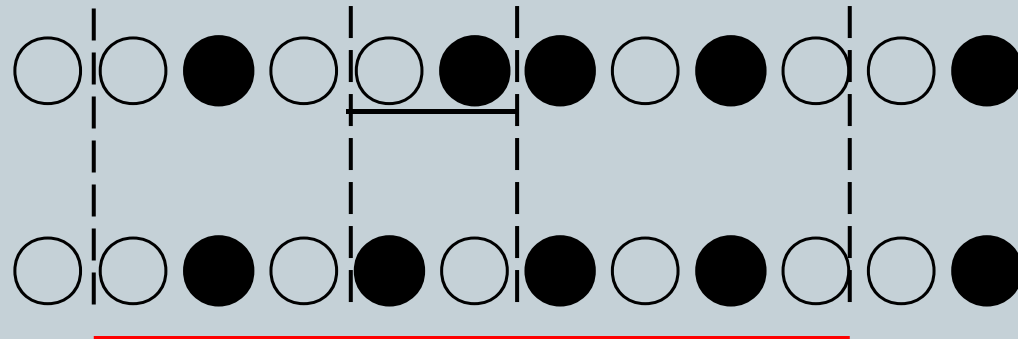
- 交互になっている区間は全部で $O(N)$ 個ある。
 - それぞれの反転を調べるのに $O(N)$
 - 全体で $O(N^2)$
 - 40%($N \leq 2,000$)の得点を得ることができる。

満点解法



- 交互列を反転させたあとについて考えると、反転させた区間は前後にある交互列1つずつを結合した区間となっている。
- この中で一番長い場所が最適解となる。

例

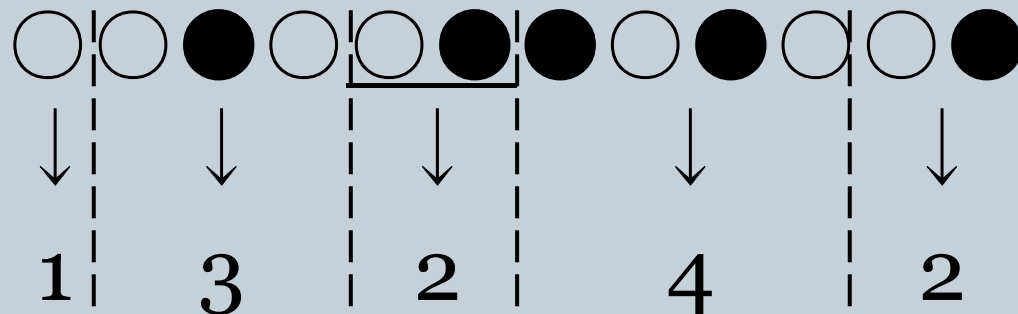


満点解法



- よって、交互列を長さの数字に変換し、最大3つ連続する場所の最大値が答えとなる。

例

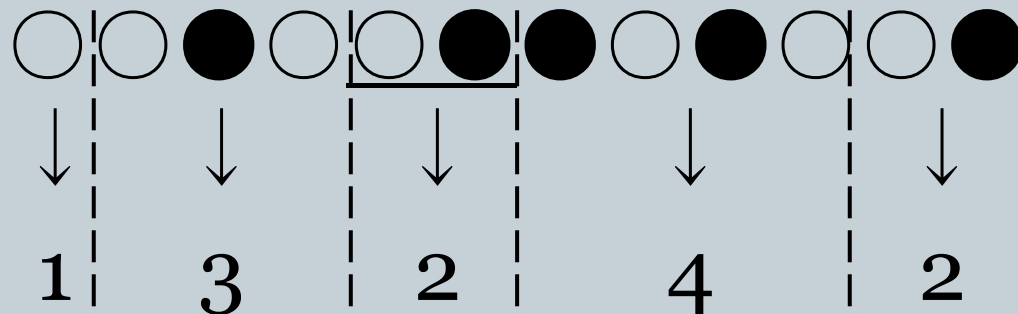


満点解法

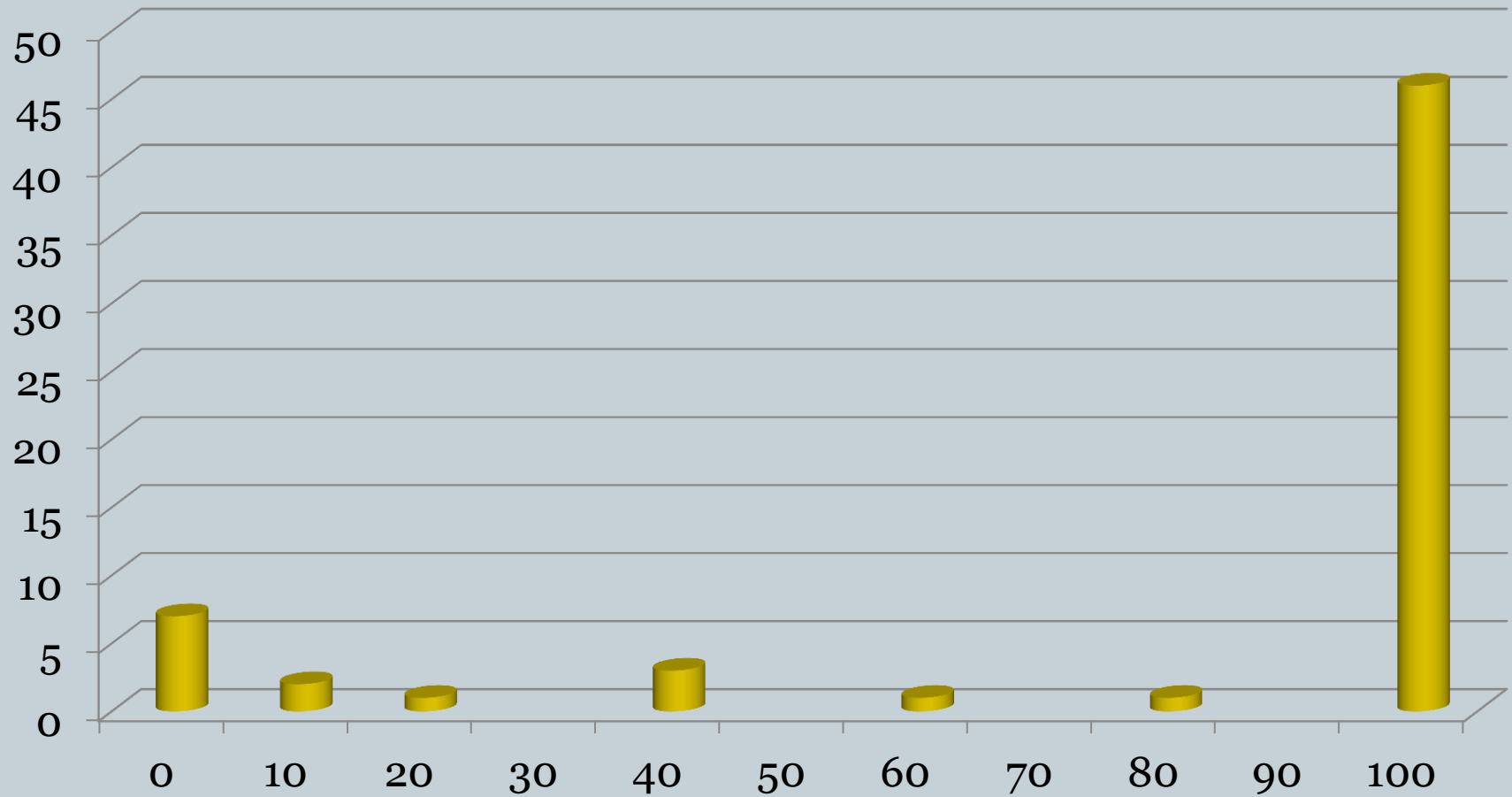


- よって、交互列を長さの数字に変換し、最大3つ連続する場所の最大値が答えとなる。
- 数字の変換に $O(N)$ かかり、その後調べるのに $O(N)$ かかるので、全体で $O(N)$ となり、満点が得られる。

例



得点分布



IOI 列車で行こう

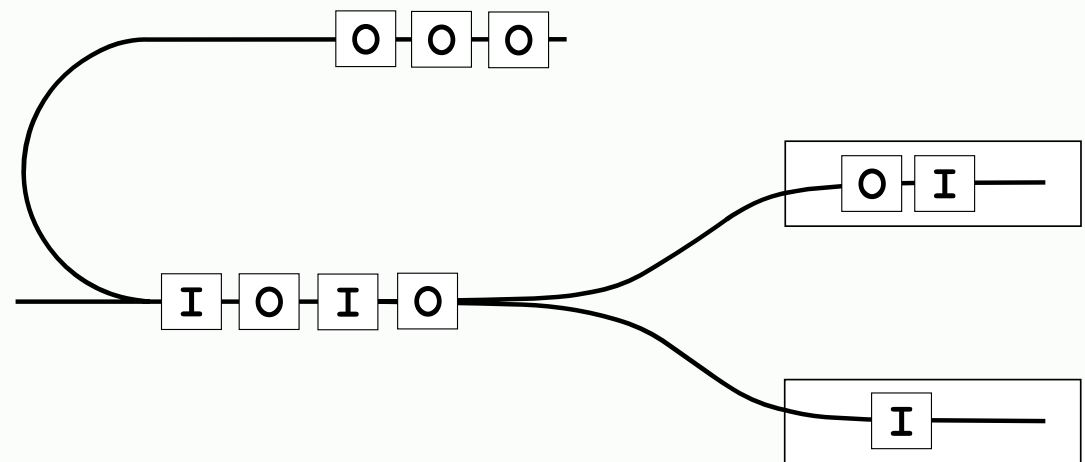
Take the 'IOI' train

今西 健介 (@japlj)

問題概要

Take the
'IOI' train

- 2つの車庫に車両がたくさん入っている
 - 車両 I, O の2種類がある
- 順に取り出して IOIO...I と連結していく
 - 最初にいくつかの車両を退避させてもOK
 - 最長の列車は？



解法説明の前に……

Take the
'IO' train

- なにやら面倒そうな条件
 - 列車編成前に**好きなだけ**車両を待機用レールに移してOK
 - 車両を使いきらずに**好きなだけ**残してOK

→ **問題の言い換え**をしよう!!!

問題の言い換え

Take the
'IOI' train

- 退避とか使い残しとか気にしない
- 全部一列に並べてしまう

O	O	O	I	O	I	O	I	O	O	I
---	---	---	---	---	---	---	---	---	---	---

出来た列からできるだけ長い

IOIOIOI...OI をとってくると考える

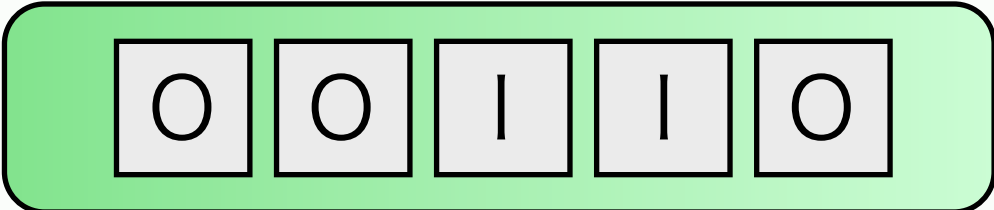
→ この列の作り方が問題になる

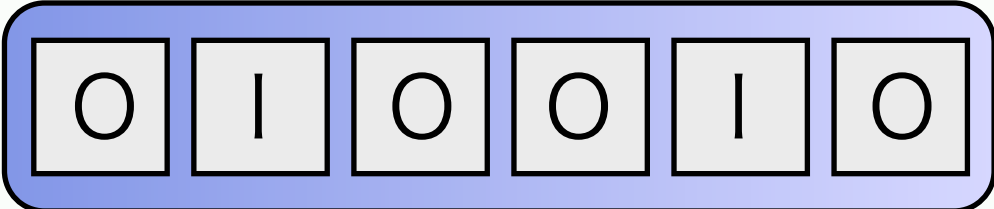
20点解法('⊙ ∨ ⊙')

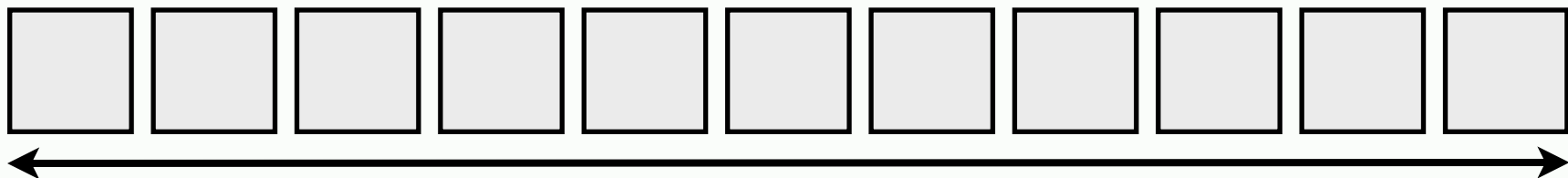
20点解法(全探索)

Take the
'IOI' train

- 作れる列を全部試すことを考えよう

車庫 S  $M = 5$

車庫 T  $N = 6$

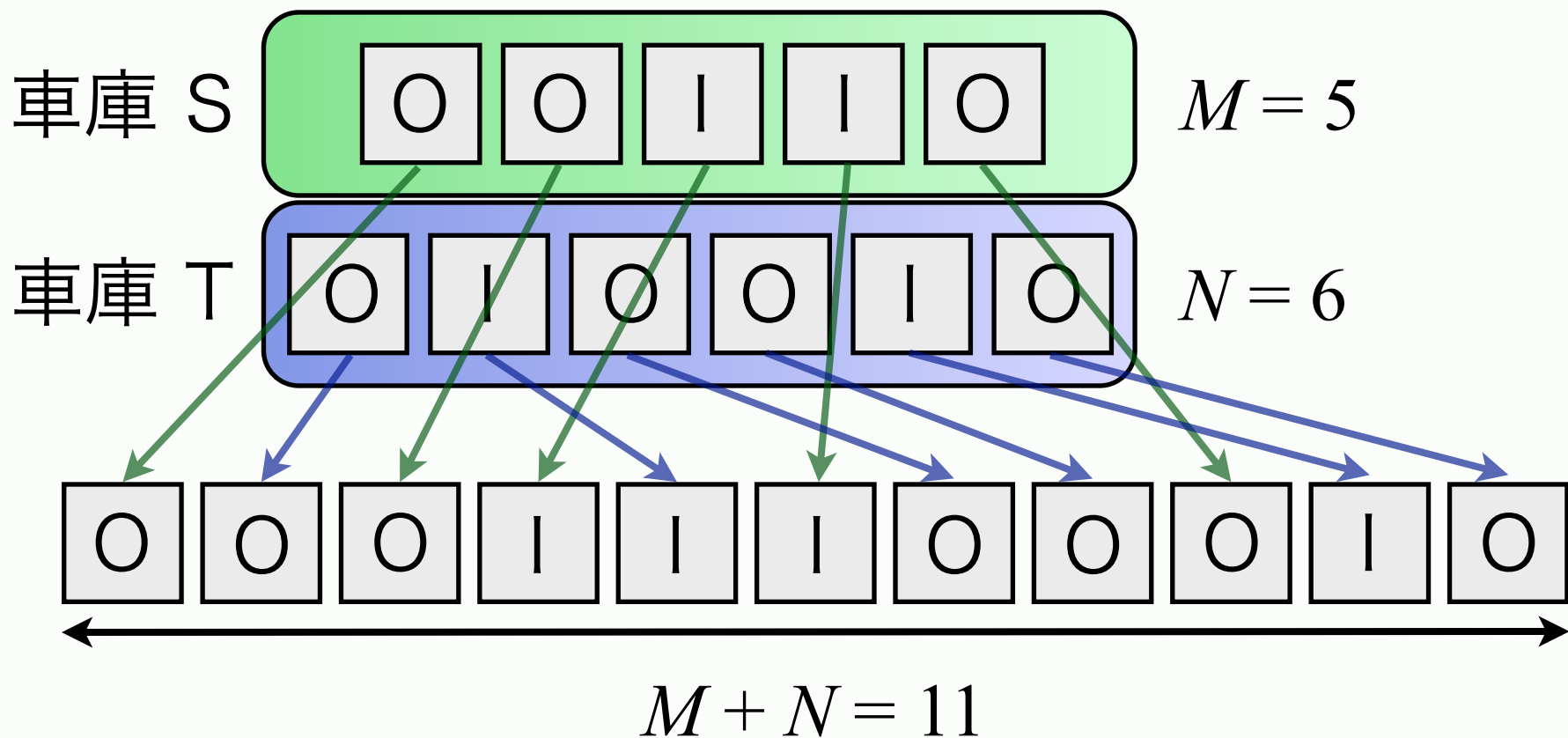


$$M + N = 11$$

20点解法(全探索)

Take the
'01' train

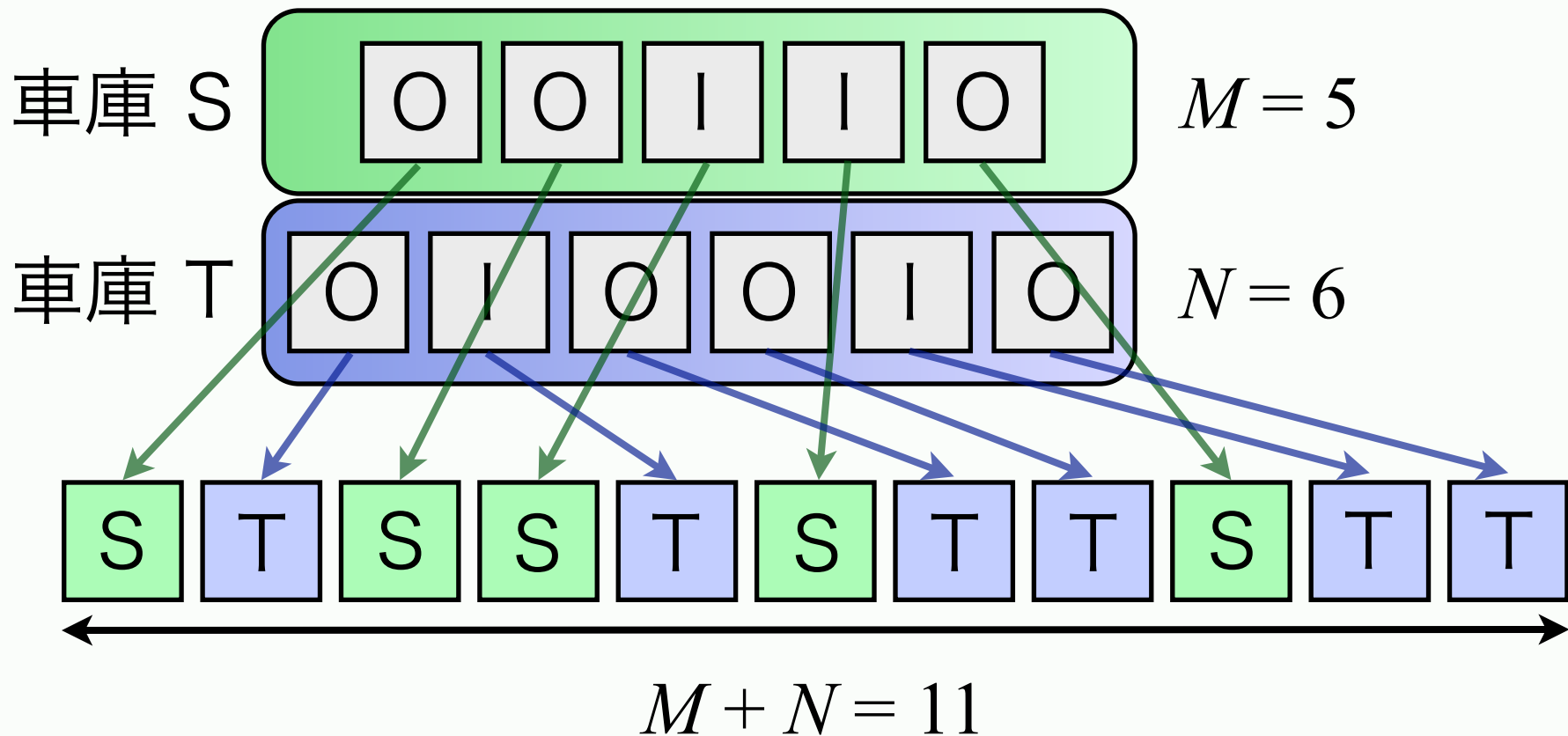
- 作れる列を全部試すことを考えよう



20点解法(全探索)

Take the
'O/I' train

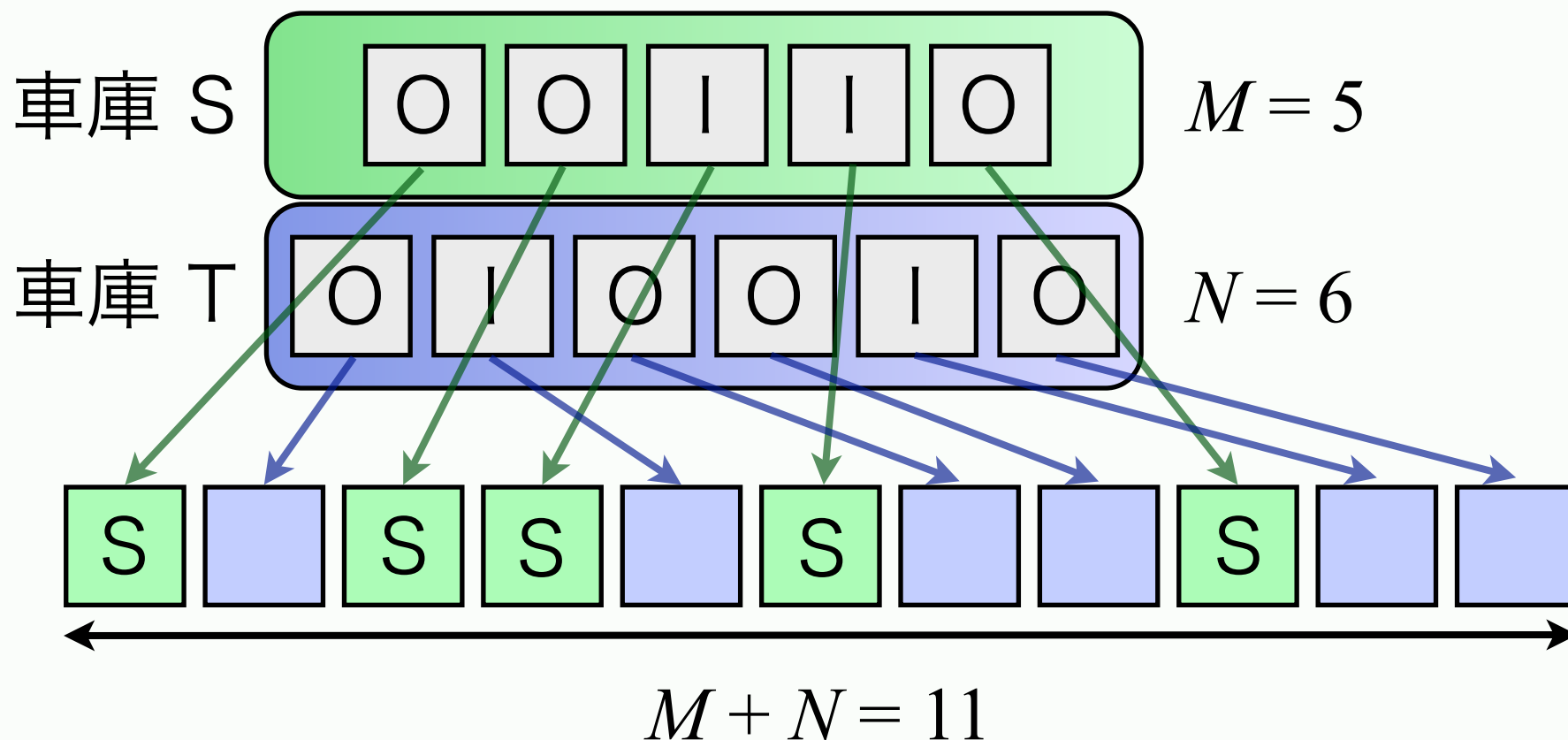
- S, Tのどちらから来るかが決まると列も決まる



20点解法(全探索)

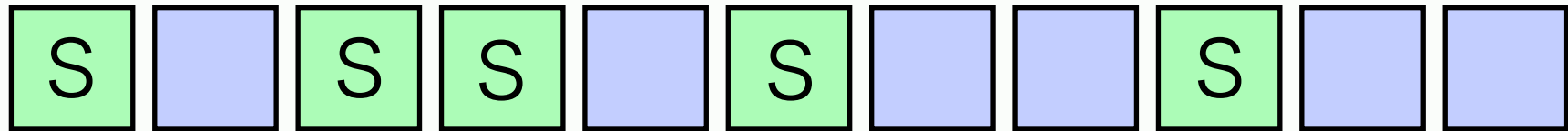
Take the
'OI' train

- といふか S だけ決めれば T も決まる



20点解法(全探索)

Take the
'IO' train



- 作れる列は全部で S の決め方と同じだけある
- これは $M+N$ 個から M 個を選ぶ**組み合わせ**

n 個のものから k 個を選ぶ方法の総数

$$\begin{aligned} {}_nC_k &= \binom{n}{k} = \frac{n!}{k!(n-k)!} \\ &= \frac{n \times (n-1) \times \cdots \times (n-k+1)}{k \times (k-1) \times \cdots \times 1} \end{aligned}$$

20点解法(全探索)

Take the
'IOI' train

- 何通りの列を調べなければならないか？
 - 20点分のケースでは $M, N \leq 10$
 - 50点分のケースでは $M, N \leq 50$
 - 100点分のケースでは $M, N \leq 2,000$
- ${}_{20}C_{10} = \mathbf{184,756} \rightarrow$ いける
- ${}_{100}C_{50} > \mathbf{10^{29}} \rightarrow$ やばい
- ${}_{4000}C_{2000} > \mathbf{10^{1202}} \rightarrow$ やばすぎ

50点解法(！ ∪ ！)

動的計画法

Take the
'IOI' train

- 20点より多くとるには**動的計画法**を使う
 - Dynamic Programming, DP
- 動的計画法ってなんだろう？
- 調べてみよう！！！！

動的計画法

Take the
'IOI' train

動的計画法

動的計画法（どうてきけいかくほう、英: Dynamic Programming, DP）は、コンピュータ科学の分野において、ある最適化問題を複数の部分問題に分割して解く際に、そこまでに求められている以上の最適解が求められないような部分問題を切り捨てながら解いていく手法である。分割統治法がトップダウン的な手法であるのに対し、動的計画法はボトムアップ的な手法といえる。

(Wikipedia)

??????

Take it easy

Take the
'101' train

(少なくとも競技プログラミングで問題を解く時は)
次のような認識でOK

- 途中までの計算結果を次の計算に利用する
- 一段階小さい(一段階前の)**状態**から, その次の**状態**の解を求める
 - 基本的に状態の数は少ないほど効率が良い

状態のとり方

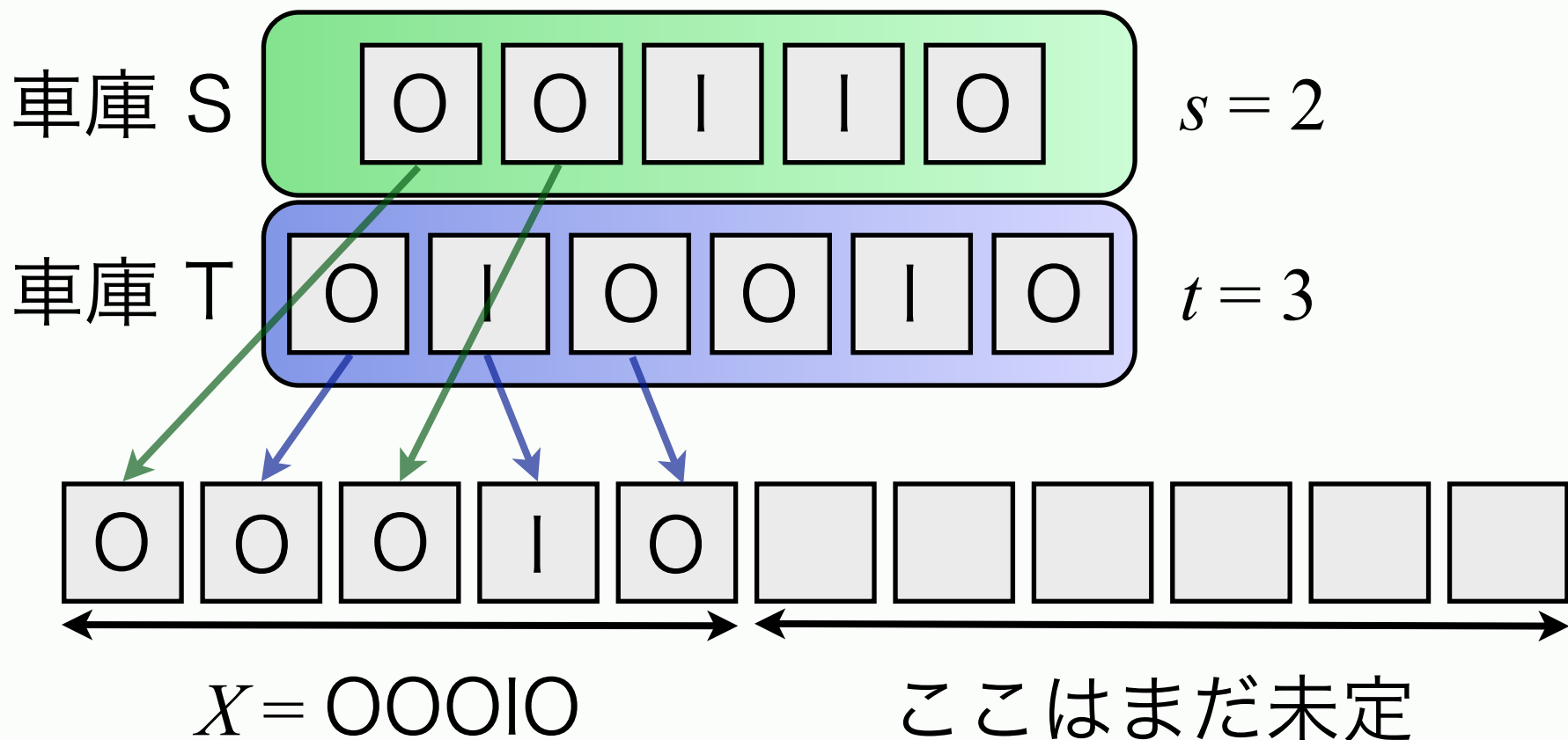
Take the
'IOI' train

- この問題で状態はどのような風にとれるだろう
 - たとえば次のわかりやすい状態のとり方だろうか？
- 車庫 S から先頭 s 両をすでに出していて
- 車庫 T から先頭 t 両をすでに出していて
- 今並んでいる車両の列が X である
- という状態

状態のとり方(例)

Take the
'IOI' train

- ここまでは決まっている, という状態



状態のとり方(例)

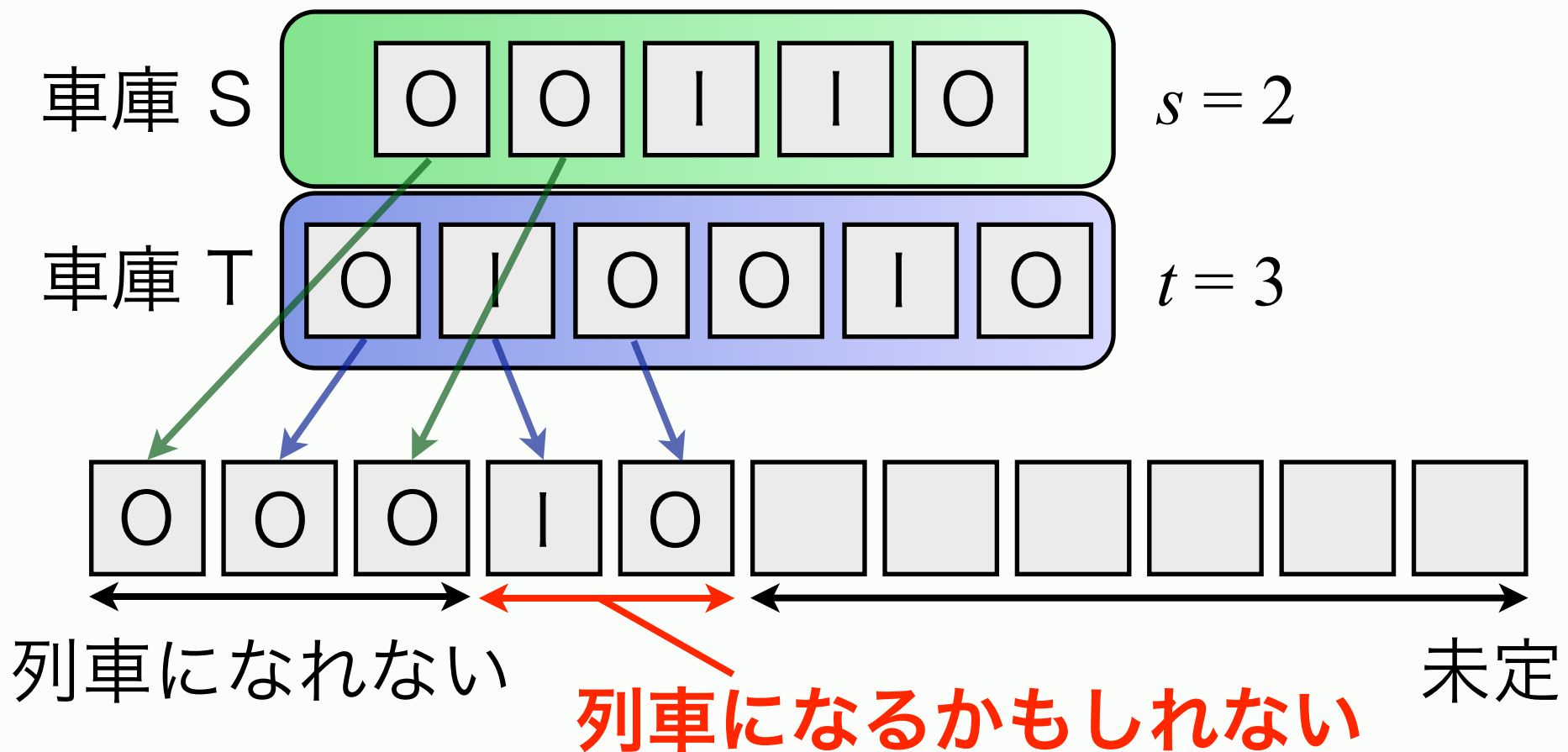
Take the
'IOI' train

- これも立派な状態のとり方だが……？
 - 状態数が**多すぎる**！
 - X は結局列の総数ぐらいのパターンがある
 - 全探索するのと変わらない！
- X から無駄を省けないか？

状態のとり方

Take the
'IOI' train

- X の末尾の列車になりうる部分に注目



状態削減

Take the
'IOI' train

- 列車になれない部分はどうでもいい
- 今後の選択や解に影響を与えない
 - こういう部分は状態から削減してしまって構わない
- X をそのまま状態とする必要はない！
- 「 X の末尾の列車になりうる部分の長さ (L)」
さえ分かっているだけでOK！！！！
 - IOIO... という形の列であることは分かっているから

動的計画法の立式

Take the
'IOI' train

- 状態のとり方を決めた
- ある状態から次の状態を計算する式を立てる
- 初期状態は $s = t = L = 0$
 - 車庫 S から T からも 1 つも車両を出していない
 - もちろん列車になれる部分の長さも 0

動的計画法の立式

Take the
'OO' train

- $dp(s, t, L) :=$
 - 初期状態から開始したときに
 - 車庫 S からは s 両
 - 車庫 T からは t 両の車両を出していて
 - 並べた列の列車になれる部分の長さが L
 - という状態に到達できるかどうか(true / false)
- もちろん $dp(0, 0, 0) = \text{true}$

動的計画法の立式

Take the
'IOI' train

- $dp(s, t, L) = \text{true}$ のとき次の1手は？
 - $s < M$ なら, 車庫 S から次の車両を出せる
 - $t < N$ なら, 車庫 T から次の車両を出せる
- L がどう変化するかをしっかり考える
 - L が奇数 $\rightarrow$ 次に車両 O が来ると L が 1 増える
車両 I が来ると L が 1 になる
 - L が偶数 $\rightarrow$ 次に車両 I が来ると L が 1 増える
車両 O が来ると L が 0 になる

動的計画法の立式(まとめ)

Take the
'IOI' train

$dp(s, t, L) = \text{true}$ であれば

$s < M$ なら車庫 S の $s + 1$ 個目の車両を出す

その結果 L の値が L' に変わったとすると

$dp(\textcolor{red}{s} + 1, t, \textcolor{red}{L}') \leftarrow \text{true}$

[t についても同じことをやる]

s, t, L について小さい方からループを回して
上の処理をやる

計算量

Take the
'IO' train

- 状態数と, 次の状態への計算量を考えよう
 - s, t はそれぞれ M, N 通り
 - L は $s + t \leq M + N$ 通り
 - 次の状態への計算は $O(1)$ でできる
- 状態数は $O(MN(M+N))$, 遷移は $O(1)$
- 全部で $O(MN(M+N))$
 - 50点分のテストデータが処理できる
 - 100点分はまだ時間制限が厳しい

100点解法(・`д・´)

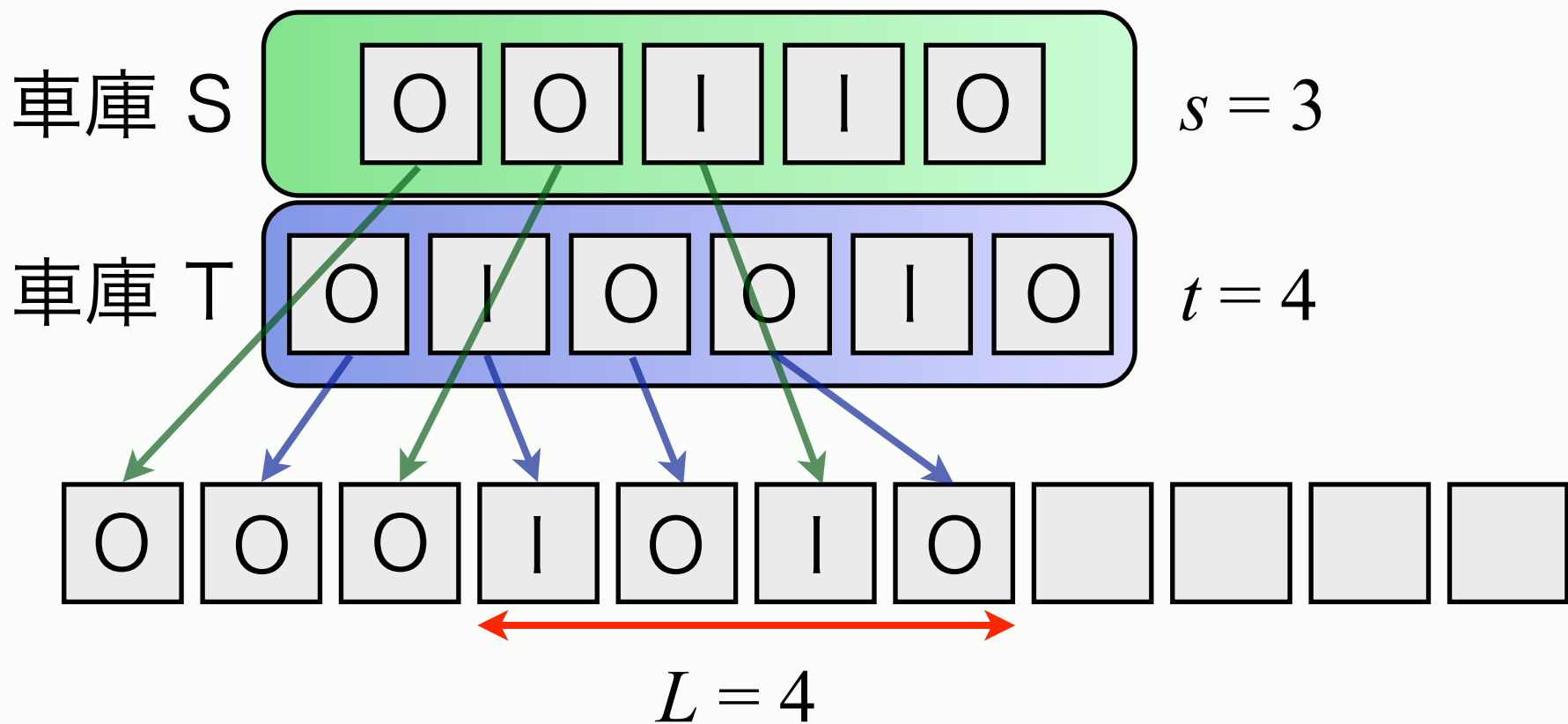
まだ無駄が？

Take the
'IOI' train

- 50点解法のDPにはまだ無駄がある？
- 次を示すふたつの状態を見てみよう
 - 列車になれる部分の長さに注目

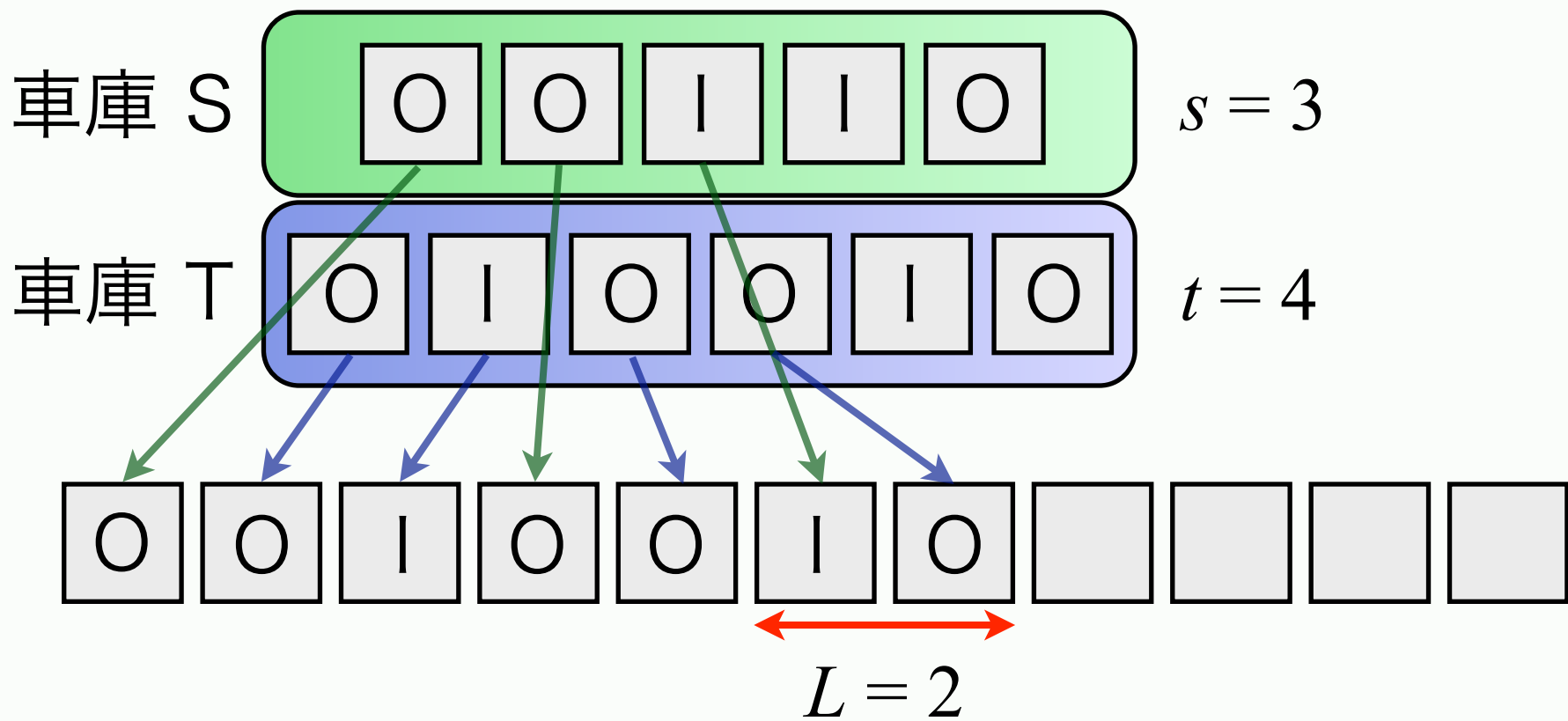
ある状態A

Take the
'OI' train



別の状態B

Take the
'OI' train



無駄はどこか？

Take the
'IOI' train

- 状態 A, B を比べよう
 - s, t の値は両方とも同じ
 - L の値が違う！ (ただし偶奇は同じ)
- s, t が同じで L の偶奇も同じなら
 L は大きいほうがいいに決まっている！
- この無駄を省いた状態のとり方を考えよう

効率的な状態のとり方

Take the
'IO' train

- $dp(s, t, p) :=$
 - 車庫 S からは s 両
 - 車庫 T からは t 両の車両を出していて
 - 列車になれる部分の末尾の車両が p (I / O) のとき
 - 列車になれる部分の長さ L の最大値

動的計画法の立式

Take the
'O!' train

- $dp(s+1, t+1, I)$ を計算するときは
 - S の $s+1$ 番目の車両が I なら $dp(s, t+1, O)+1$ を考慮
 - T の $t+1$ 番目の車両が I なら $dp(s+1, t, O)+1$ を考慮
- して, 考慮した値のうち大きい方を採用する

$res \leftarrow 0$

if $S[s+1] = I$ then $res \leftarrow \max\{res, dp(s, t+1, O)+1\}$

if $T[t+1] = I$ then $res \leftarrow \max\{res, dp(s+1, t, O)+1\}$

$dp(s+1, t+1, I) \leftarrow res$

計算量

Take the
'IO' train

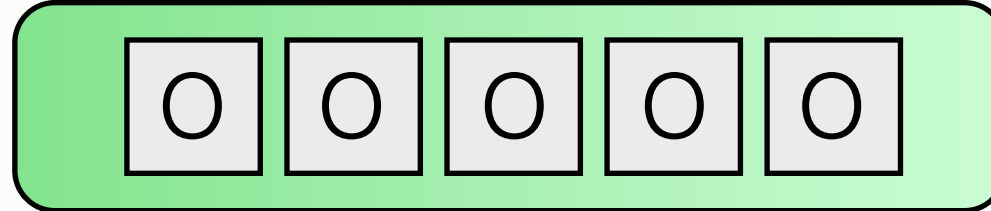
- 状態からは L が削減, その偶奇だけ考える
- よって状態数 $O(MN)$
- 遷移は変わらず $O(1)$
- 合わせて全体で $O(MN)$
- $M, N \leq 2,000$ でも間に合う!!!
- ようやく 100 点
 - おつかれさまでした

コーナーケース

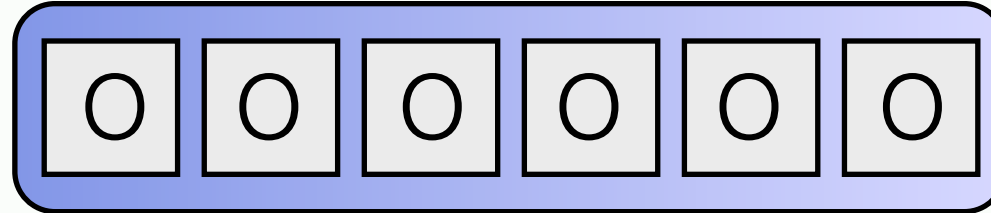
Take the
'0' train

- 鉄道事業を成功させる気がない
 - 問題文にはちゃんと書いてあります
 - 「列車が 1 つも編成できない場合は, 0 を出力せよ. 」

車庫 S

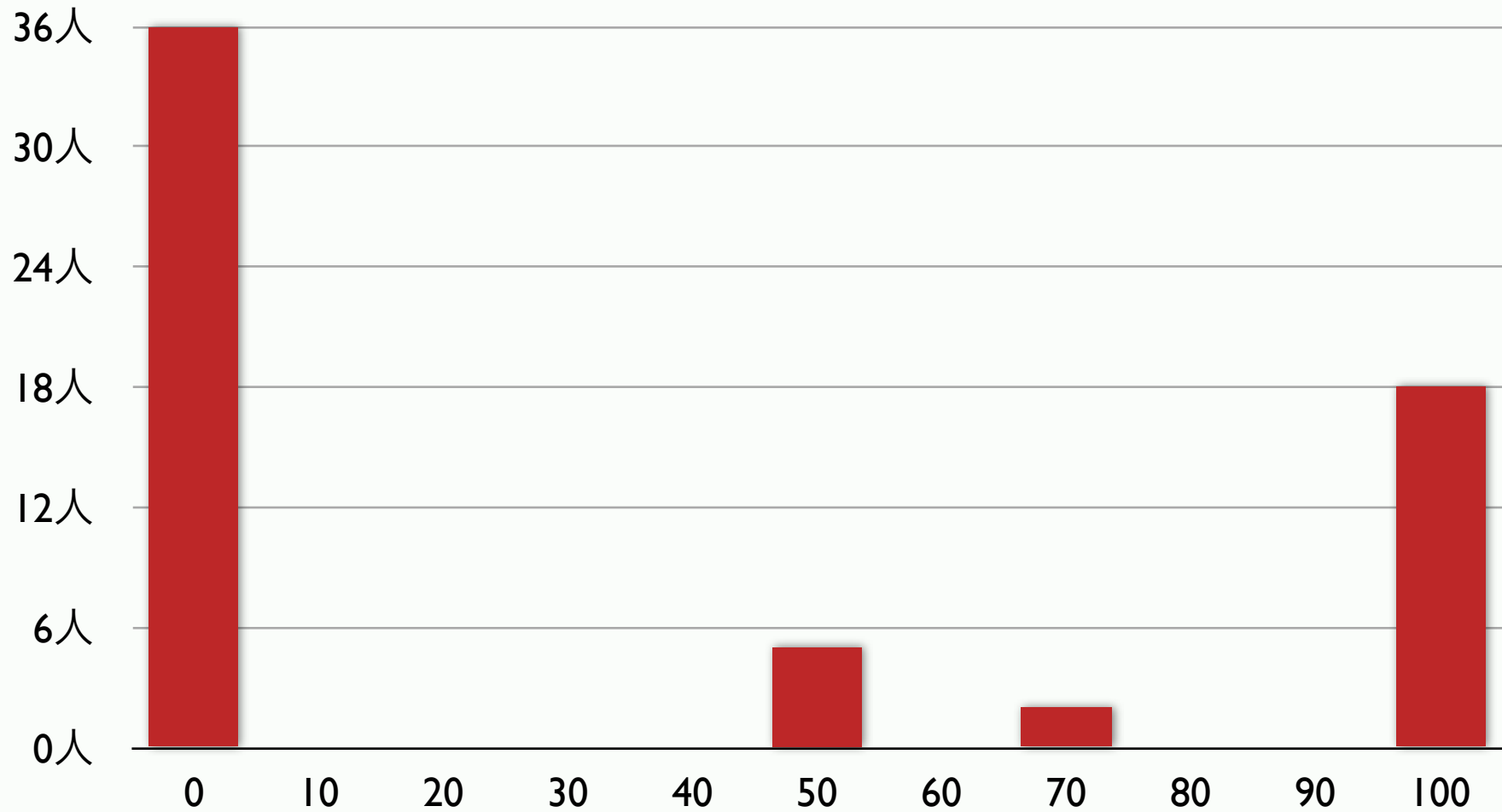


車庫 T



得点分布

Take the
'IOI' train



ちなみに

配るDP, 貰うDP

Take the
'IOI' train

- 50点解法で使った方法を**配るDP**
- 100点解法で使った方法を**貰うDP**
 - ということもあります
 - この問題ではどちらの方が効率がよいということはなく, 単に実装方法の違いです
 - 基本的には自分の書きやすい方でよいでしょう(もちろんどっちも書けたほうがいいです)

Further Reading

Take the
'IOI' train

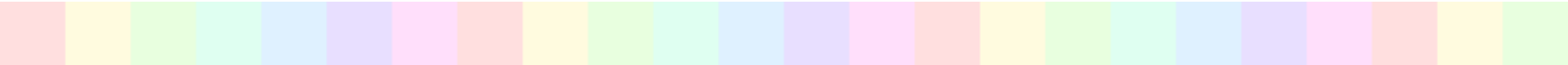
- プログラミングコンテストでの動的計画法
 - <http://www.slideshare.net/iwiwi/ss-3578511>



動的計画法 iwiwi

Google 検索

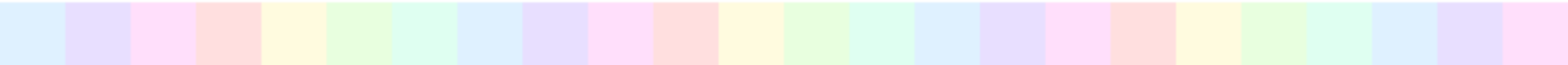
I'm Feeling Lucky

A horizontal bar at the top of the slide consisting of a sequence of colored squares in shades of red, yellow, green, blue, and purple.

現代的な屋敷 (Modern Mansion)

JOI 2013 本選 問題 3

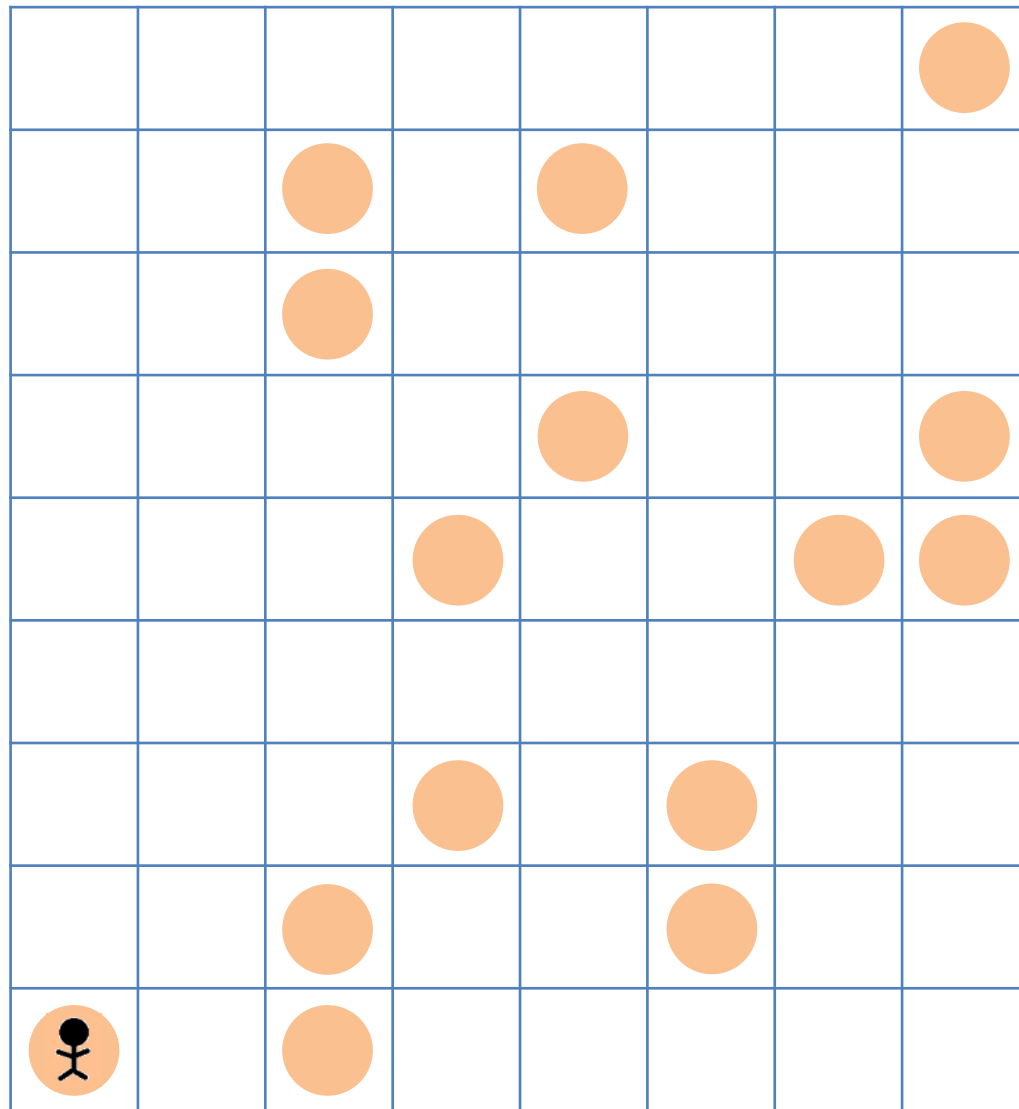
解説： 保坂 和宏

A horizontal bar at the bottom of the slide consisting of a sequence of colored squares in shades of blue, purple, pink, red, yellow, green, and cyan.

問題概要

- 横 $M \times$ 縦 N のマス目の屋敷
 - 縦横 1 マスの移動に 1 分
 - 最初は縦移動のみ可能
- スイッチ：縦移動と横移動が切り替わる
 - K 個のマスにある
 - 押すのに 1 分
- 左下から右上まで行くための最短時間
- $M, N \leq 100,000, K \leq 200,000$

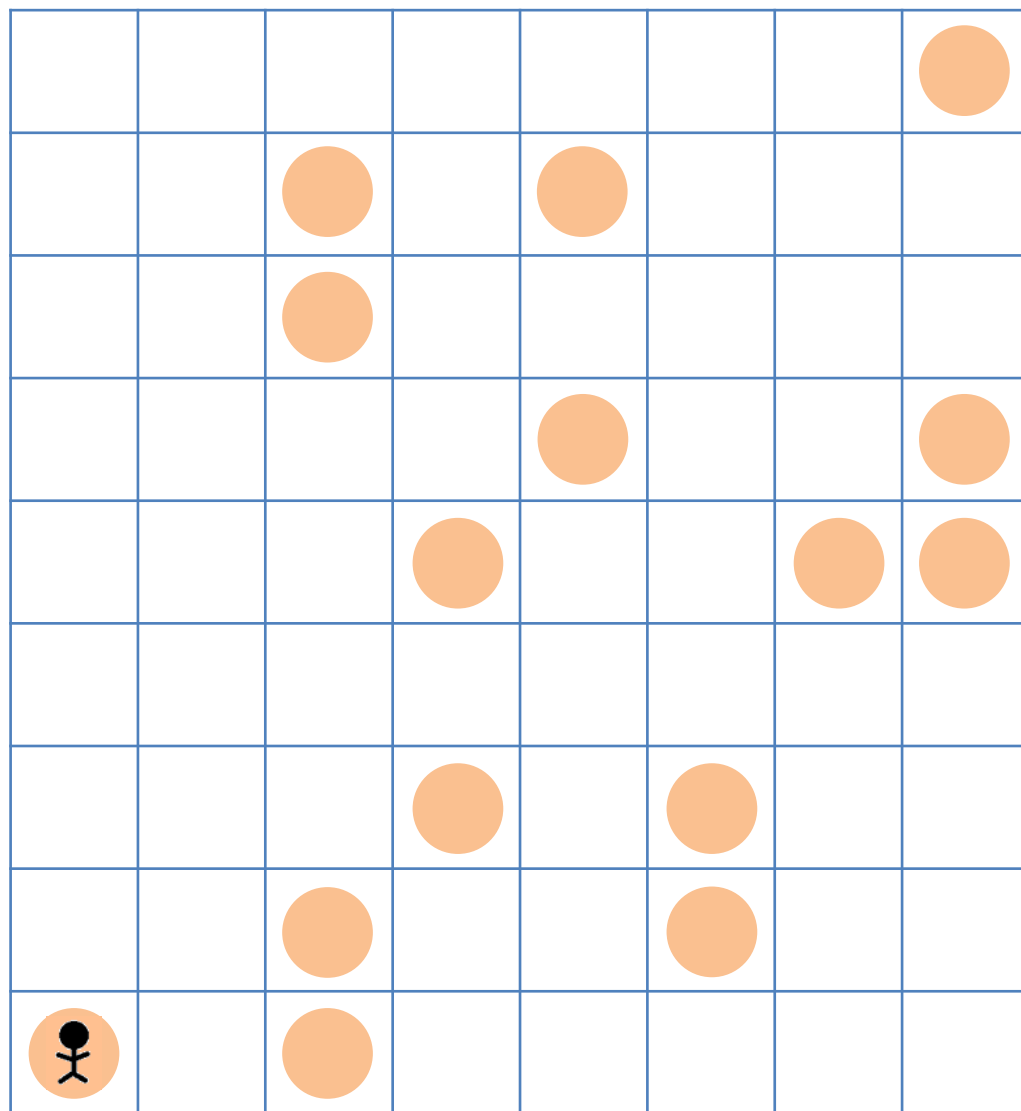
問題概要



移動：縦

時間：0

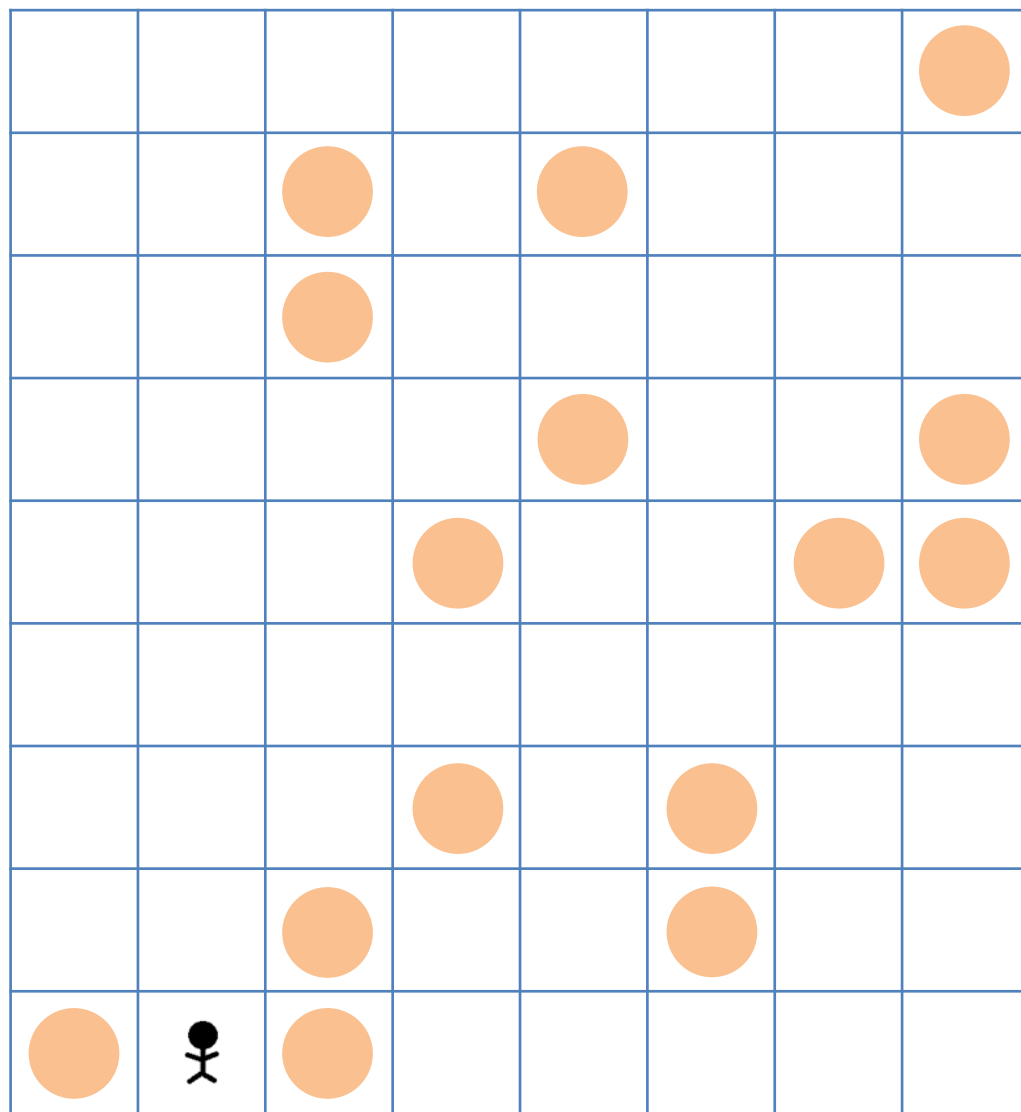
問題概要



移動：横

時間： 1

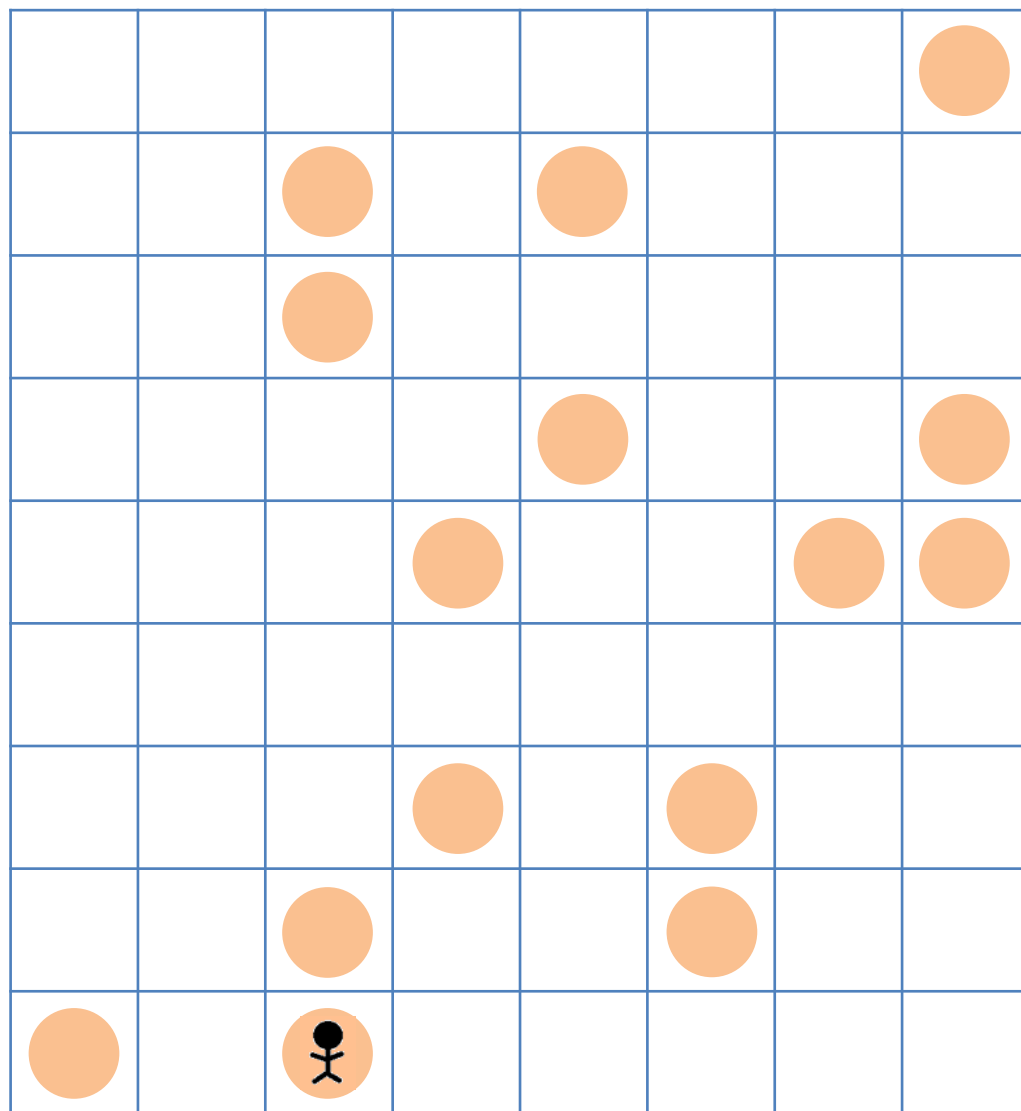
問題概要



移動：横

時間： 2

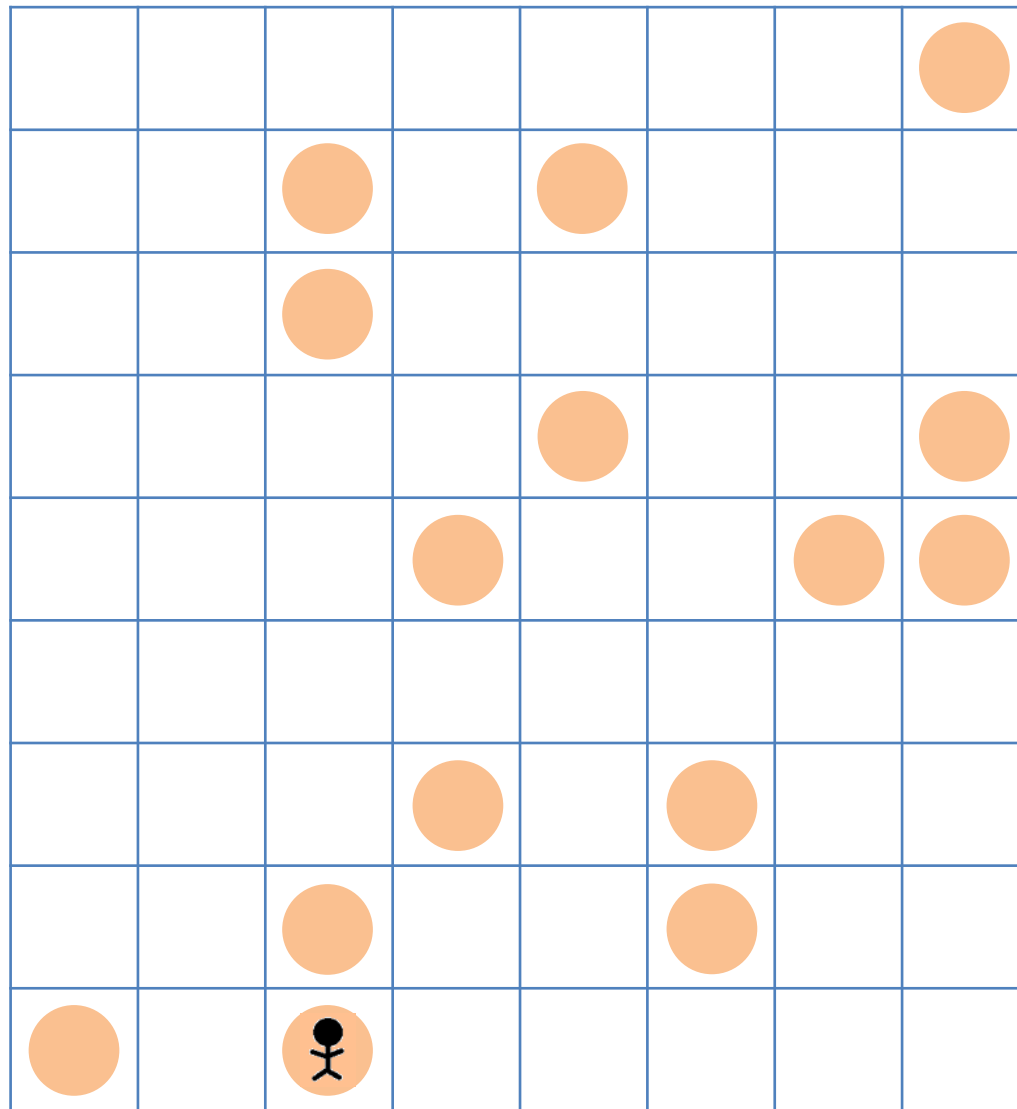
問題概要



移動：横

時間： 3

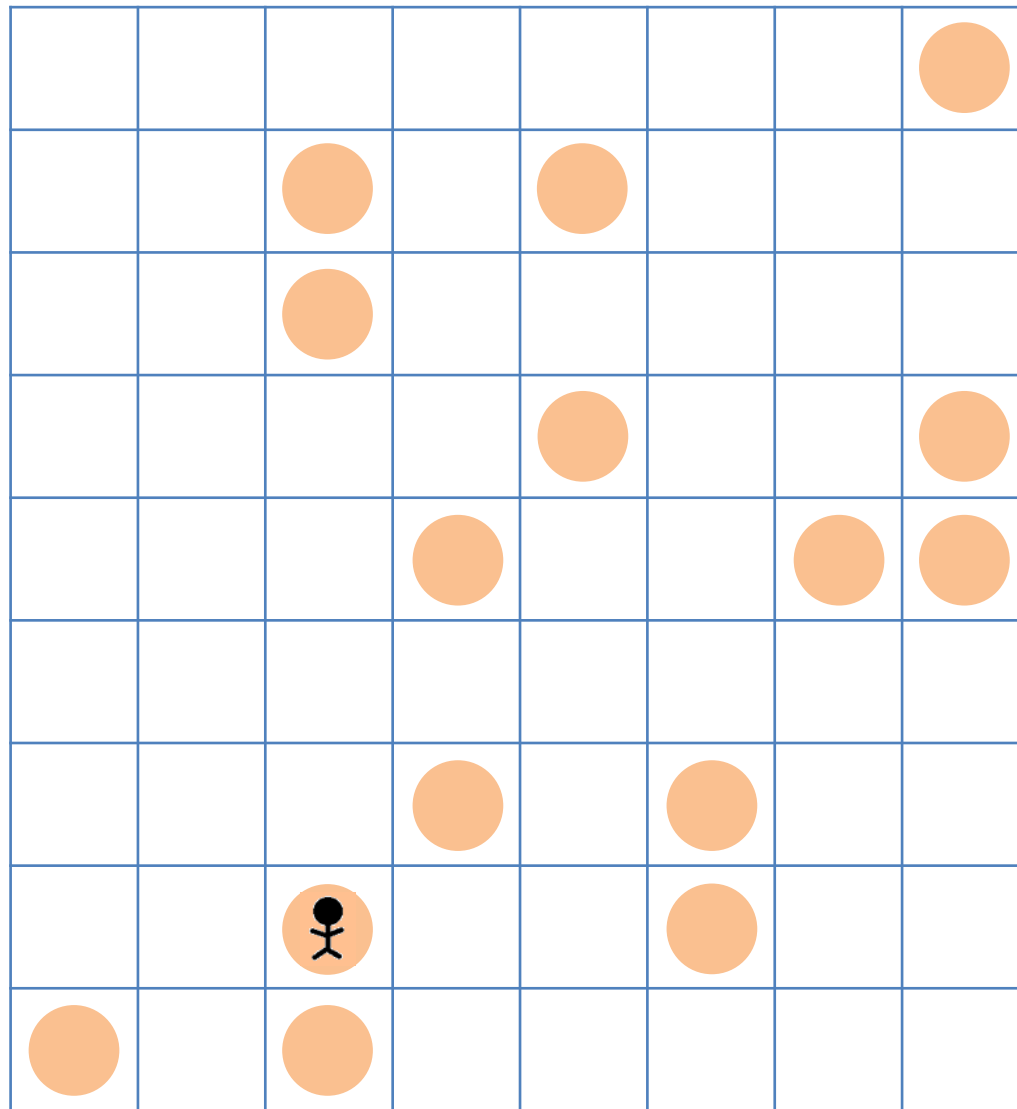
問題概要



移動：縦

時間：4

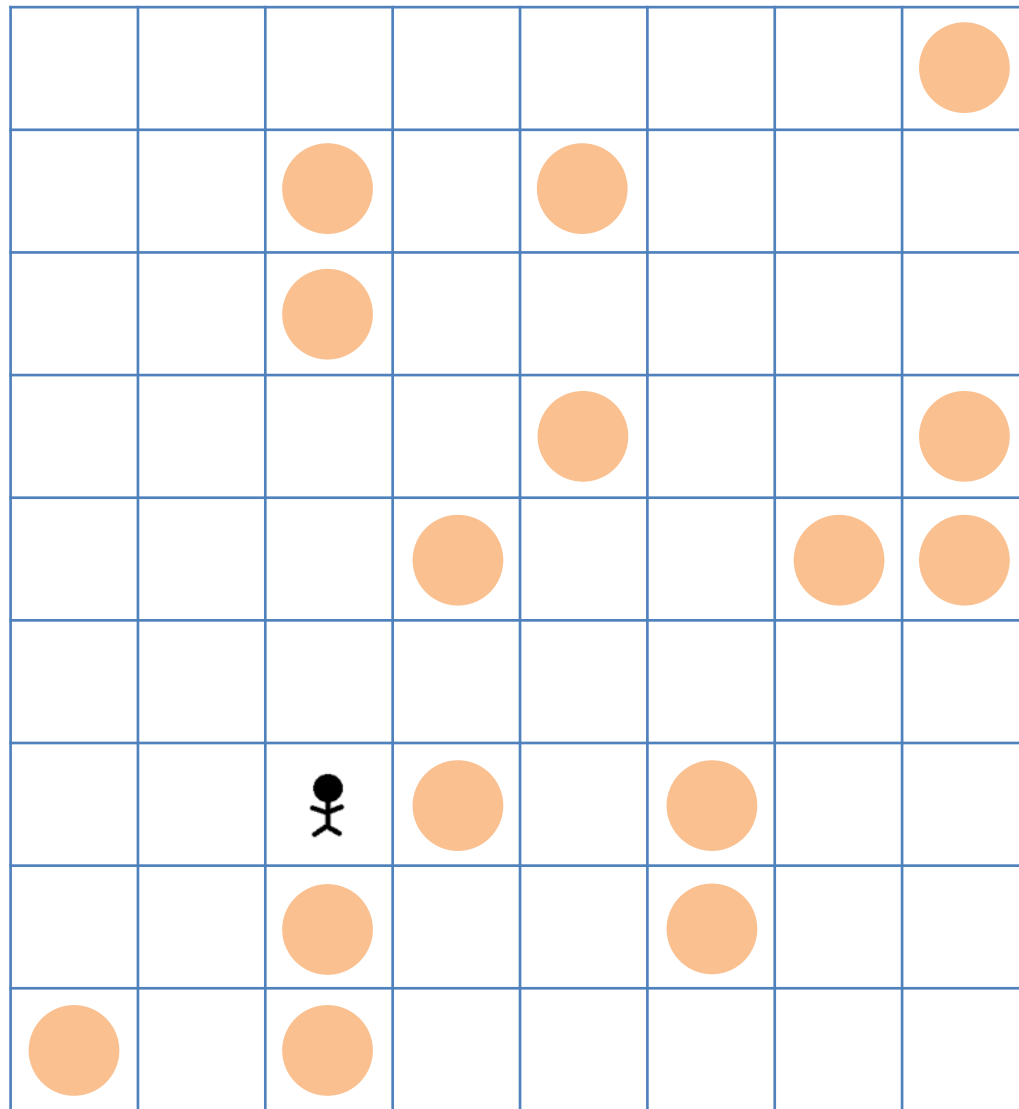
問題概要



移動：縦

時間： 5

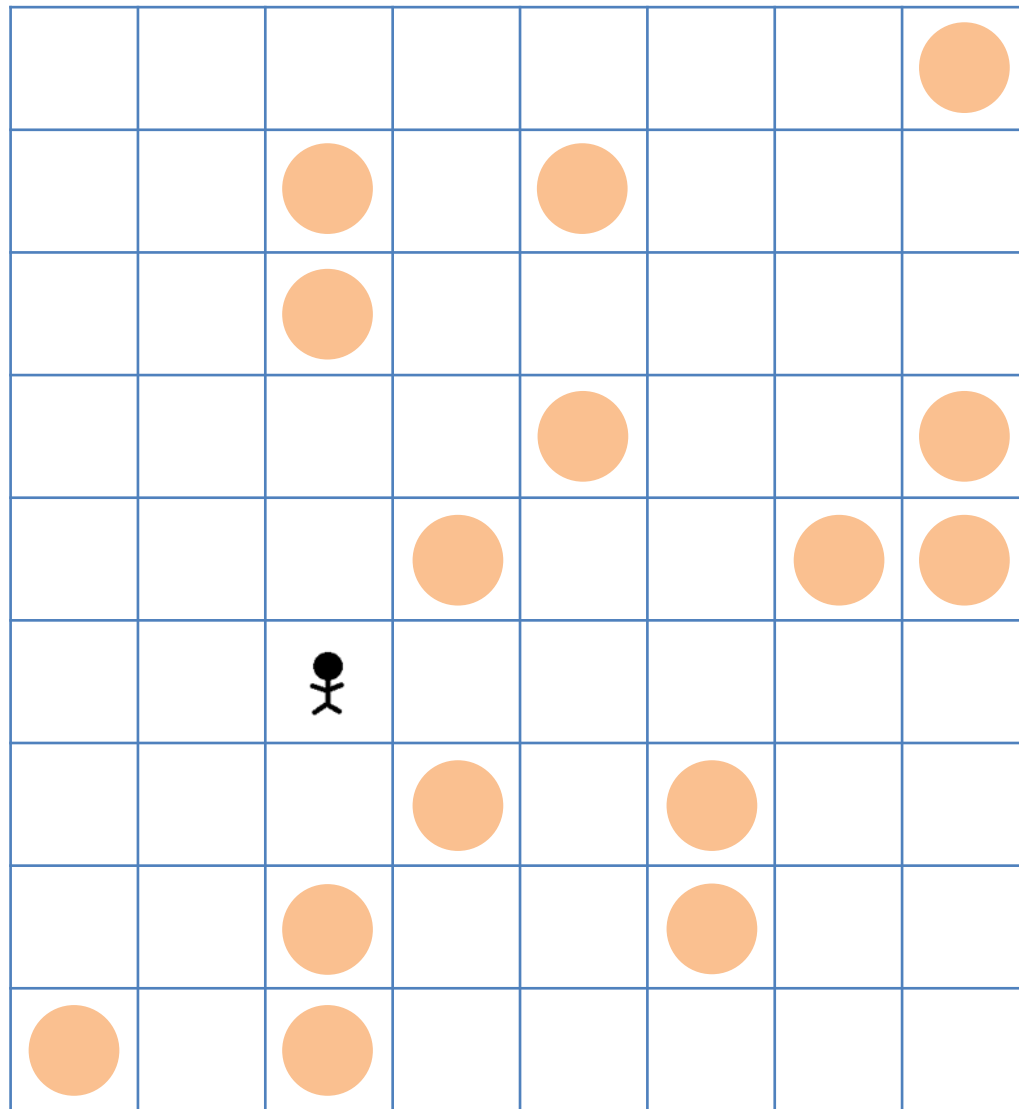
問題概要



移動：縦

時間： 6

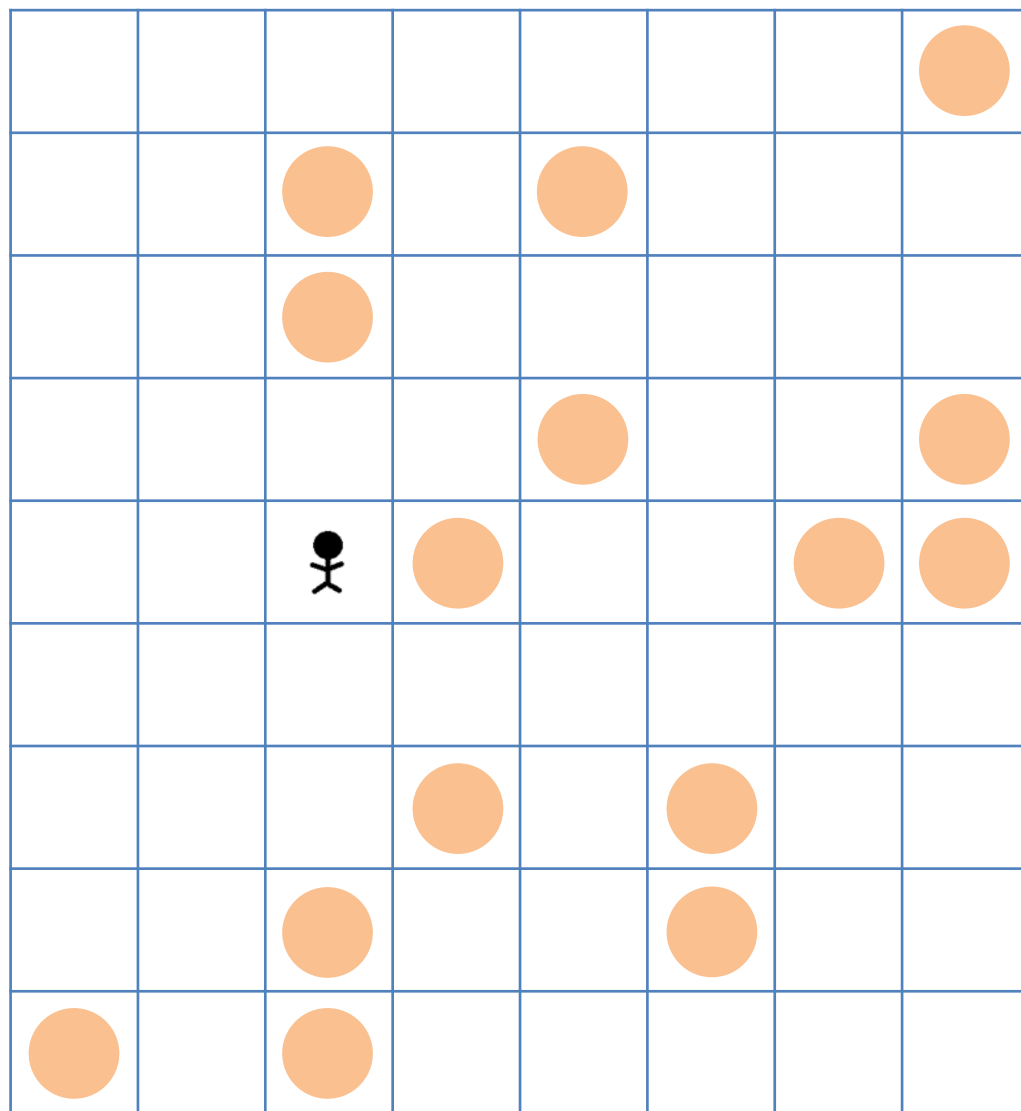
問題概要



移動：縦

時間： 7

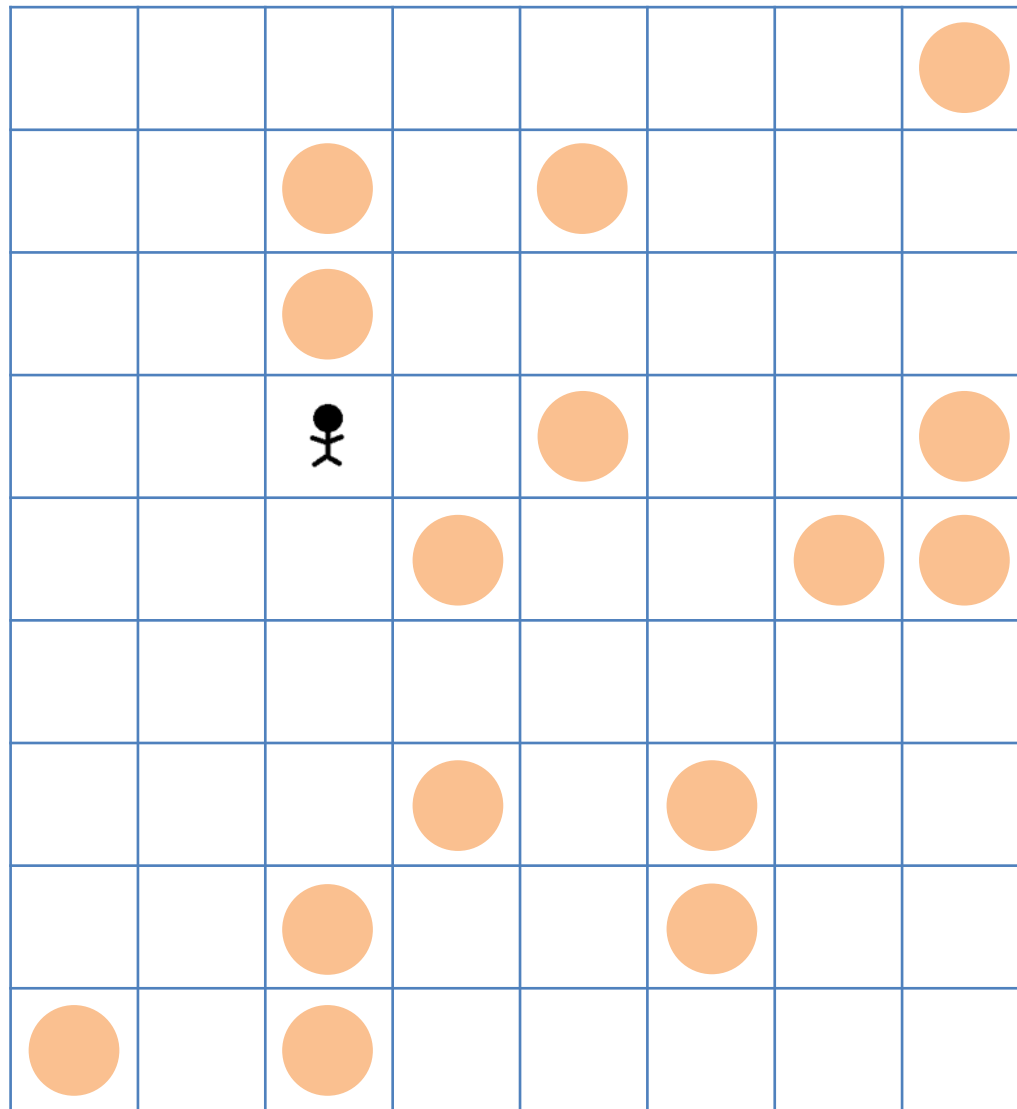
問題概要



移動：縦

時間：8

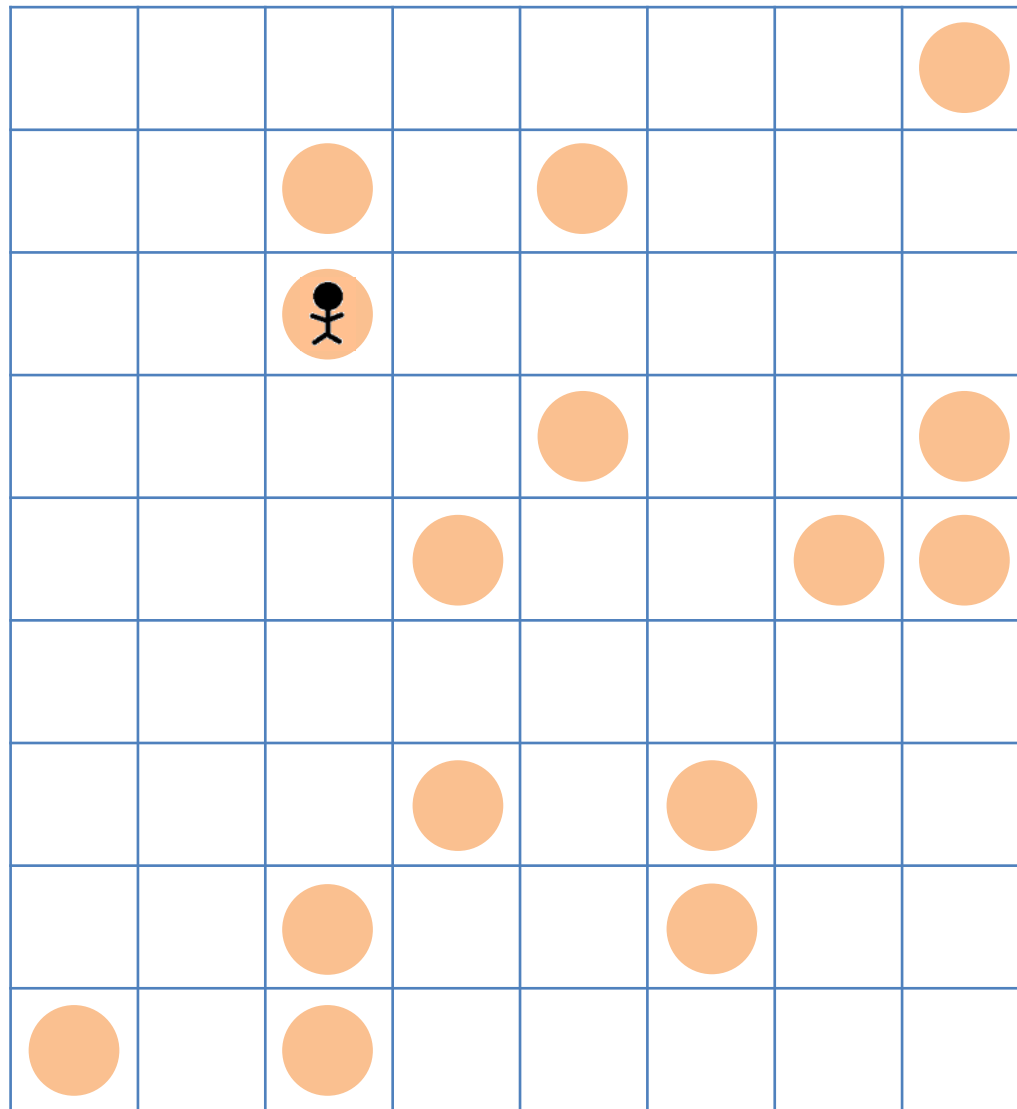
問題概要



移動：縦

時間： 9

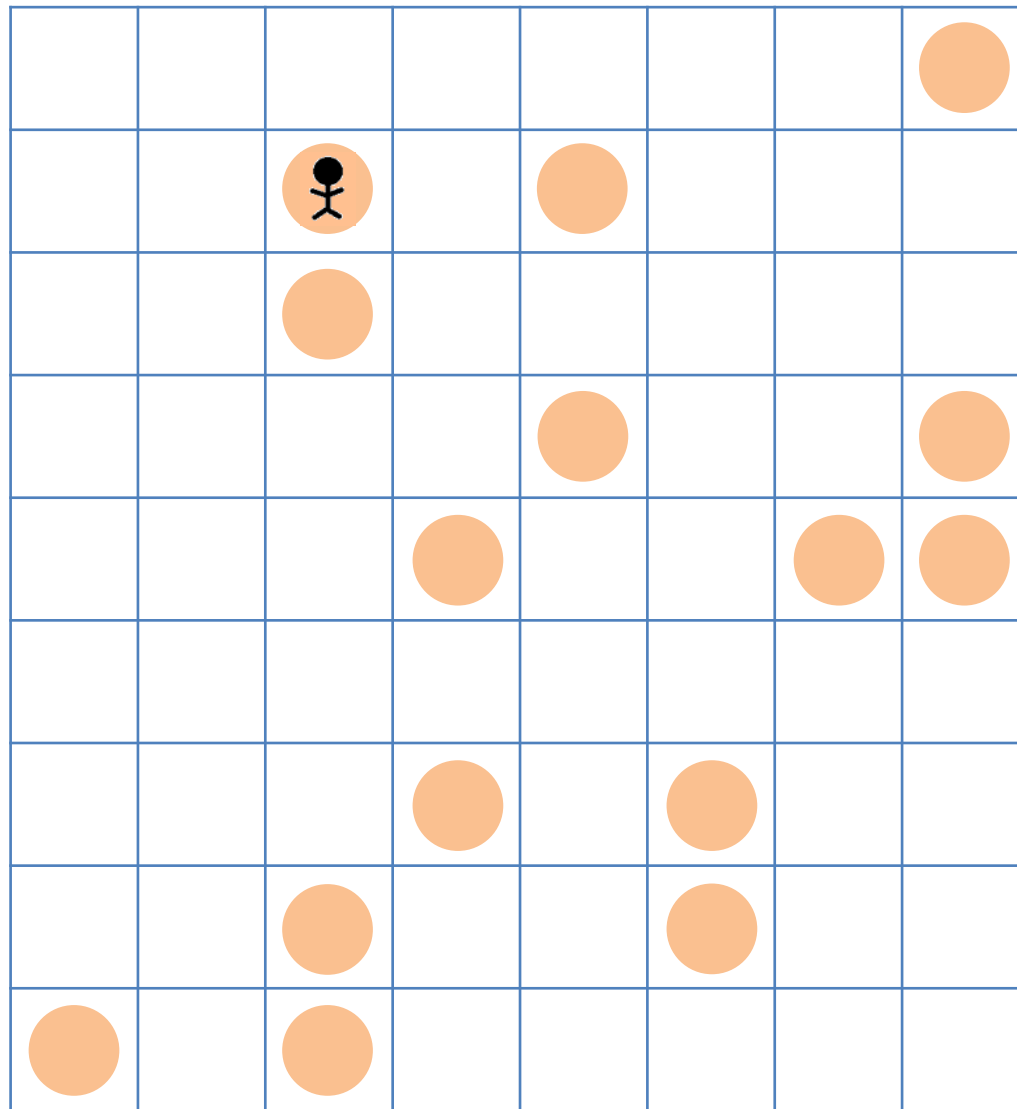
問題概要



移動：縦

時間：10

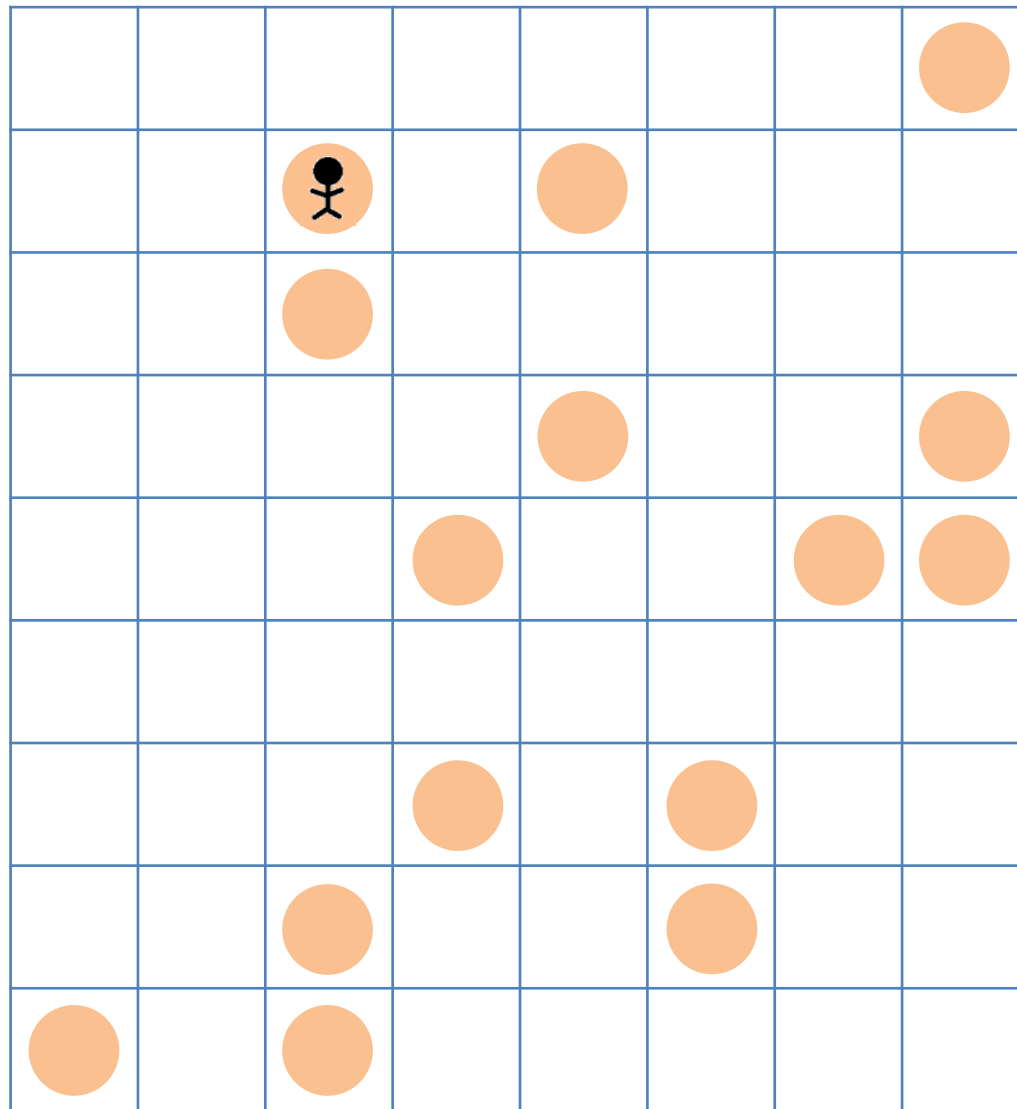
問題概要



移動：縦

時間：11

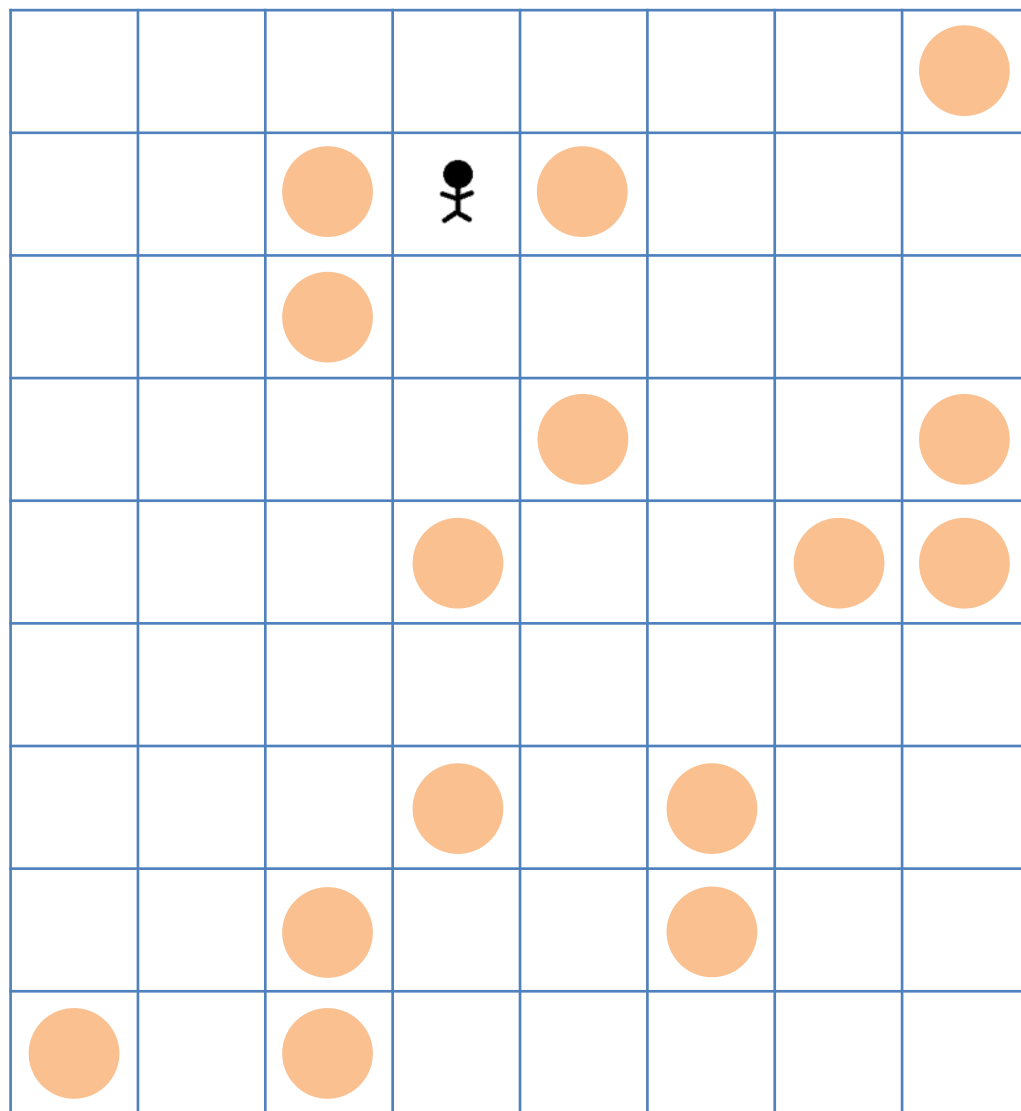
問題概要



移動：横

時間：12

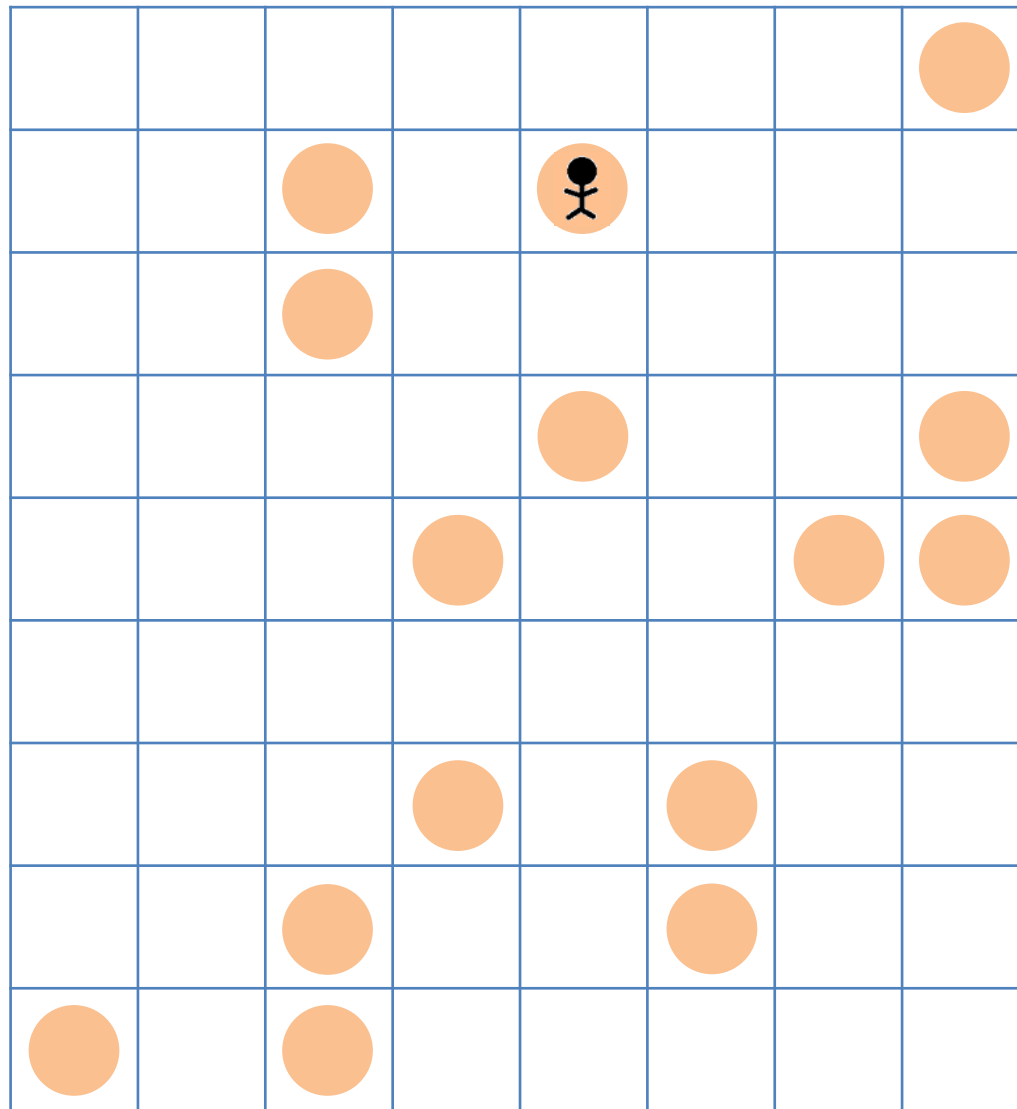
問題概要



移動：横

時間：13

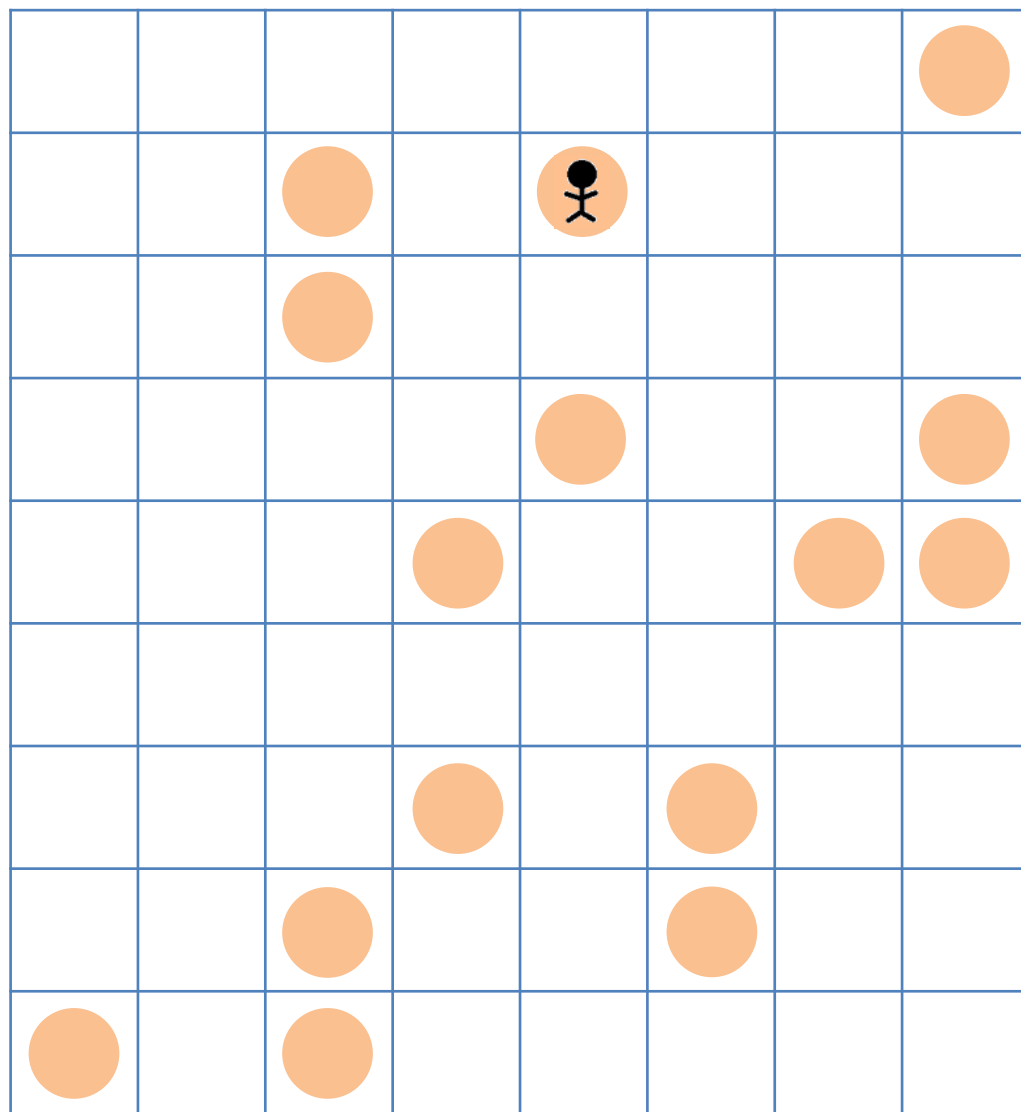
問題概要



移動：横

時間：14

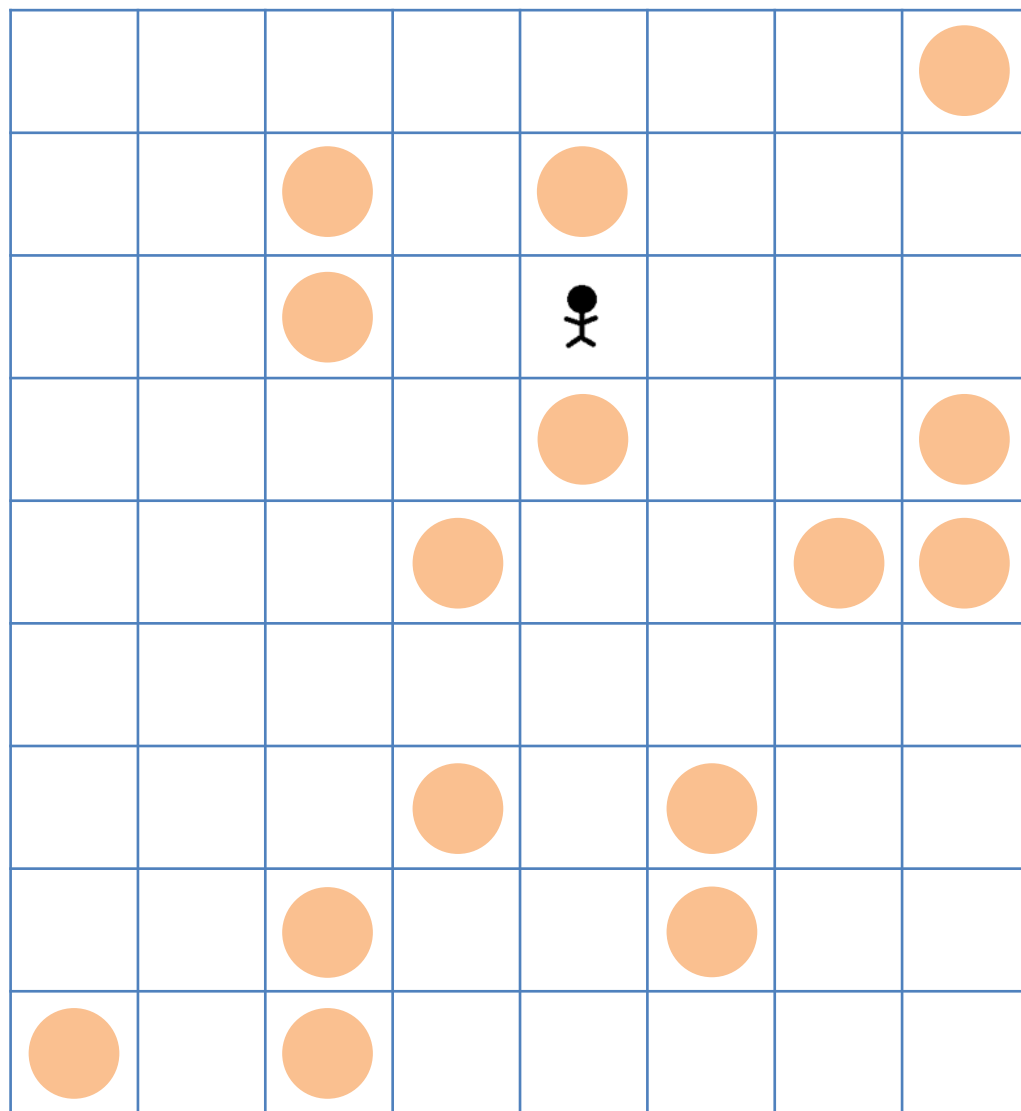
問題概要



移動：縦

時間：15

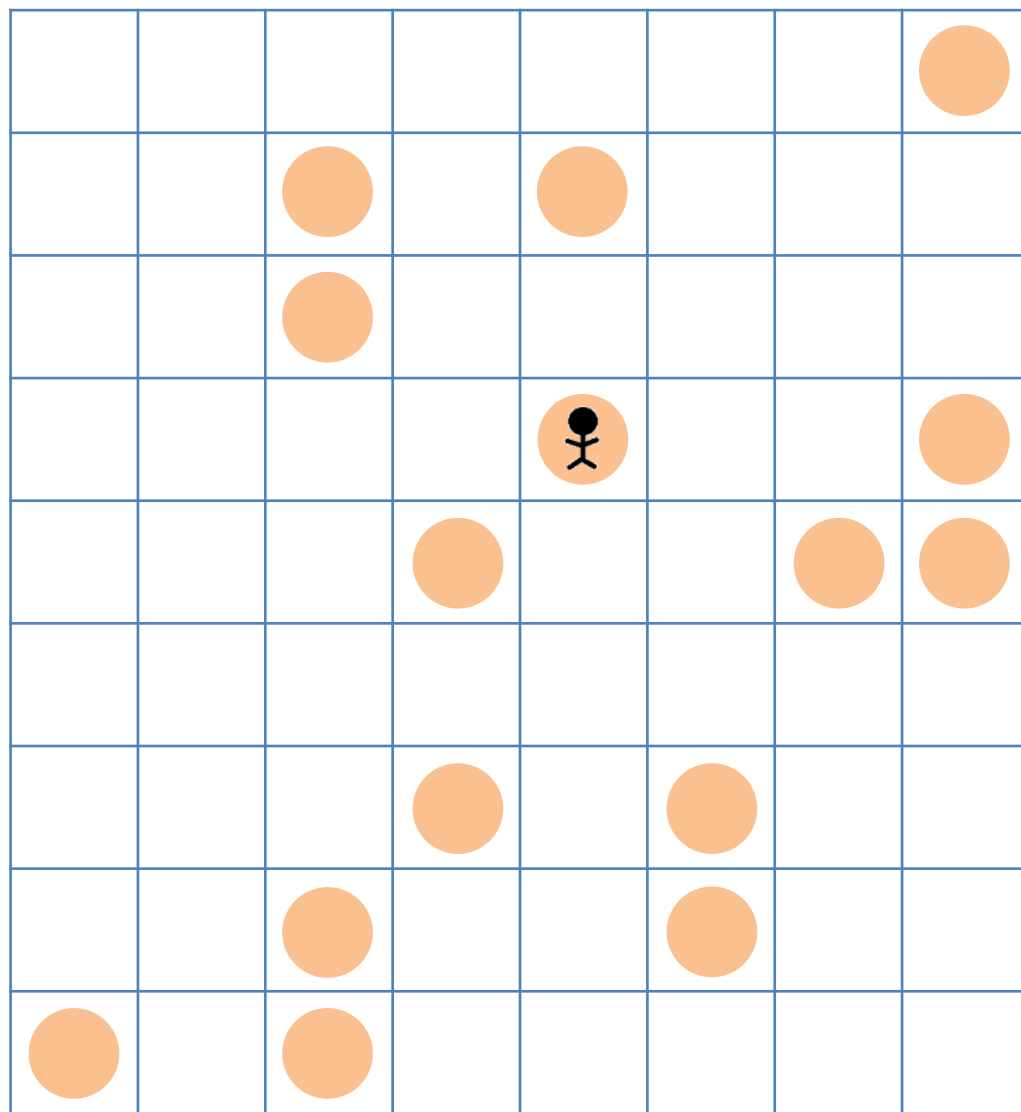
問題概要



移動：縦

時間：16

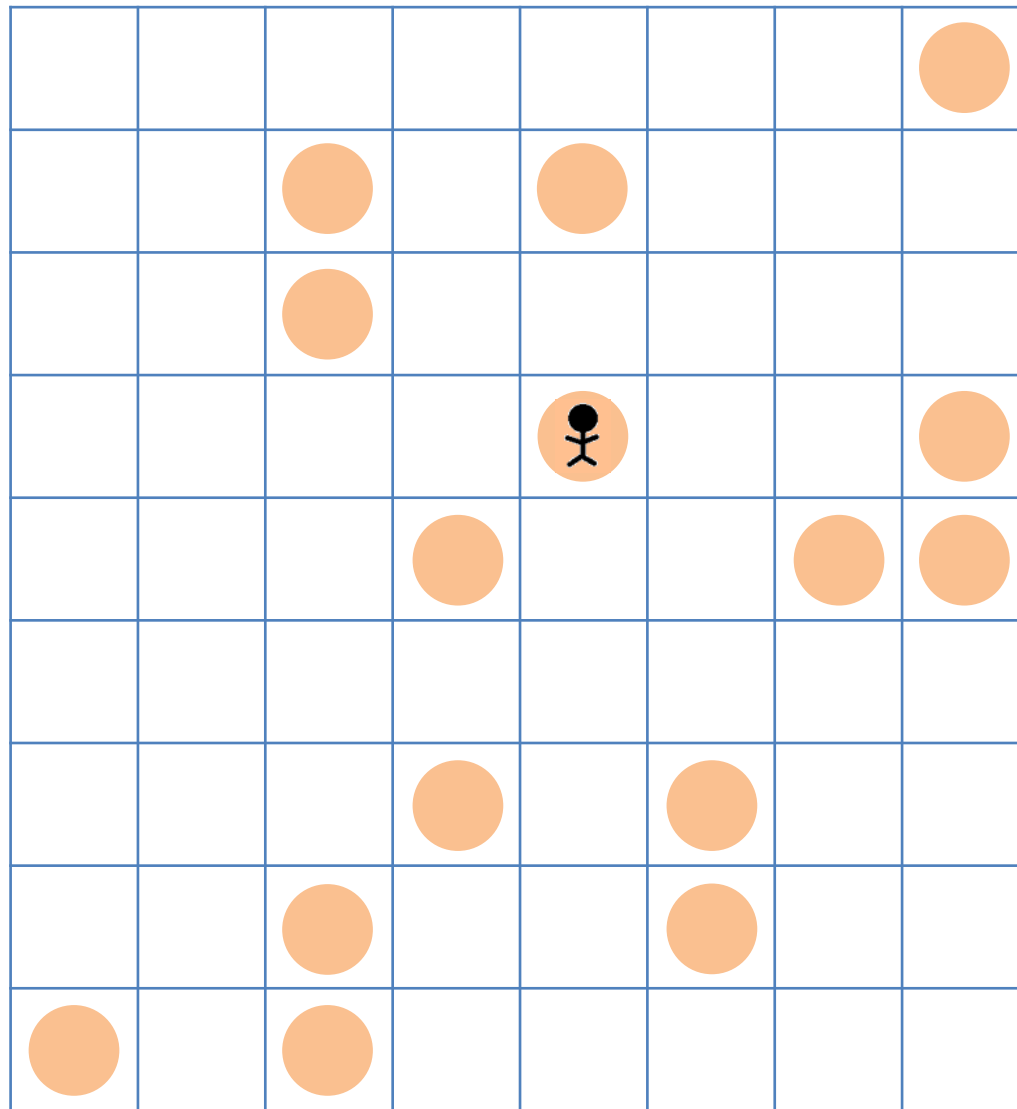
問題概要



移動：縦

時間：17

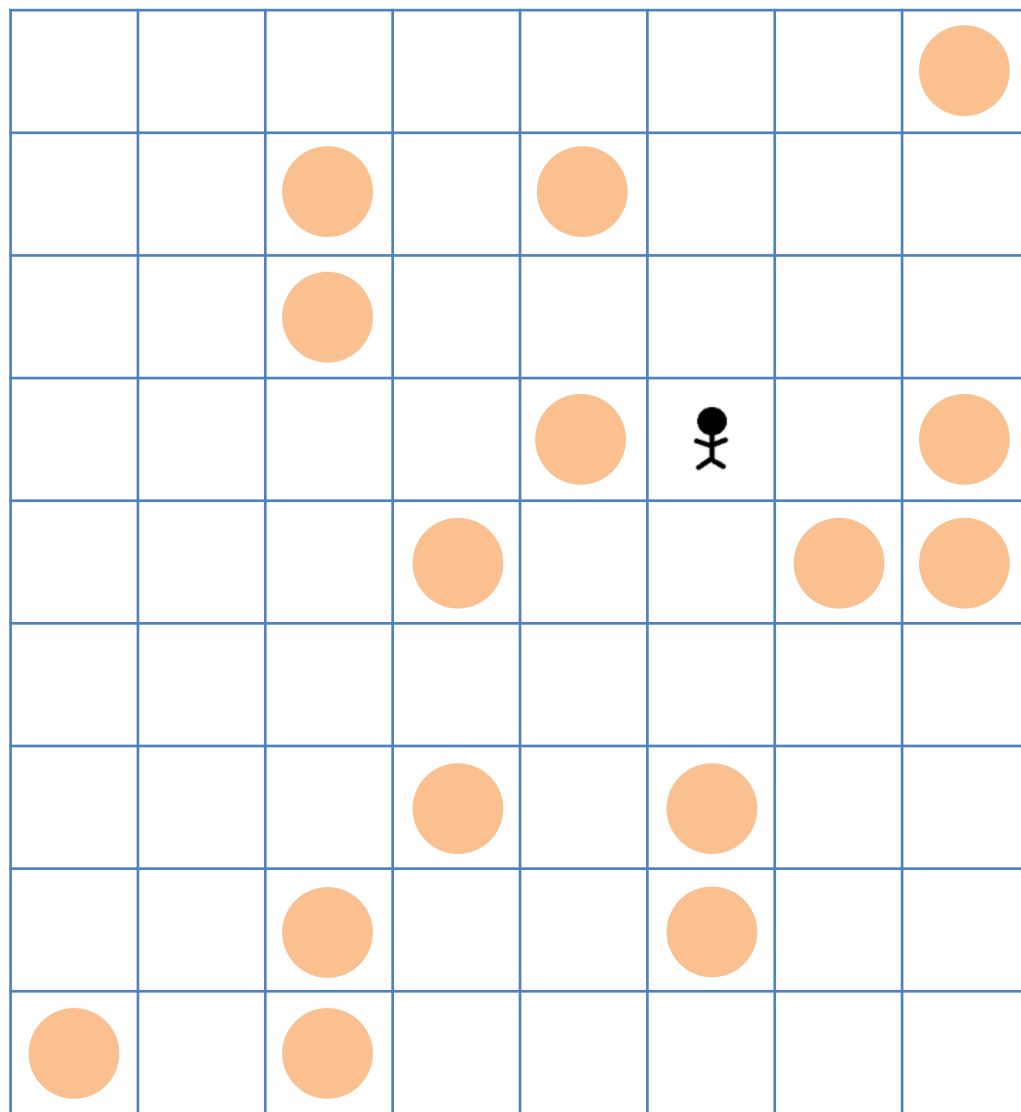
問題概要



移動：横

時間：18

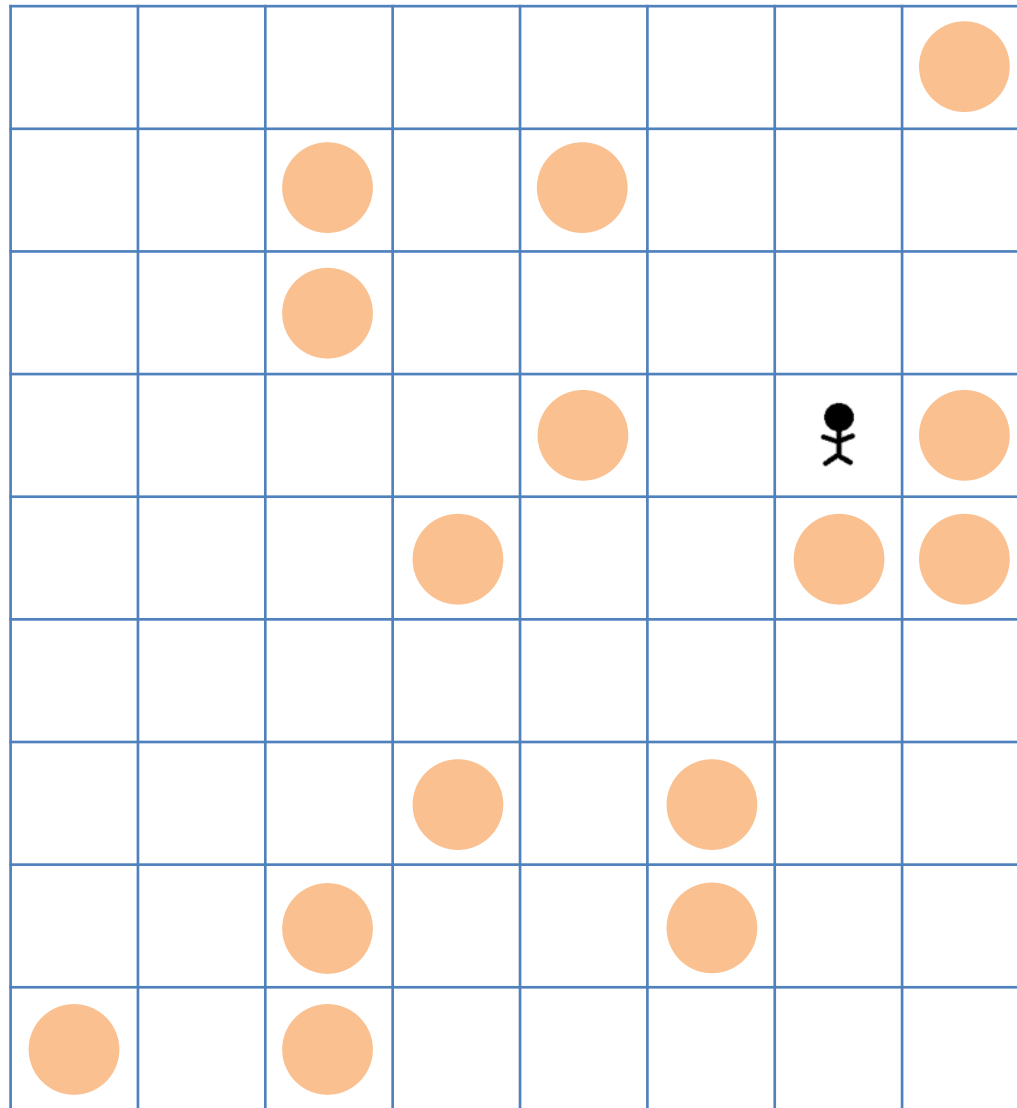
問題概要



移動：横

時間：19

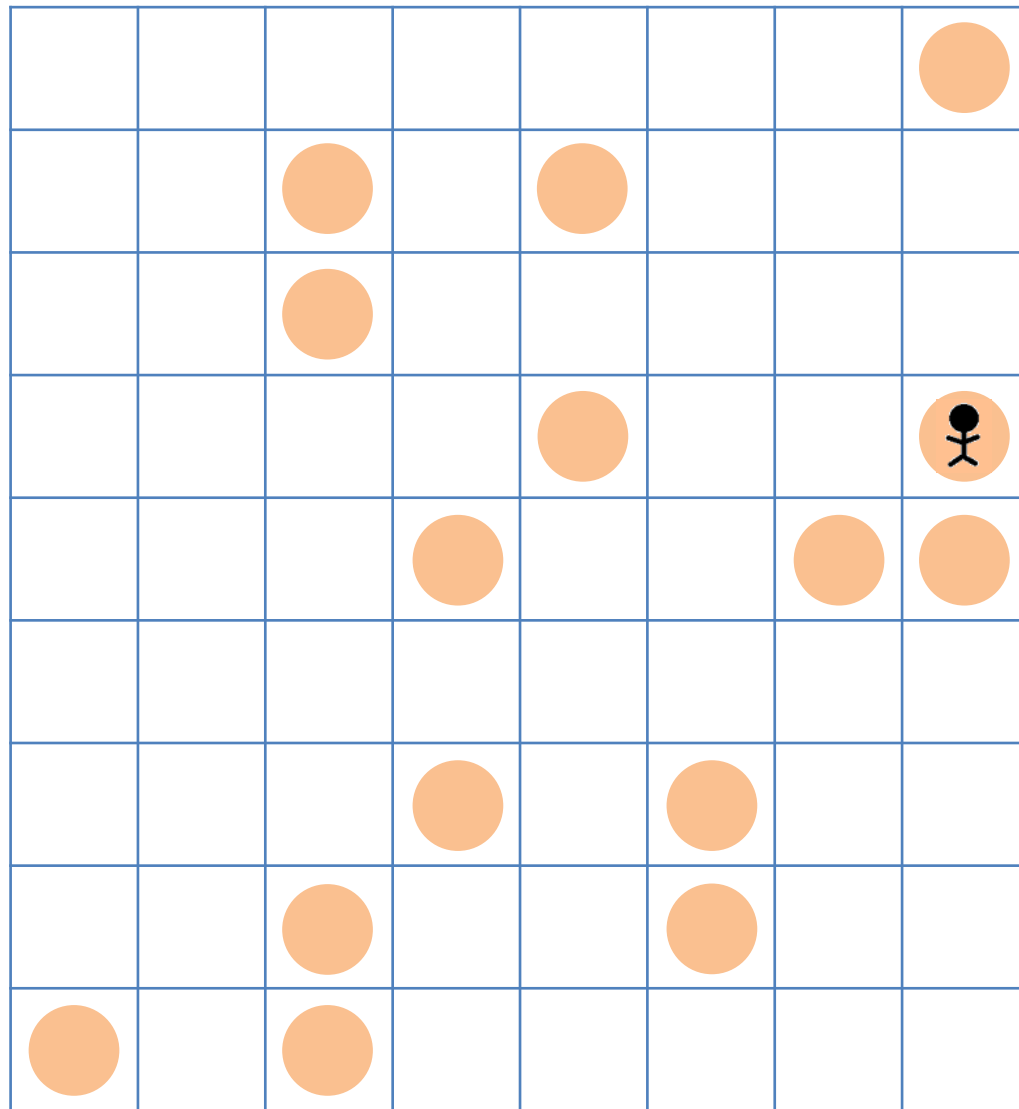
問題概要



移動：横

時間：20

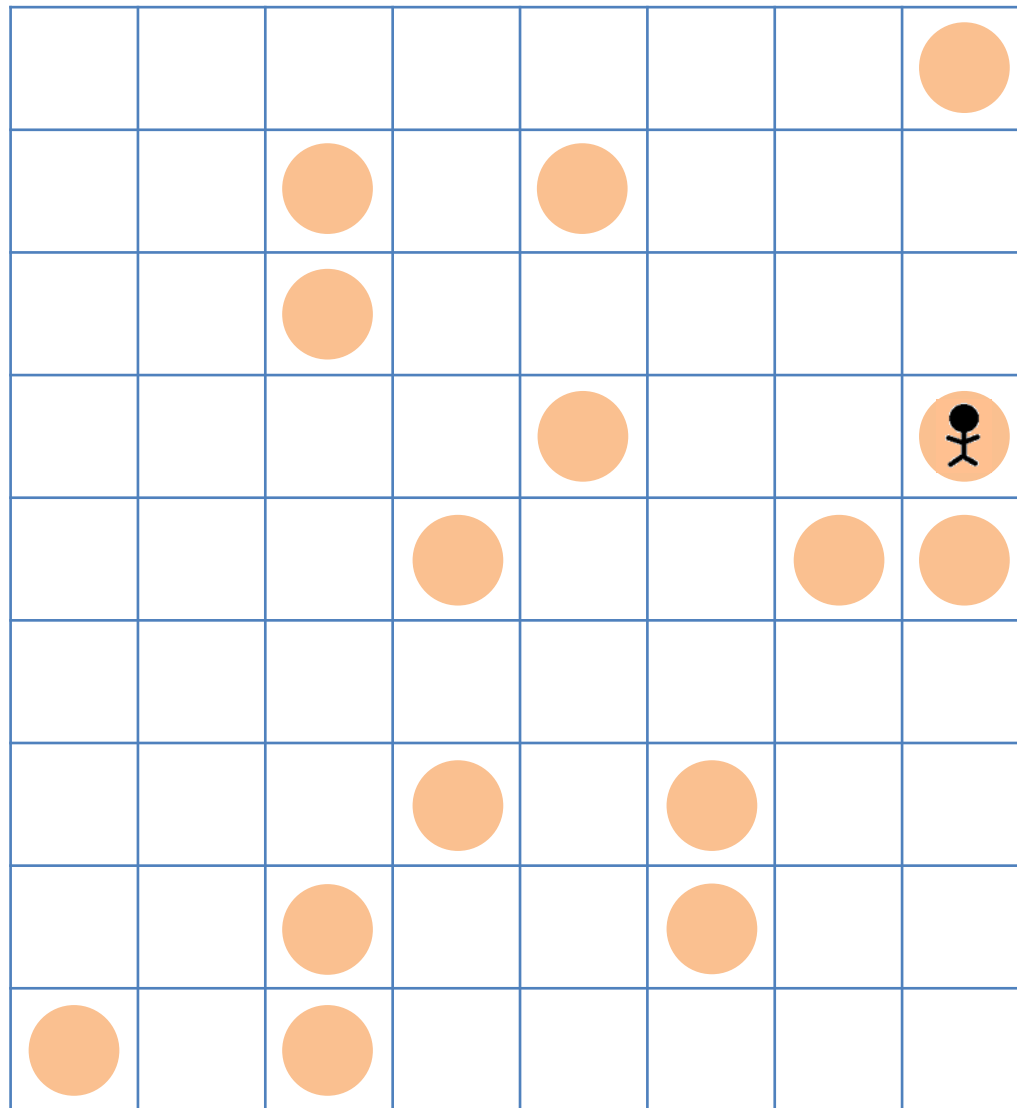
問題概要



移動：横

時間：21

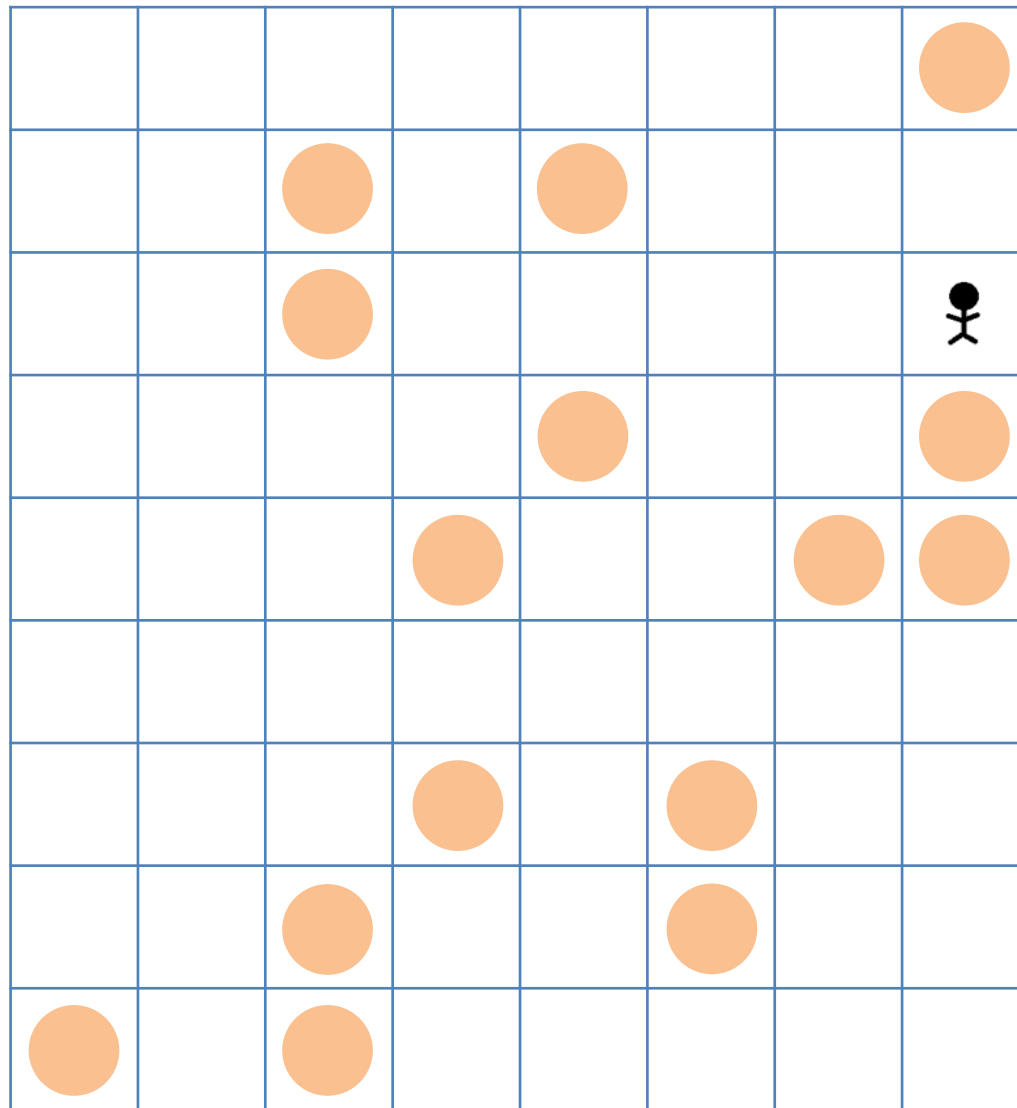
問題概要



移動：縦

時間：22

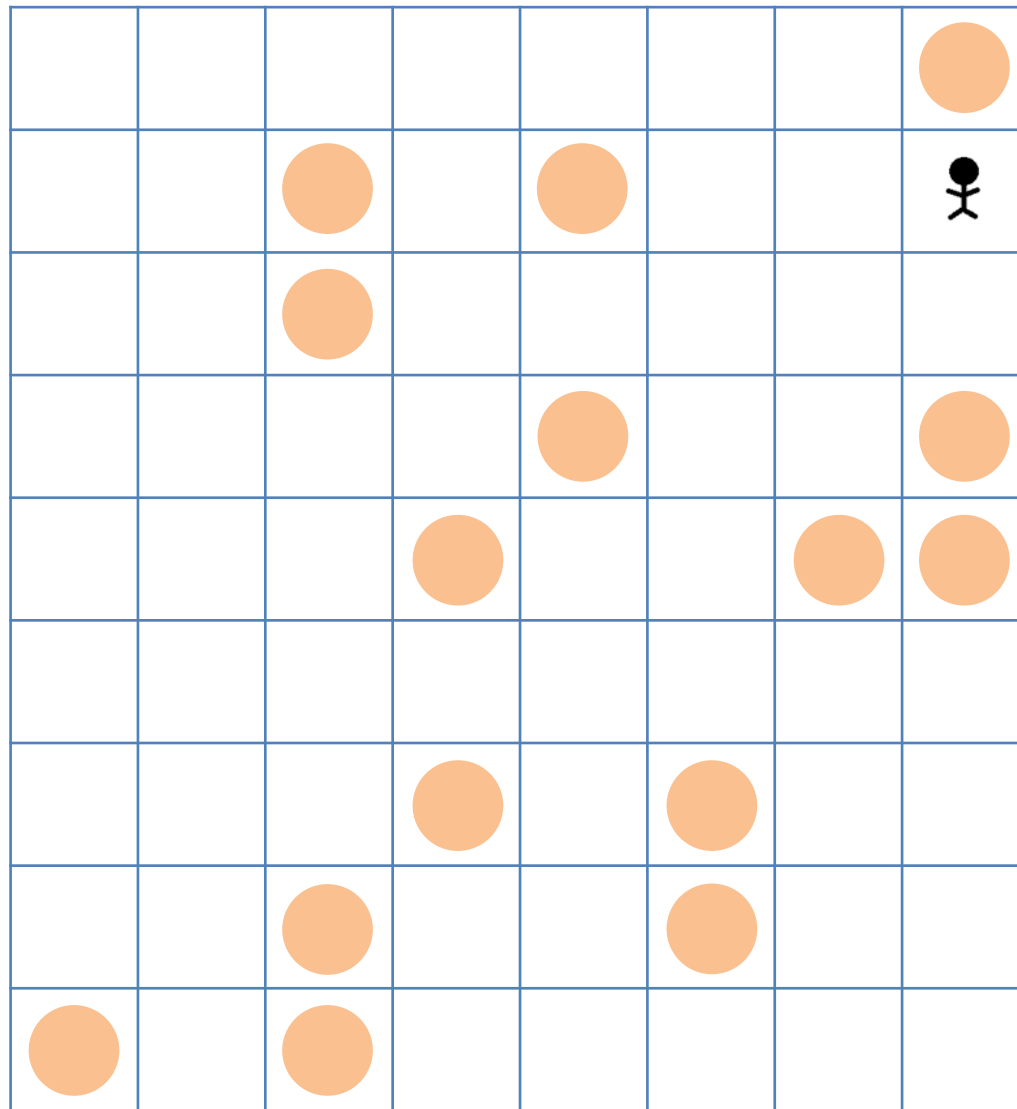
問題概要



移動：縦

時間：23

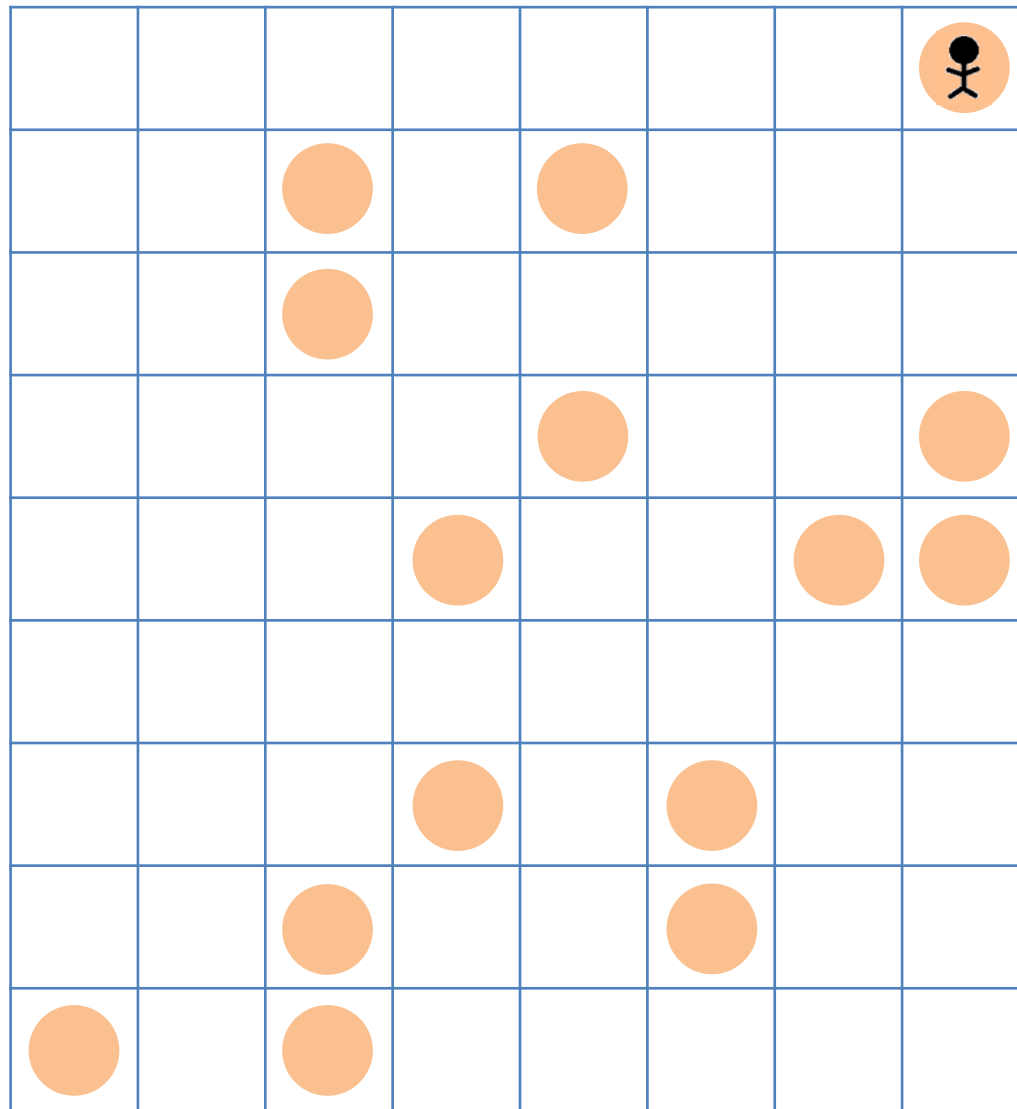
問題概要



移動：縦

時間：24

問題概要



移動：縦

時間：25

部分点解法 (1)

- 20 点 : $M, N \leq 1,000$
- (今いるマス, 移動できる方向) の組を「状態」として **幅優先探索 (BFS)**
- 状態の遷移
 - 移動できる方向に 1 マス移動
 - スイッチがあるならば押す
- 計算量 : $O(MN)$

部分点解法 (1)

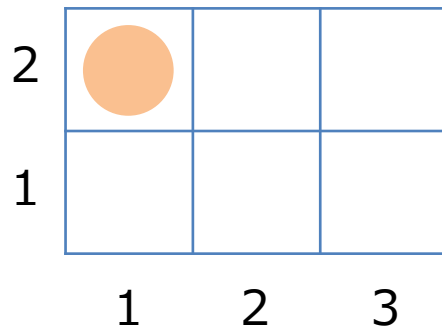
- 幅優先探索
 - (スタート地点, 縦移動) には時間 0 で到達
 - 時間 1 で到達できる状態, 時間 2 で到達できる状態, ……を順に求める
 - キューを利用
- 問題例
 - JOI 2010-2011 予選 問題 5 「チーズ」

部分点解法 (2)

- 30 点 : $K \leq 2,000$
- スイッチのあるマス (とスタート・ゴール) のみに着目したい
 - 他のマスは縦か横に素通りするだけ

部分点解法 (2)

- スイッチ・スタート・ゴールのみに着目



(1, 1)
縦移動

(1, 1)
横移動

(1, 2)
縦移動

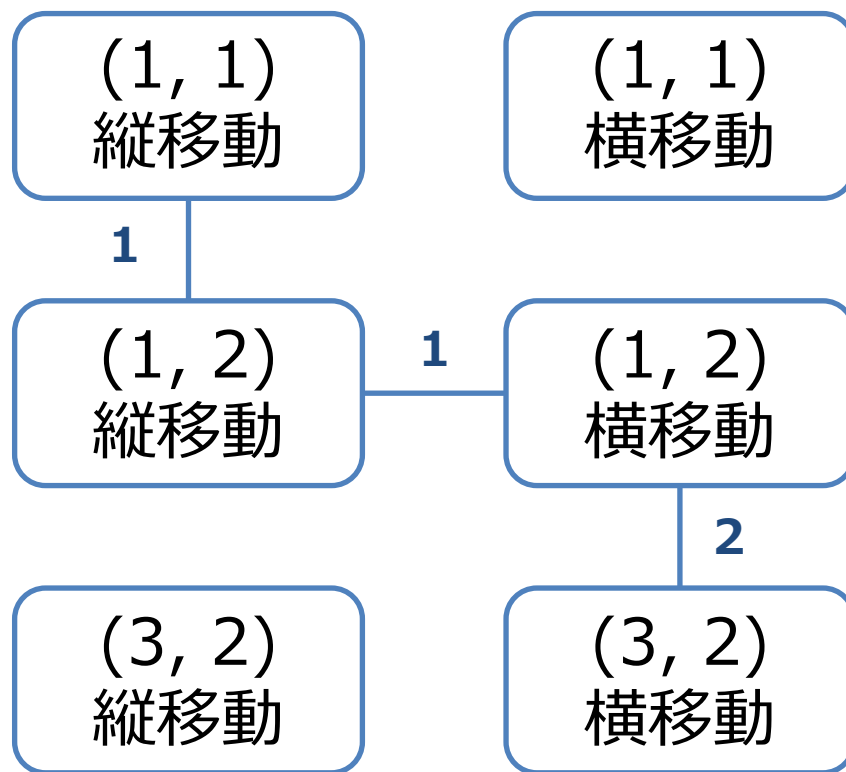
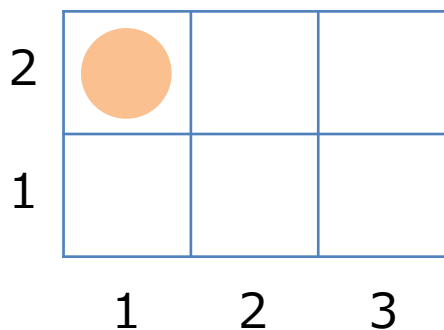
(1, 2)
横移動

(3, 2)
縦移動

(3, 2)
横移動

部分点解法 (2)

- スイッチ・スタート・ゴールのみに着目



部分点解法 (2)

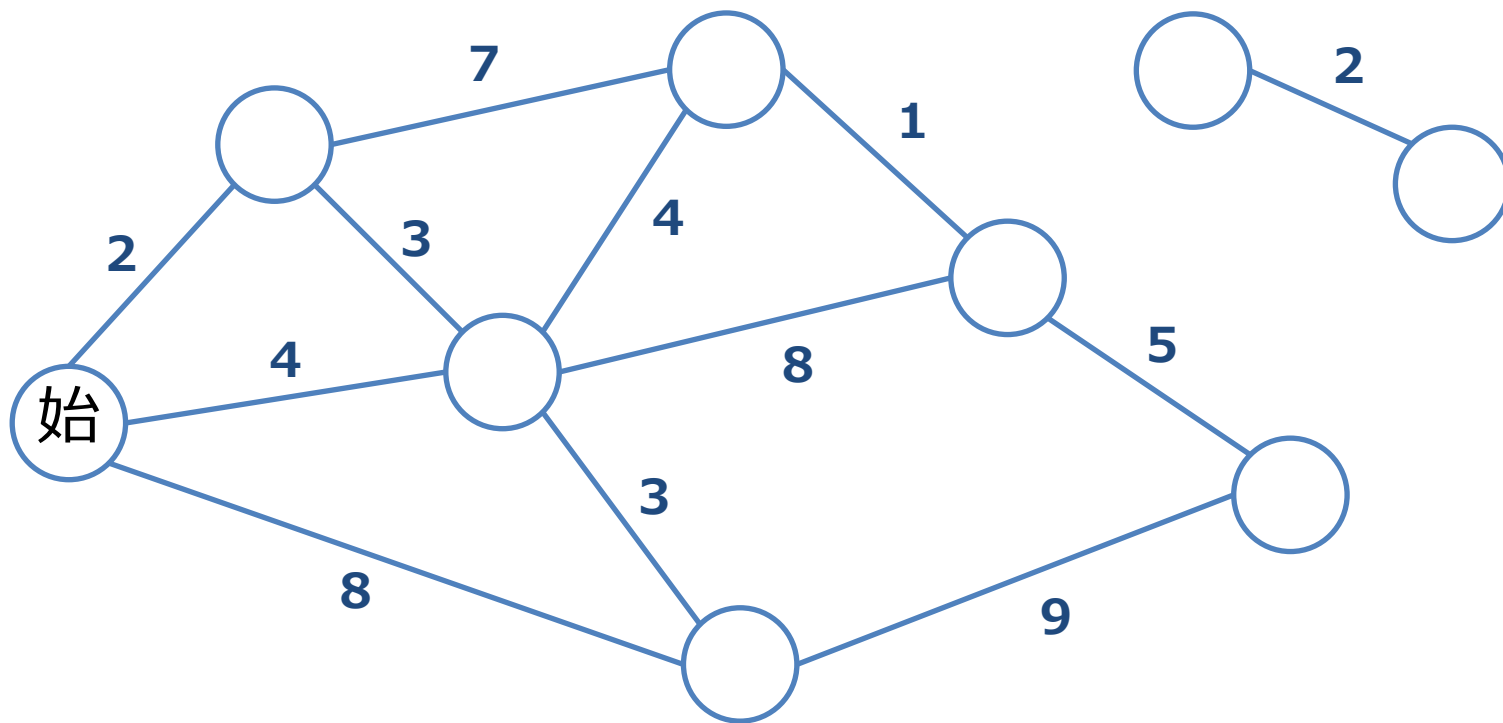
- グラフの最短路を求める問題
 - 頂点：(スイッチ or スタート or ゴール のマス, 移動できる方向) の組
 - 始点： $((1, 1), \text{縦移動})$
 - 終点： $((M, N), \text{縦移動}), ((M, N), \text{横移動})$
 - 辺：
 - 縦 or 横 に並んだ 2 マスの 縦移動 or 横移動 に対応する頂点を結ぶ辺 (コストは距離)
 - スイッチのあるマスに対して, 縦移動に対応する頂点と横移動に対応する頂点を結ぶ辺 (コスト 1)

部分点解法 (2)

- グラフの最短路を求める問題
- **Dijkstra (ダイクストラ) 法**
 - コストが非負のグラフについて, 1 つの始点から他の頂点への最短距離を求める
 - 問題例
 - JOI 2010-2011 本選 3 「JOI 国の買い物事情」

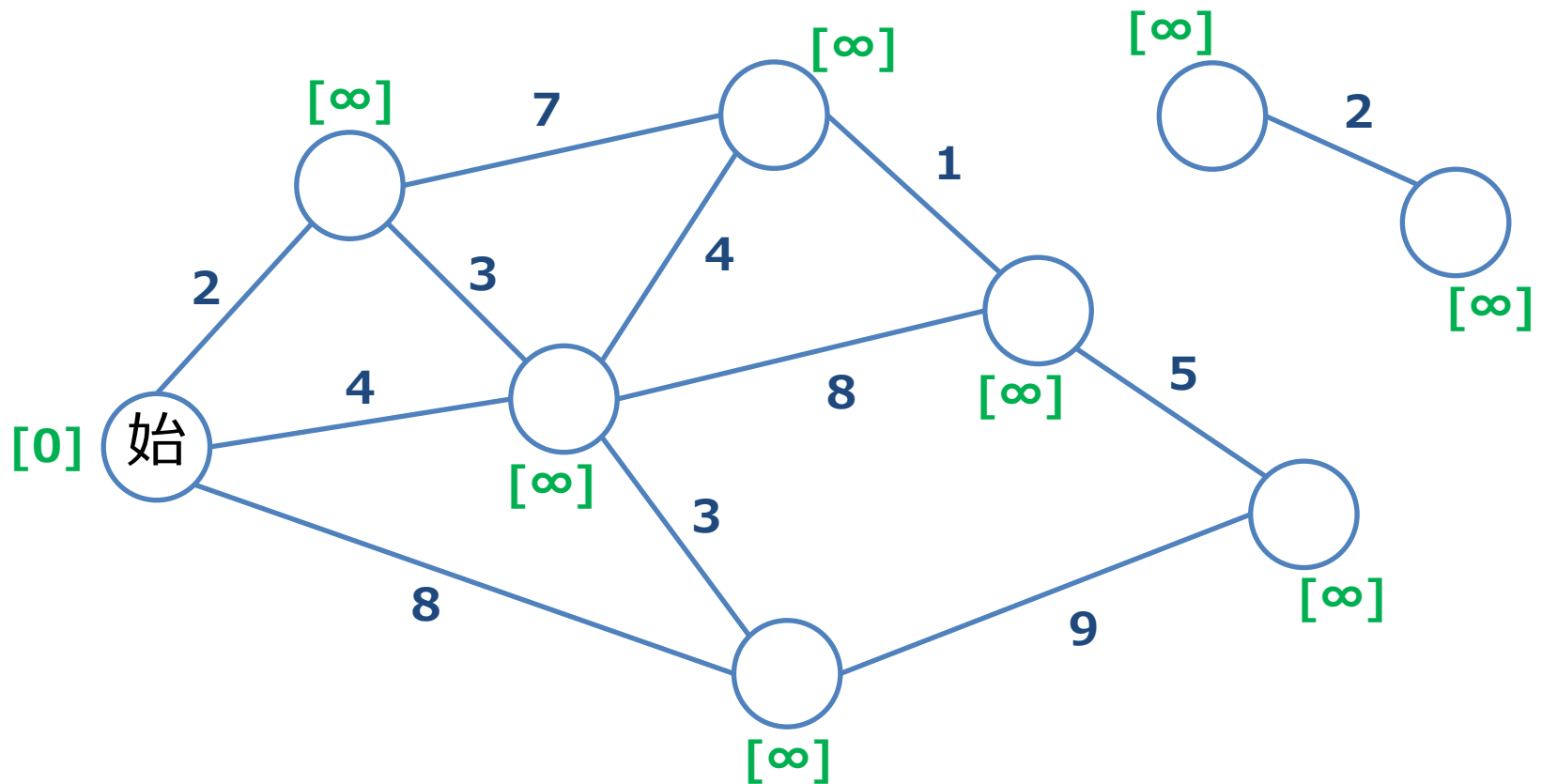
Dijkstra 法

- 方針：始点から近い順に確定させていく



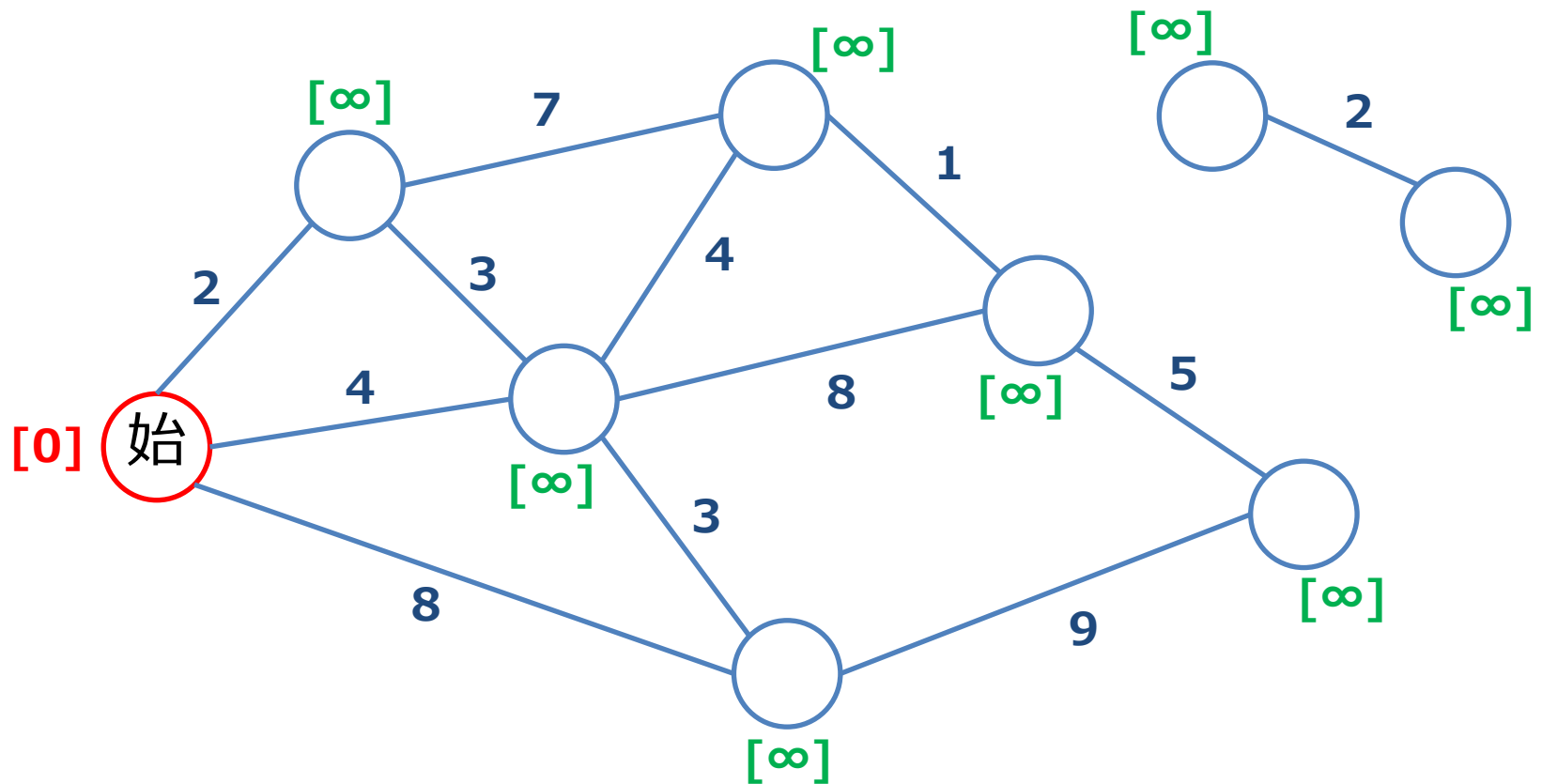
Dijkstra 法

- 方針：始点から近い順に確定させていく



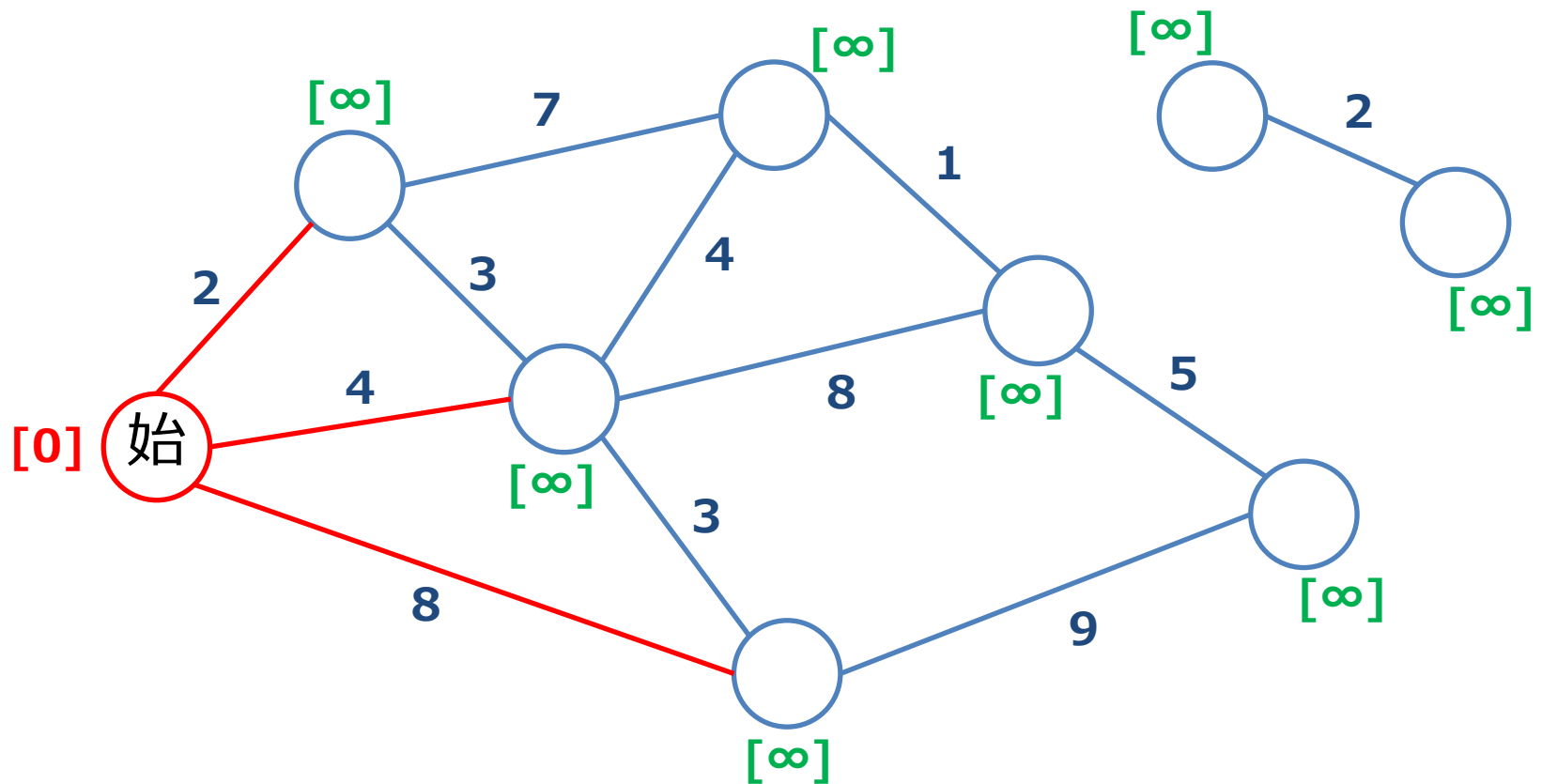
Dijkstra 法

- 方針：始点から近い順に確定させていく



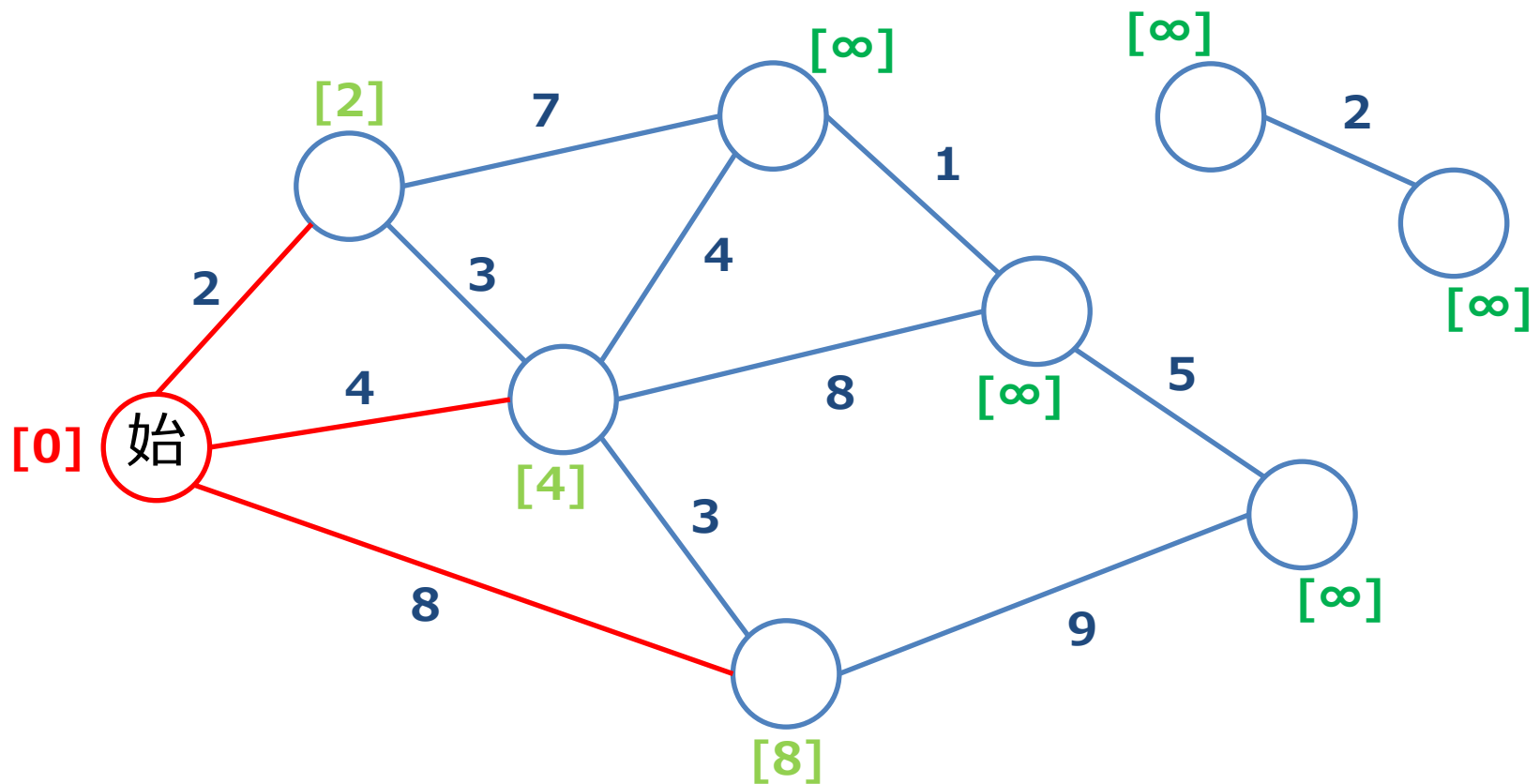
Dijkstra 法

- 方針：始点から近い順に確定させていく



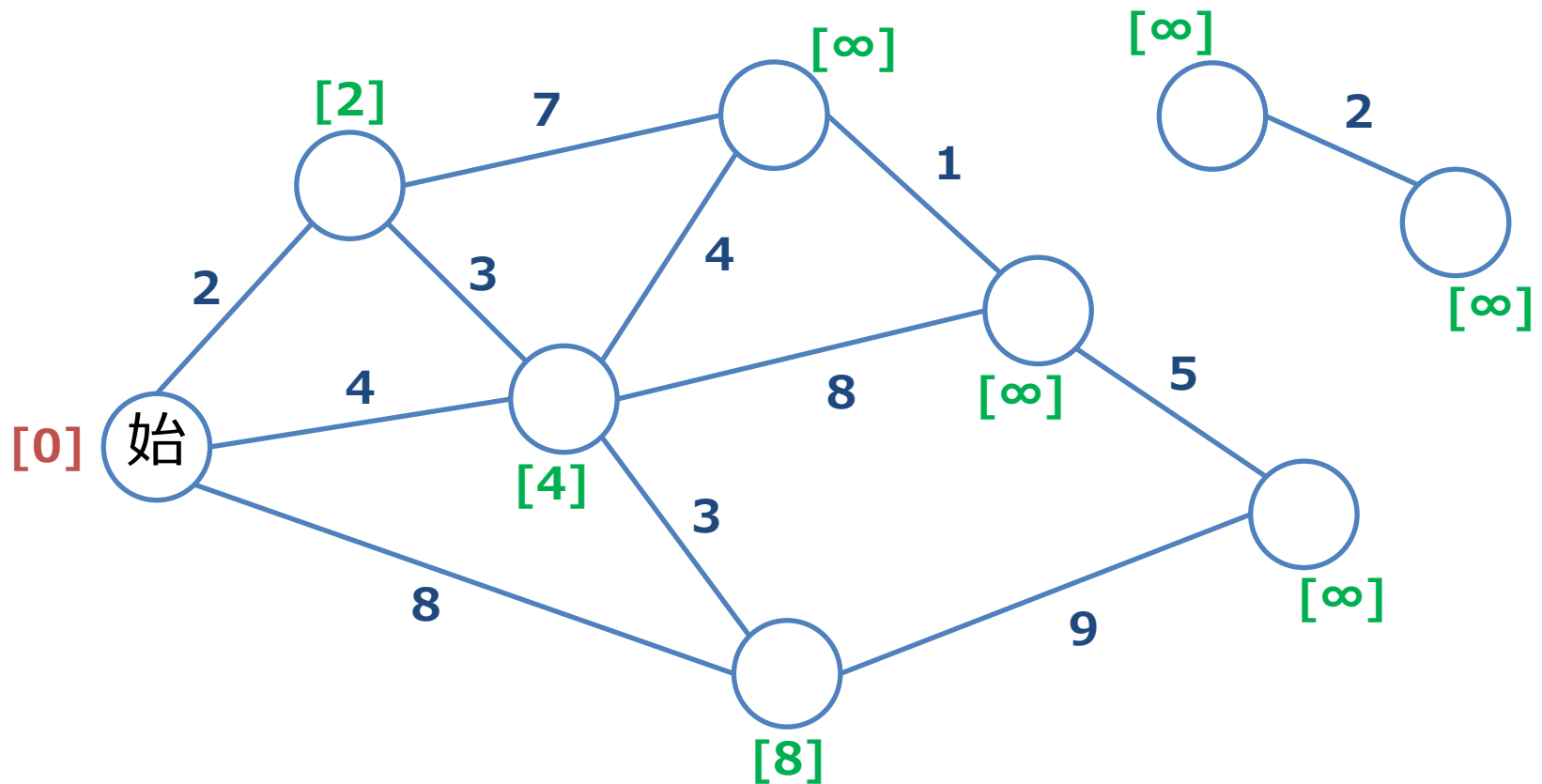
Dijkstra 法

- 方針：始点から近い順に確定させていく



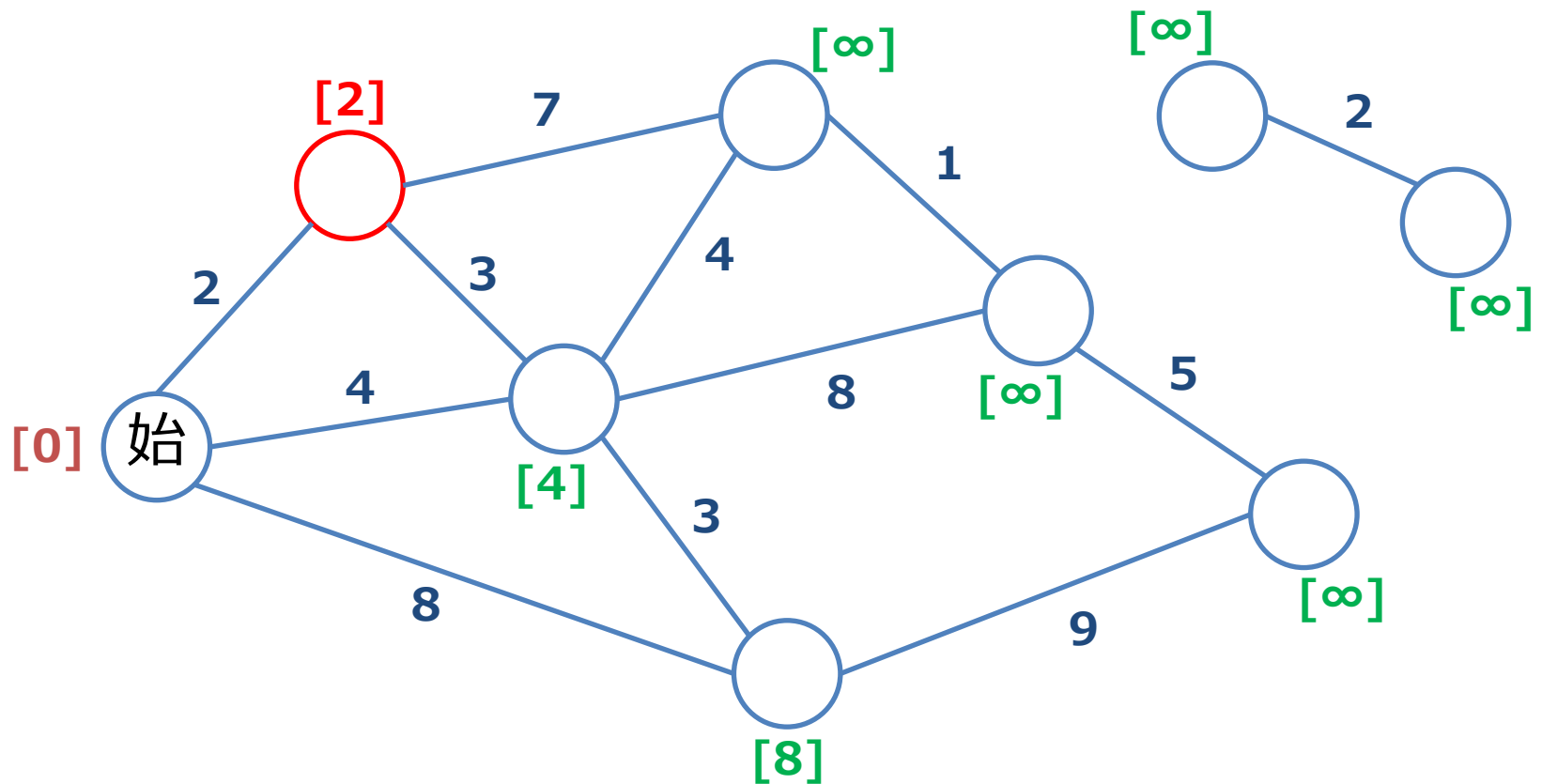
Dijkstra 法

- 方針：始点から近い順に確定させていく



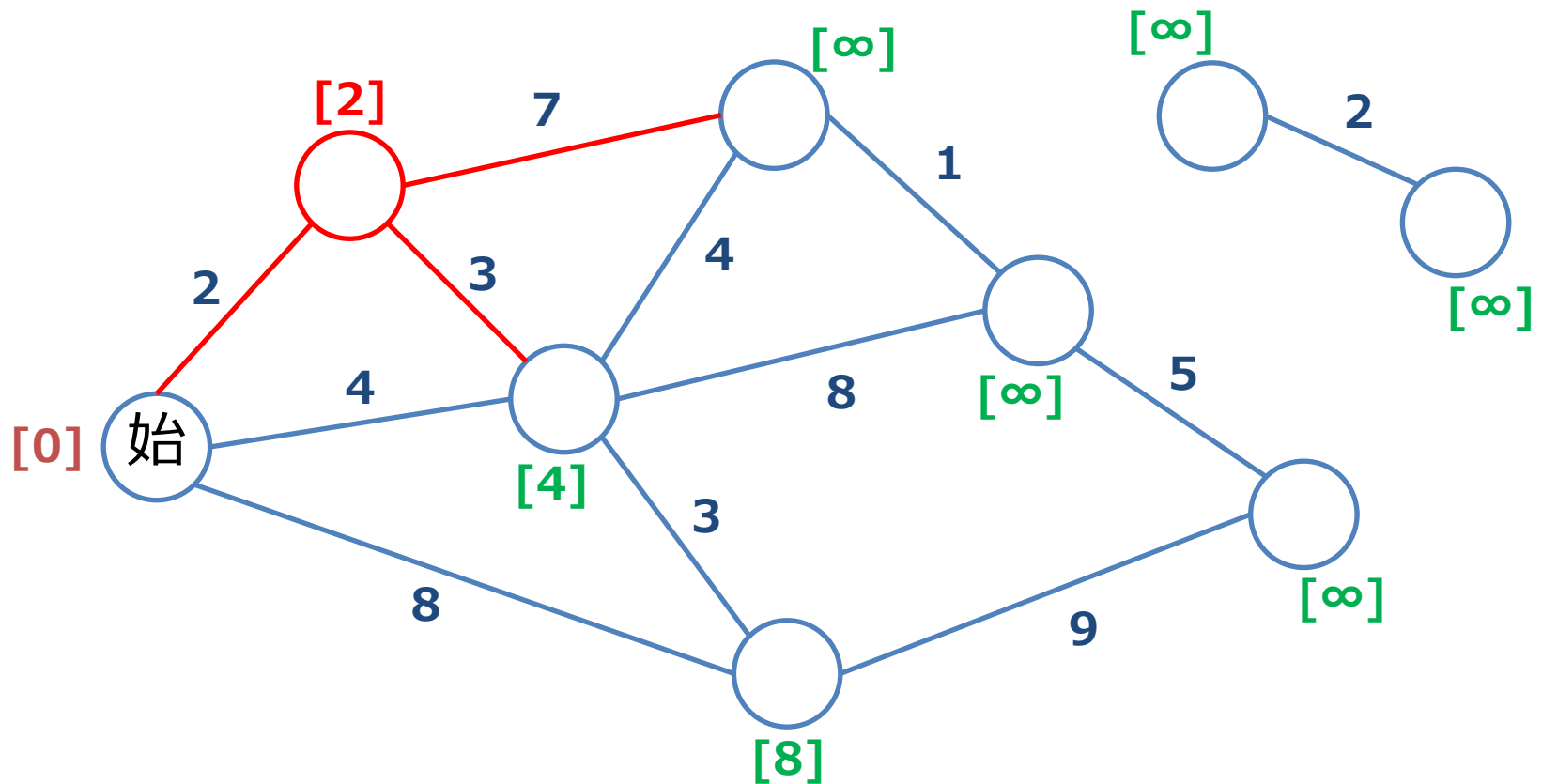
Dijkstra 法

- 方針：始点から近い順に確定させていく



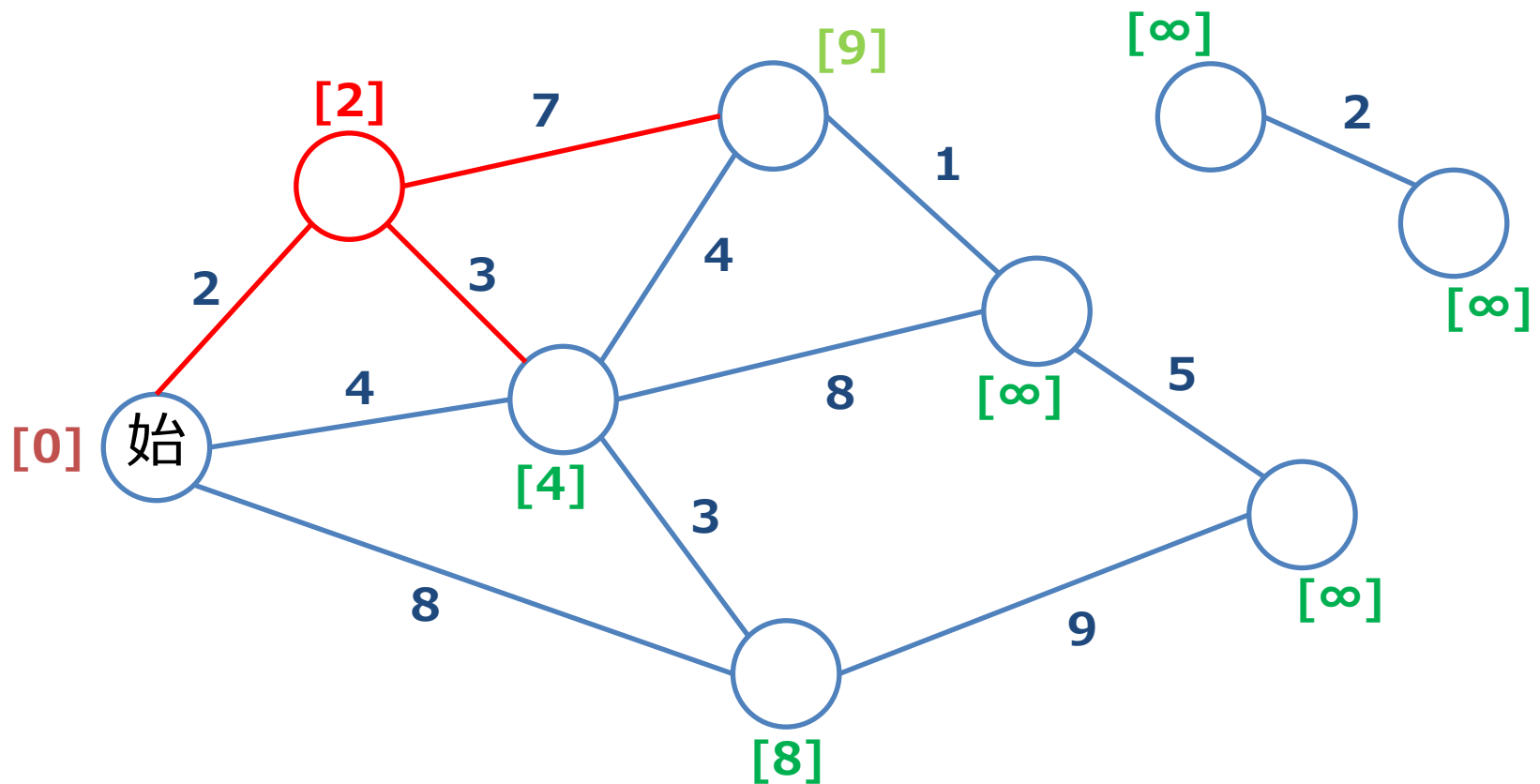
Dijkstra 法

- 方針：始点から近い順に確定させていく



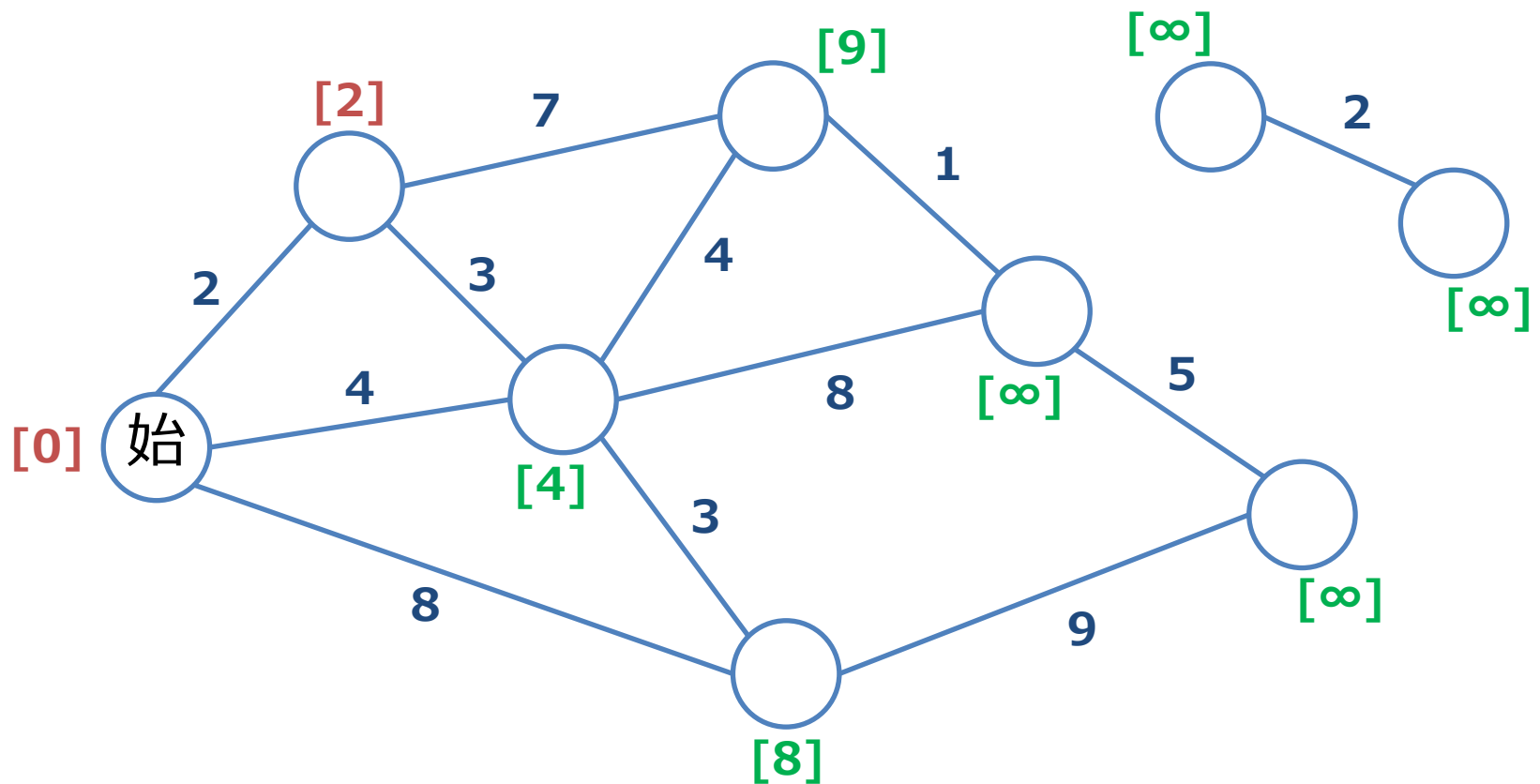
Dijkstra 法

- 方針：始点から近い順に確定させていく



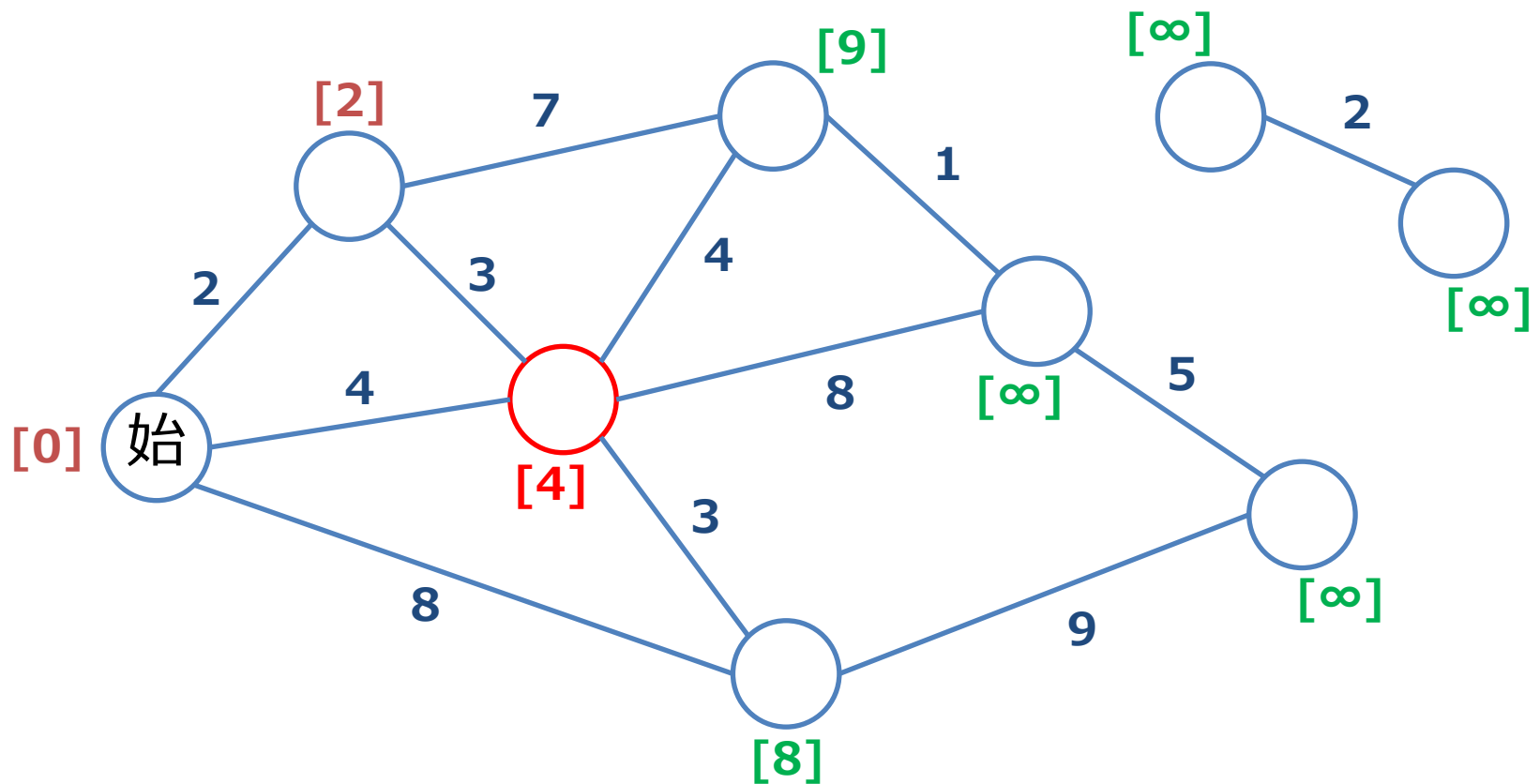
Dijkstra 法

- 方針：始点から近い順に確定させていく



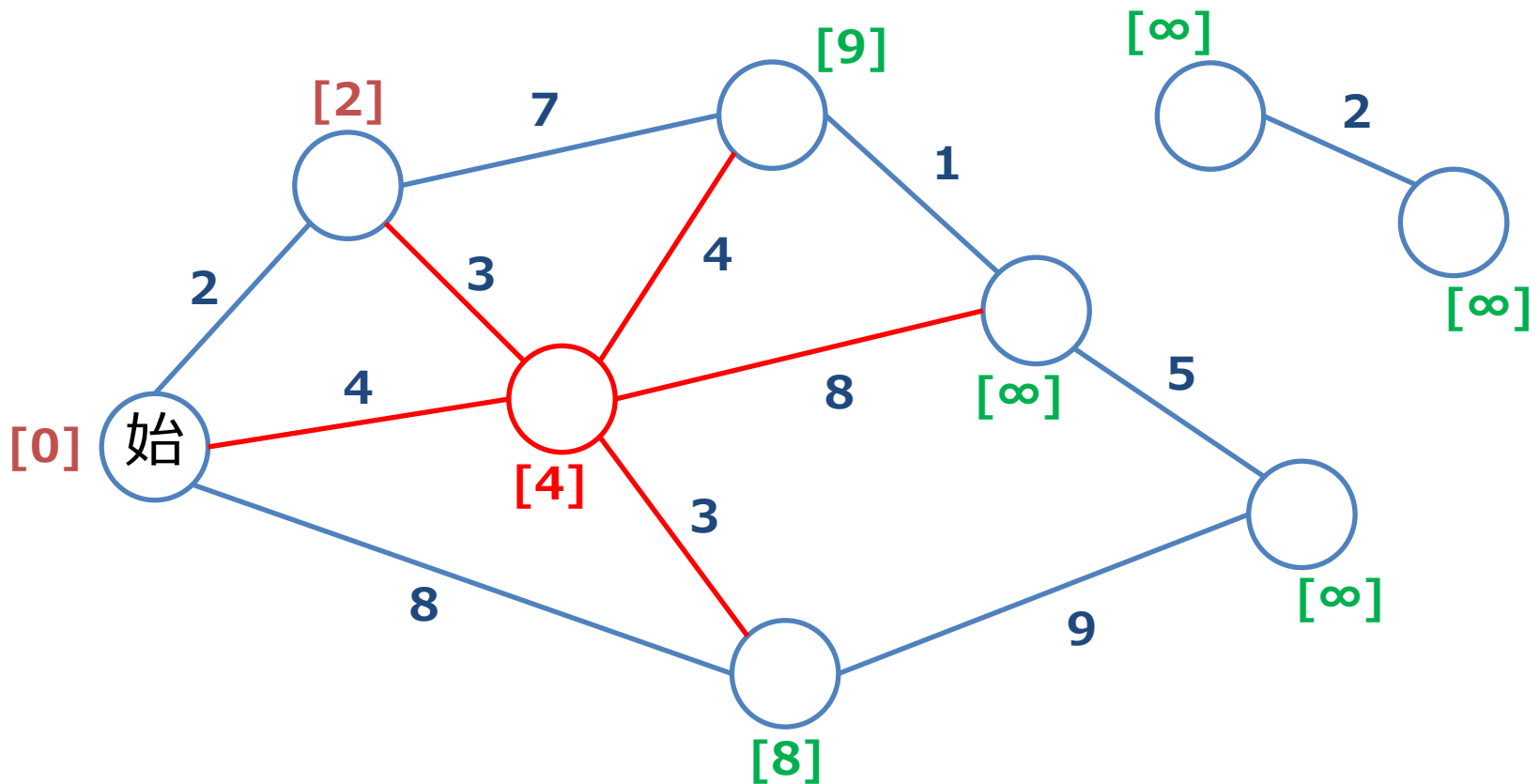
Dijkstra 法

- 方針：始点から近い順に確定させていく



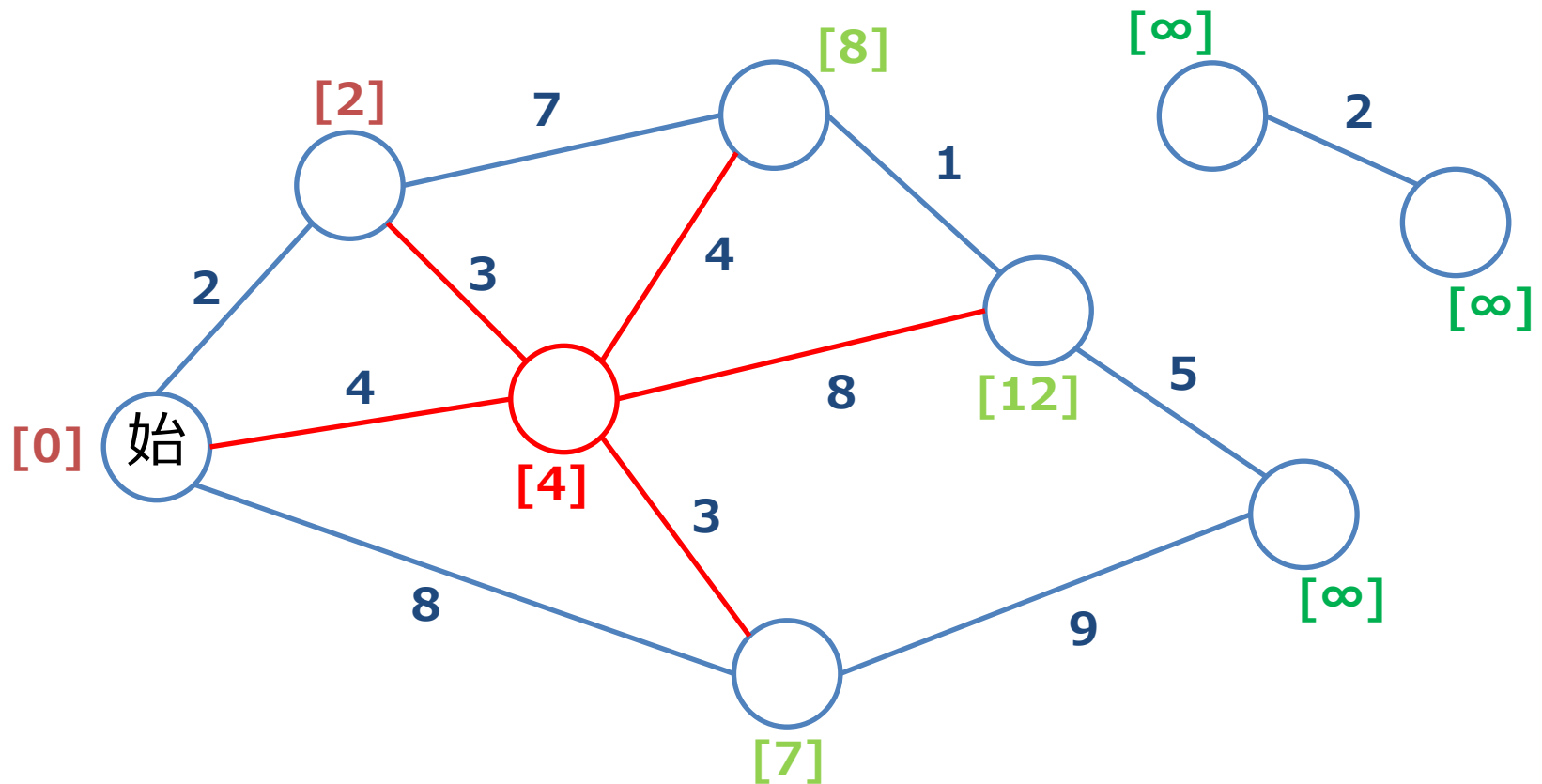
Dijkstra 法

- 方針：始点から近い順に確定させていく



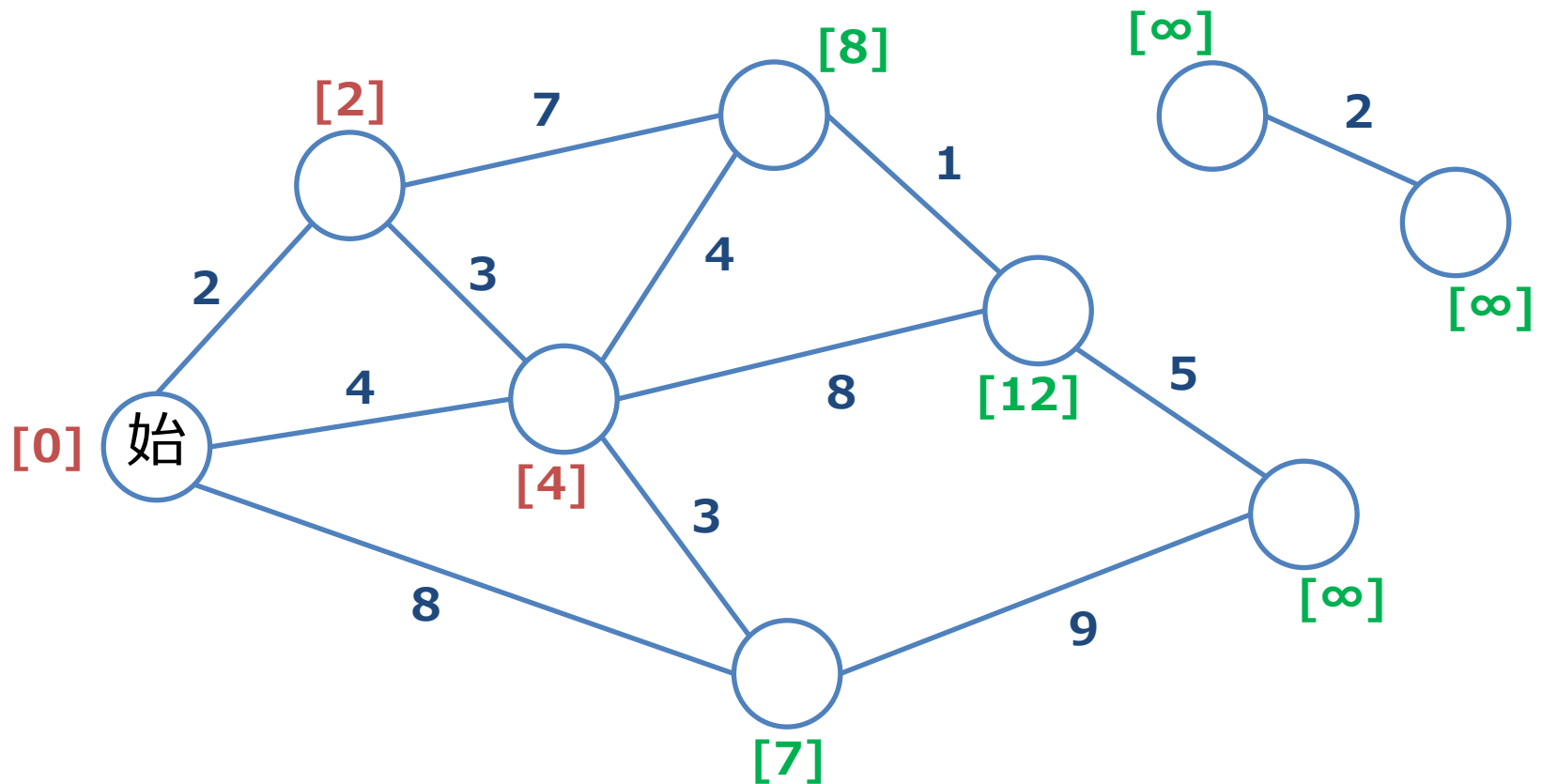
Dijkstra 法

- 方針：始点から近い順に確定させていく



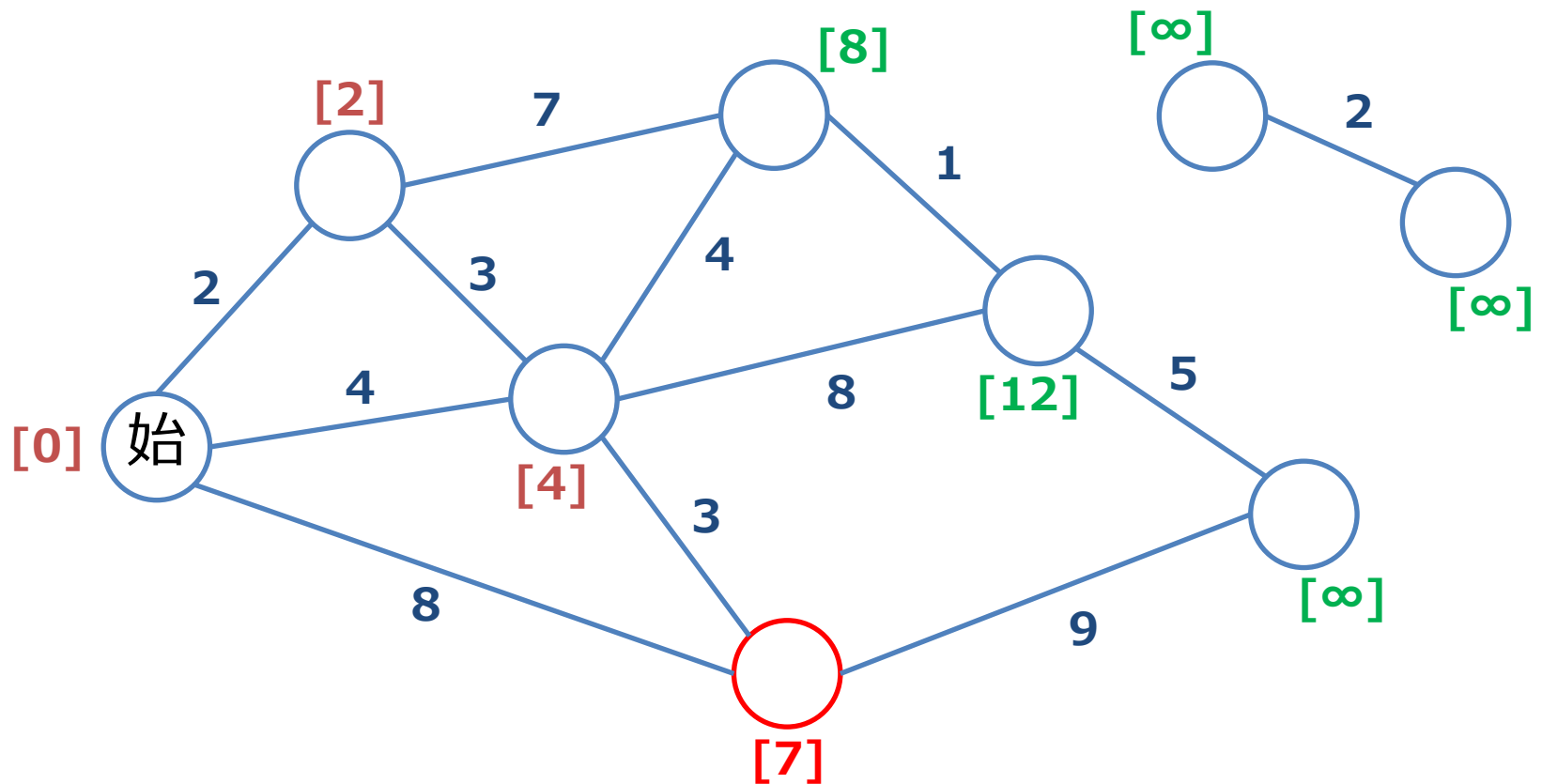
Dijkstra 法

- 方針：始点から近い順に確定させていく



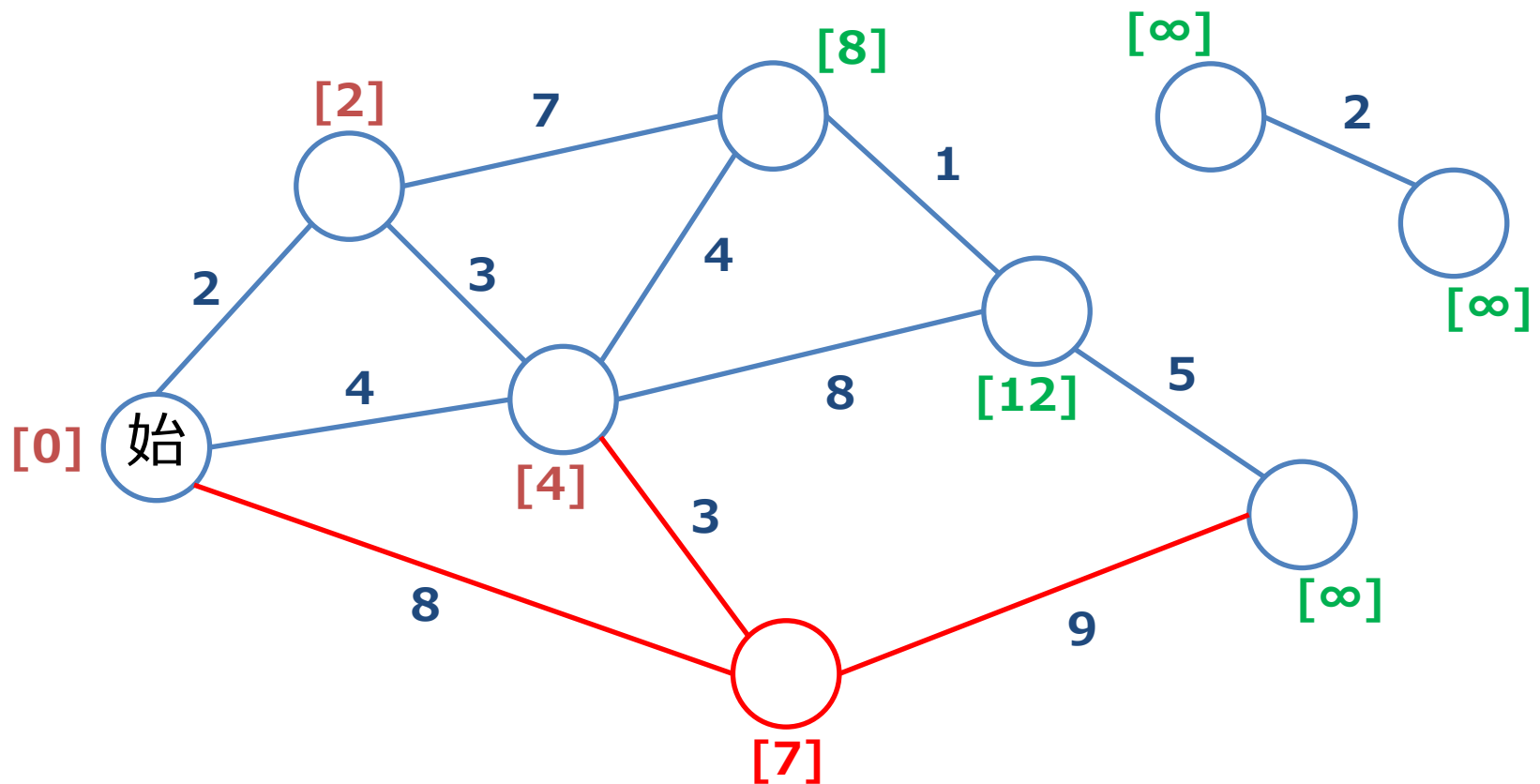
Dijkstra 法

- 方針：始点から近い順に確定させていく



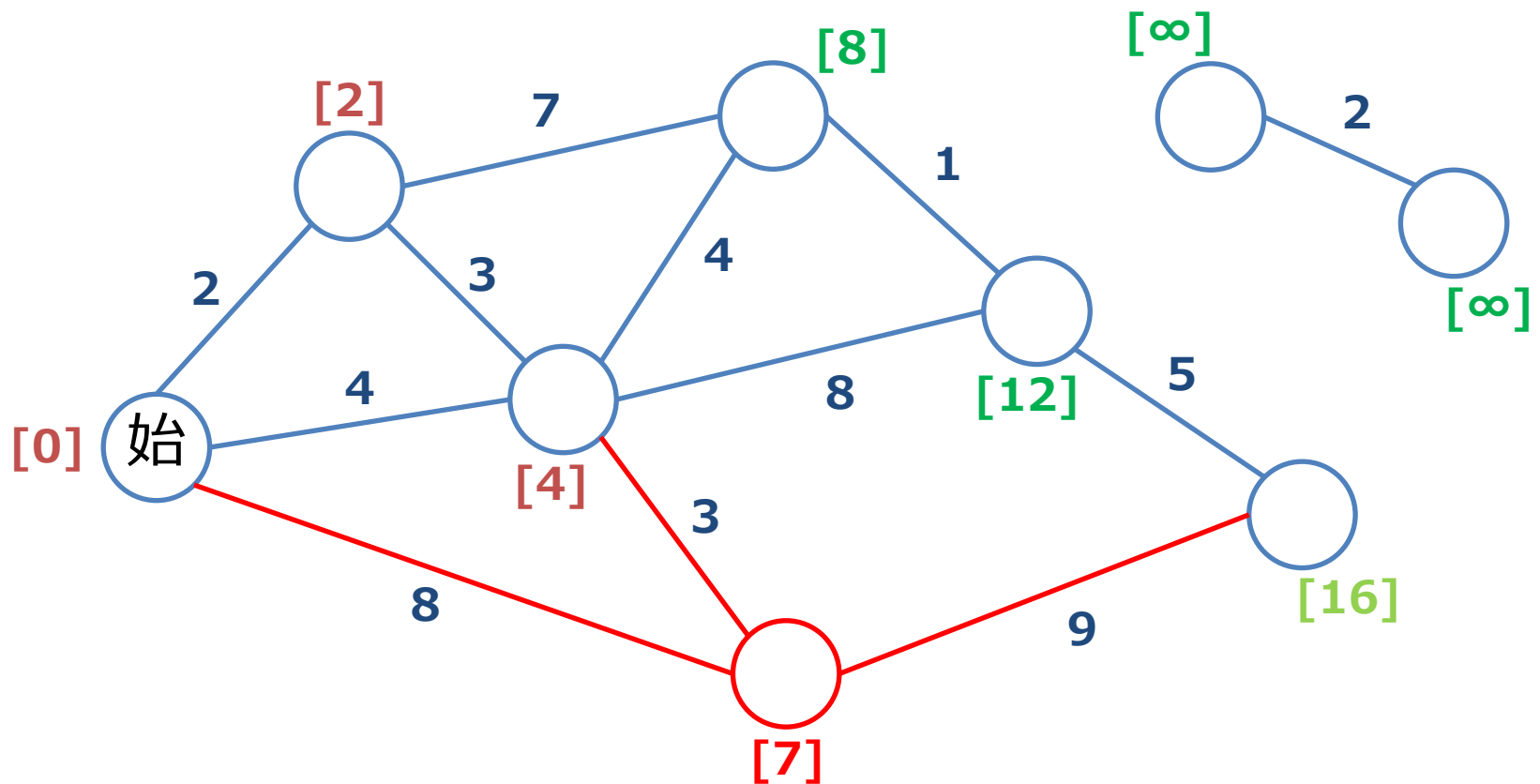
Dijkstra 法

- 方針：始点から近い順に確定させていく



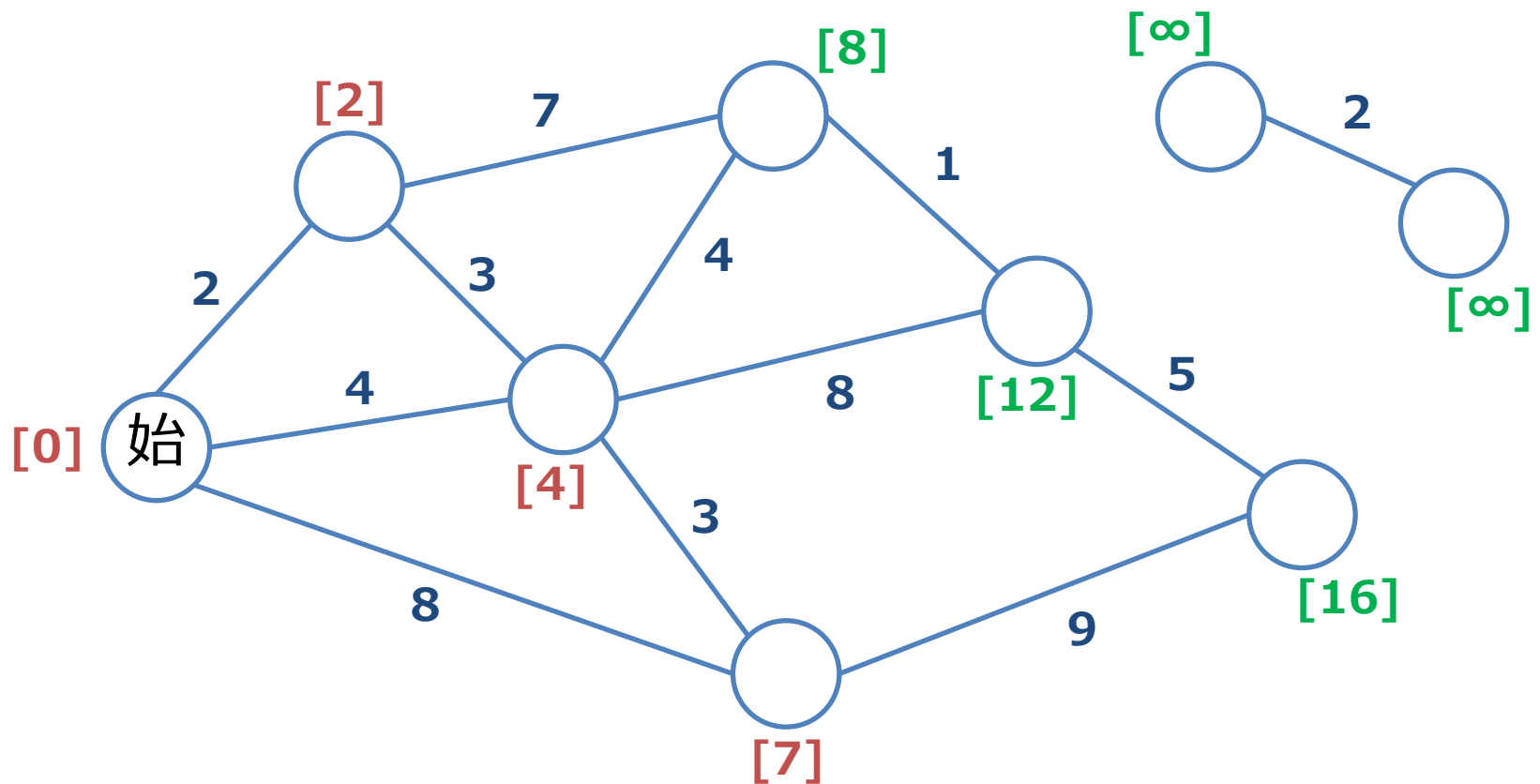
Dijkstra 法

- 方針：始点から近い順に確定させていく



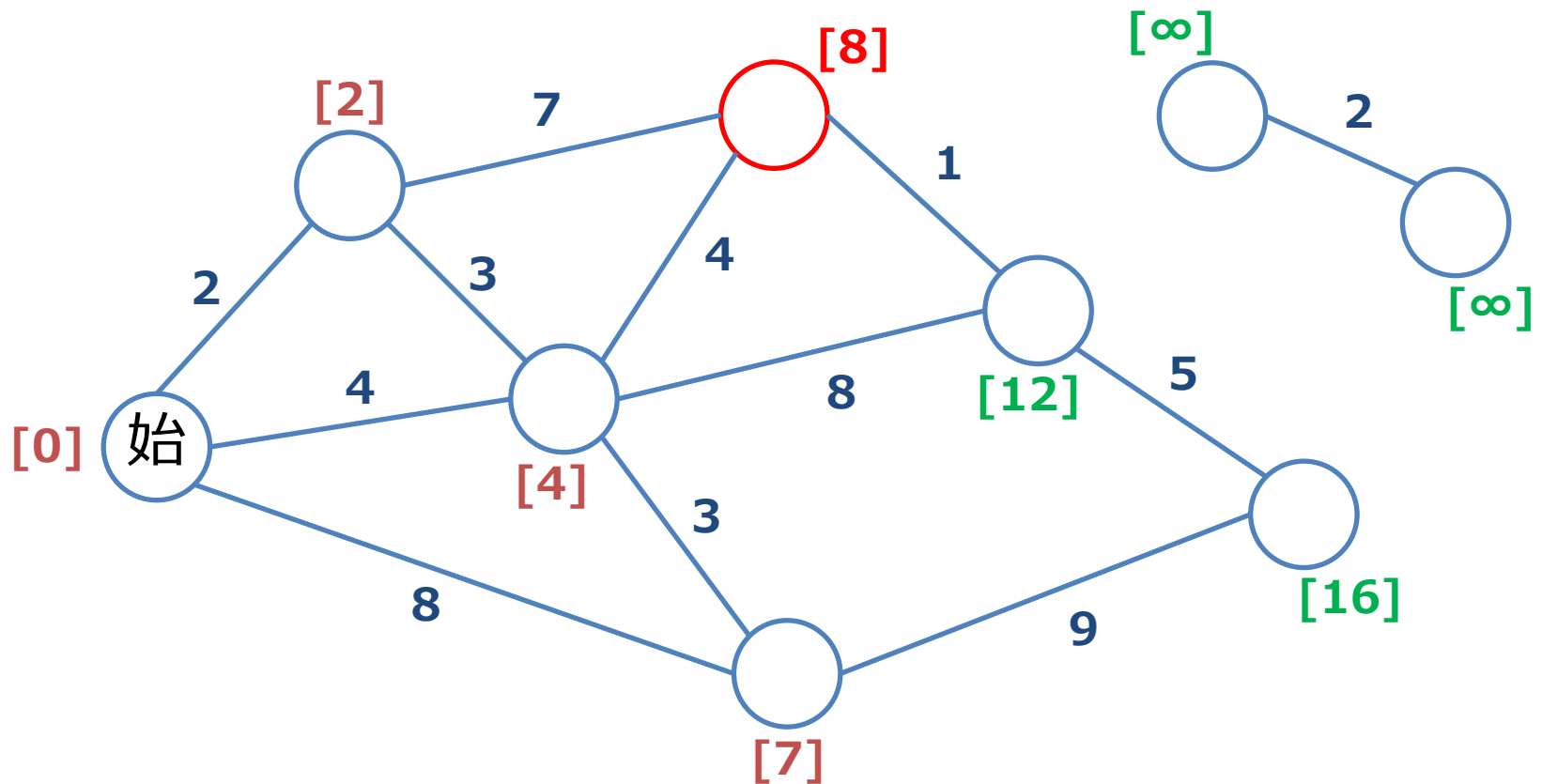
Dijkstra 法

- 方針：始点から近い順に確定させていく



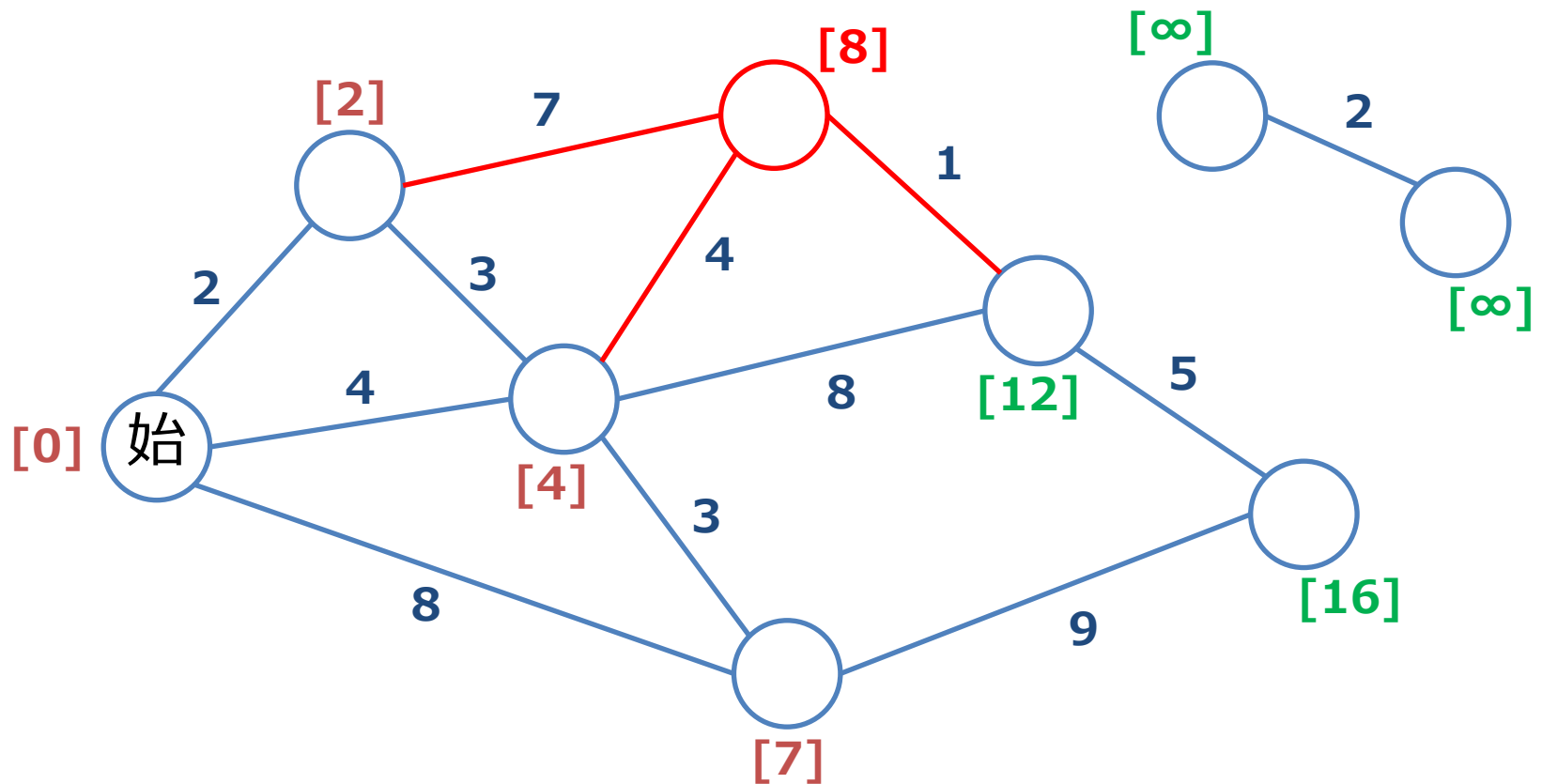
Dijkstra 法

- 方針：始点から近い順に確定させていく



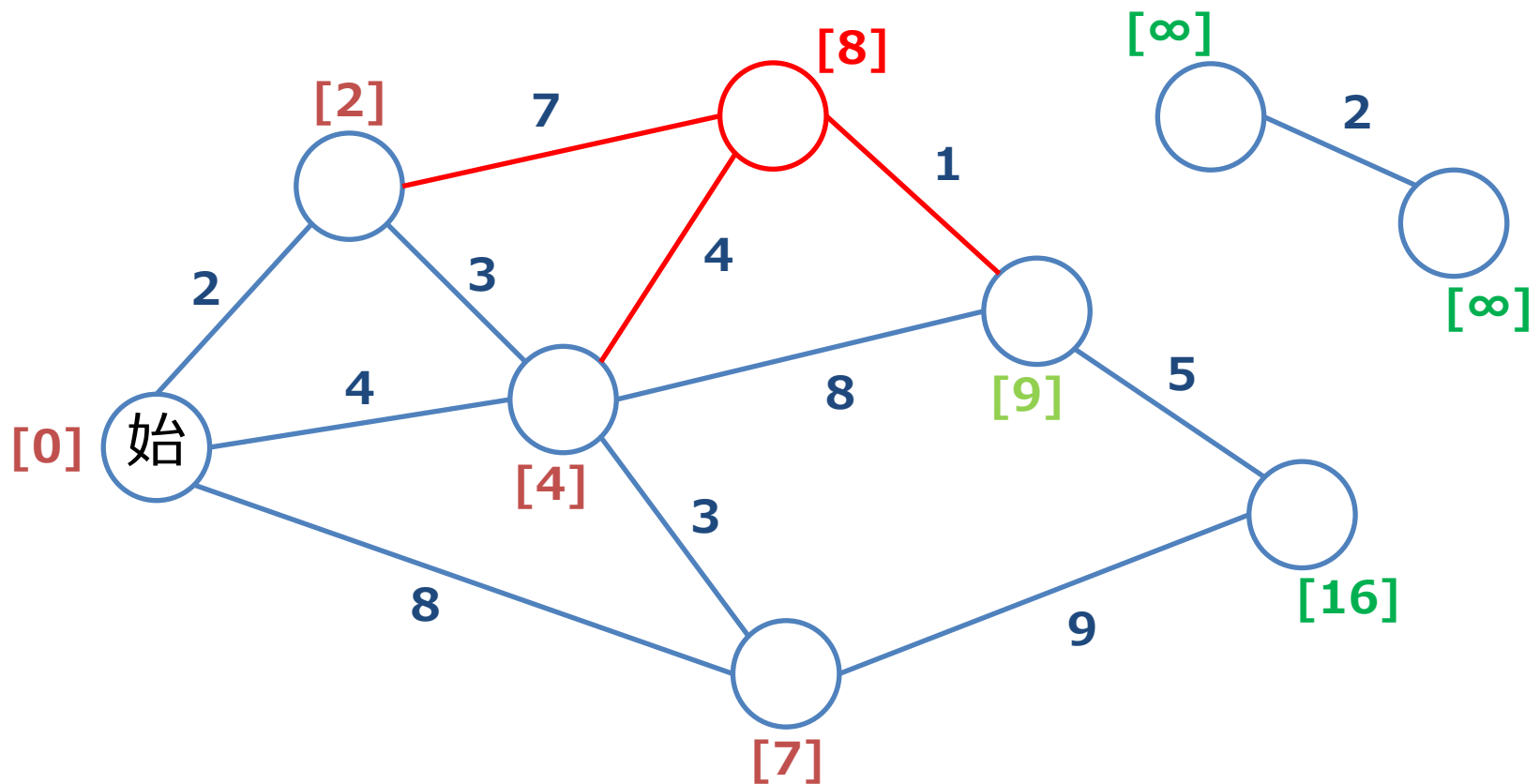
Dijkstra 法

- 方針：始点から近い順に確定させていく



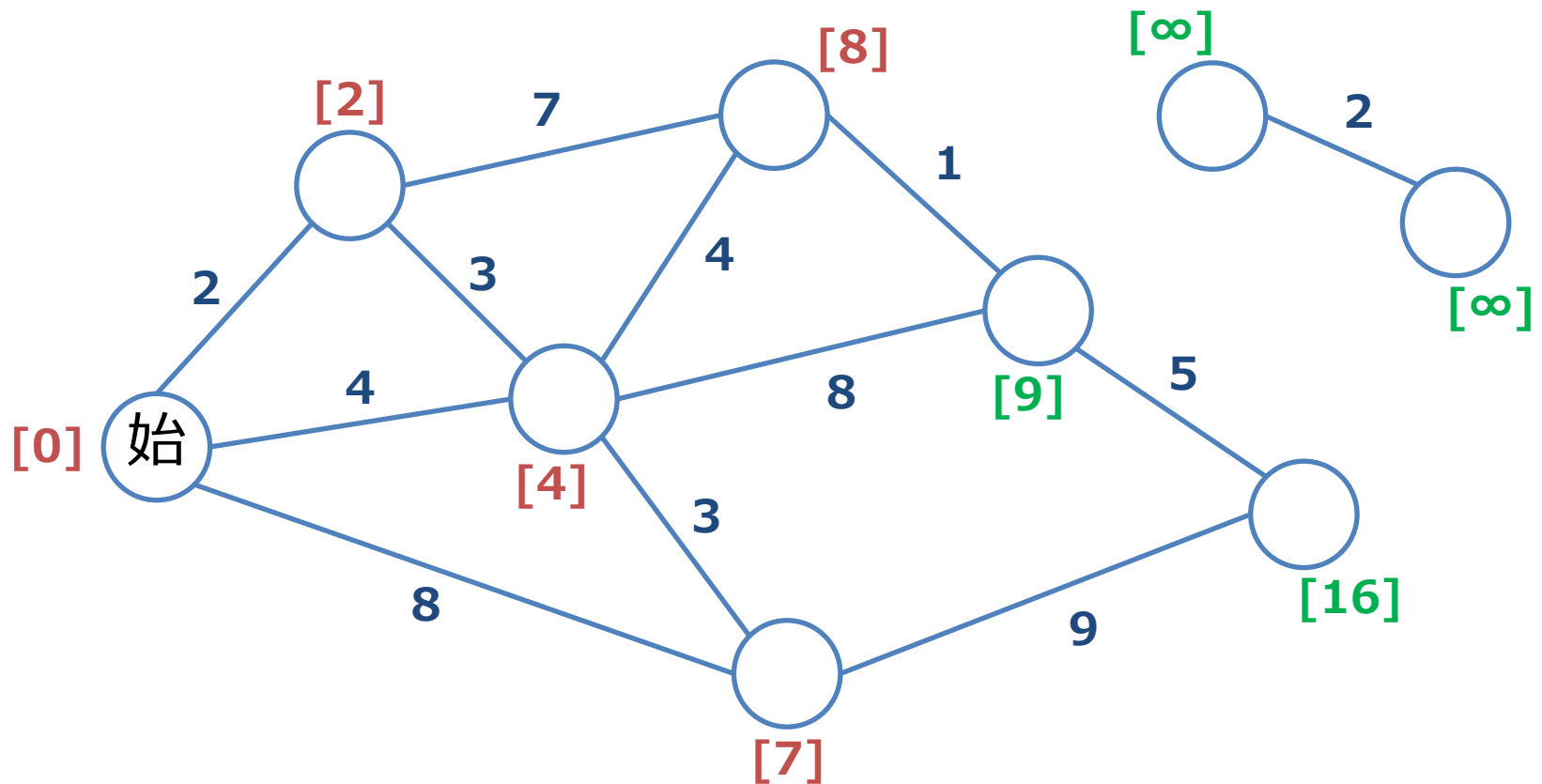
Dijkstra 法

- 方針：始点から近い順に確定させていく



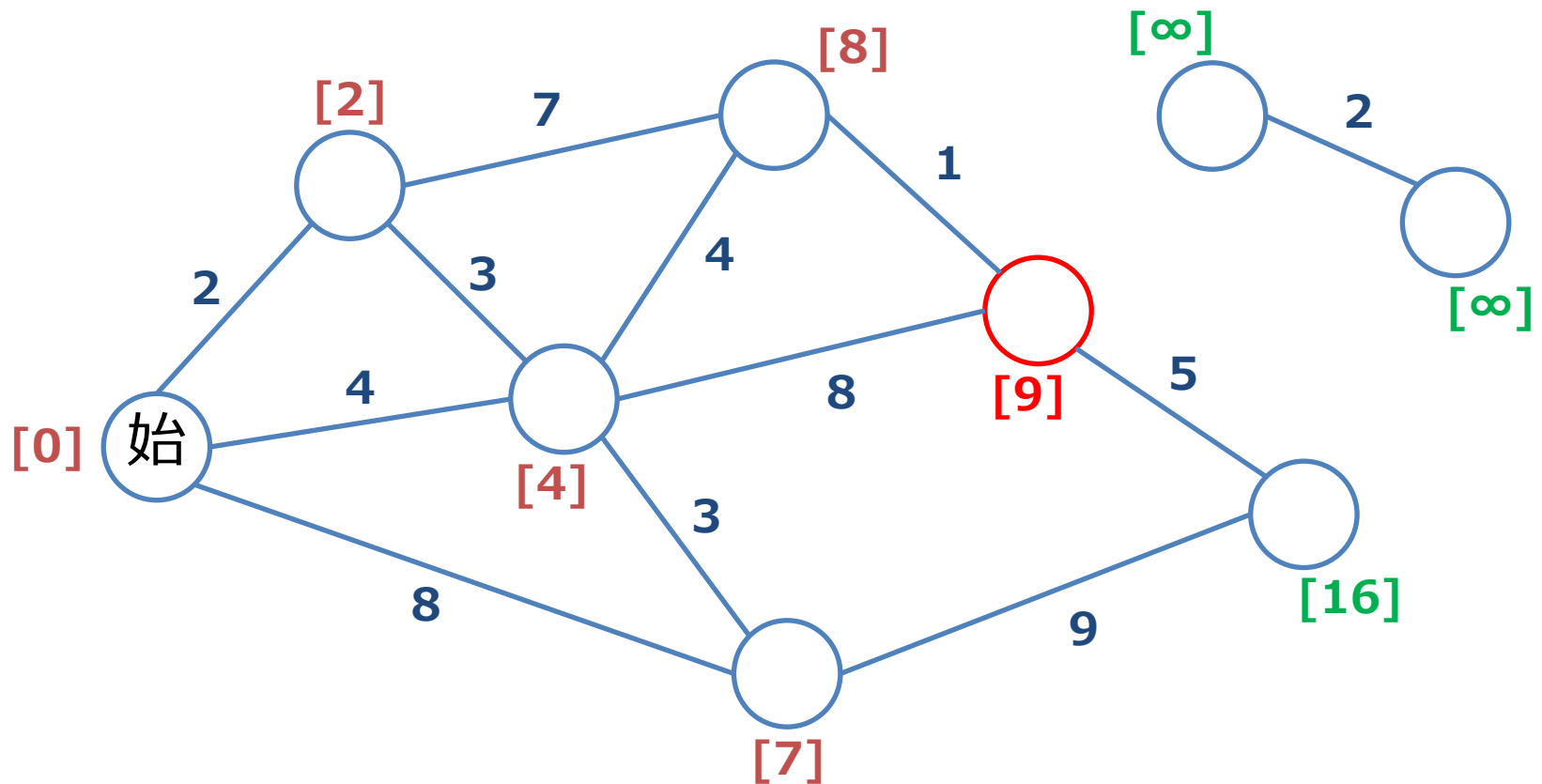
Dijkstra 法

- 方針：始点から近い順に確定させていく



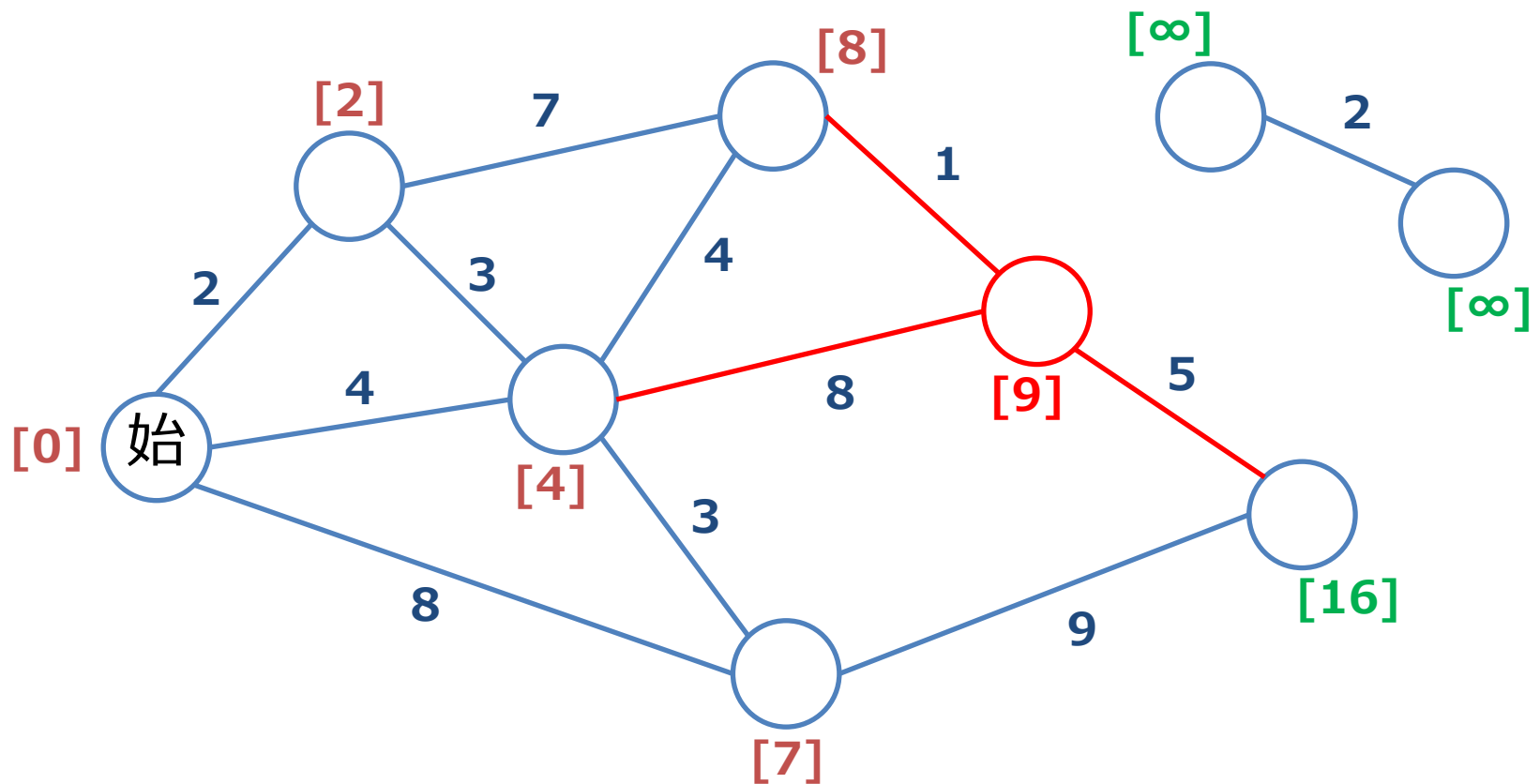
Dijkstra 法

- 方針：始点から近い順に確定させていく



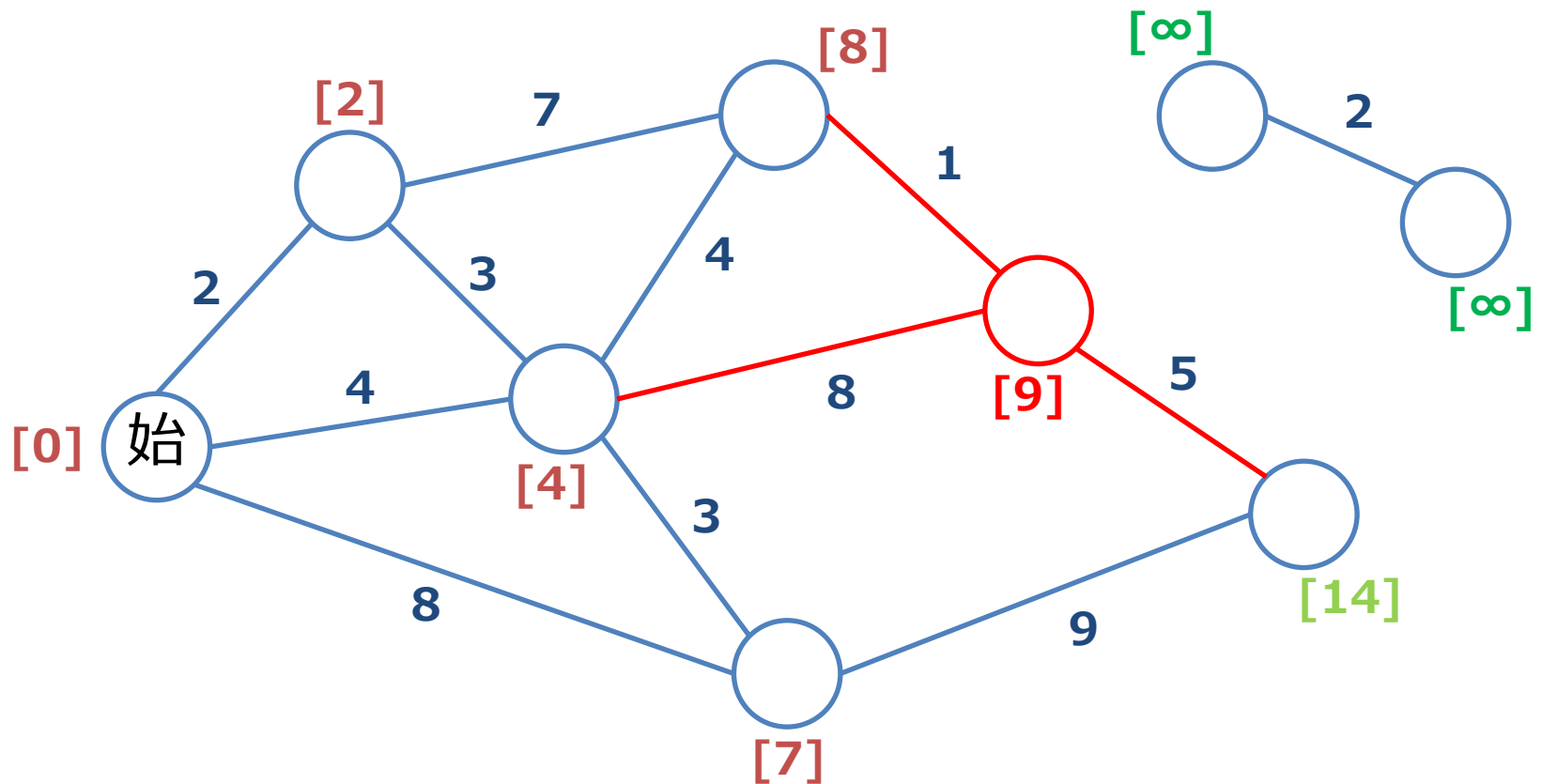
Dijkstra 法

- 方針：始点から近い順に確定させていく



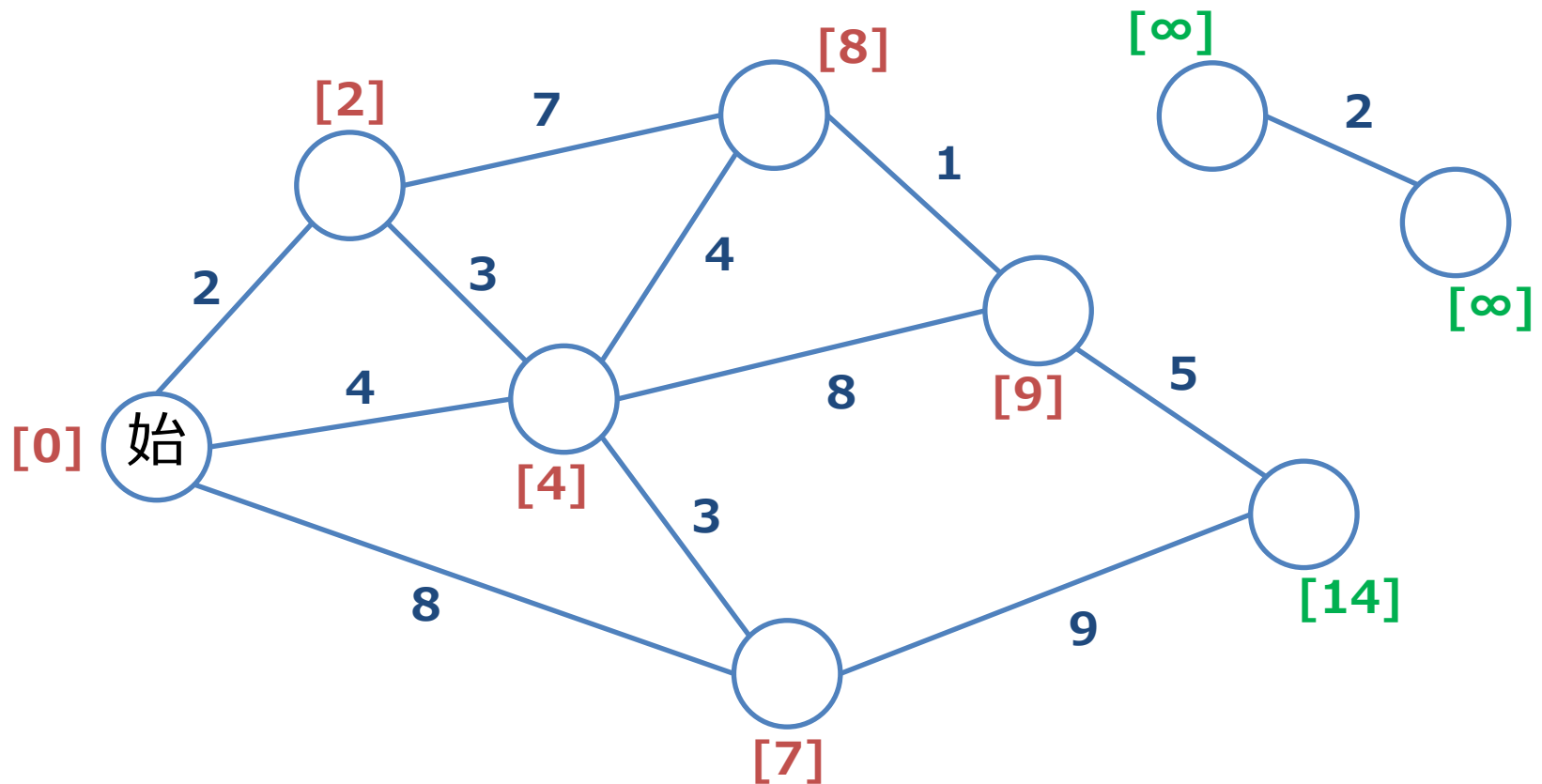
Dijkstra 法

- 方針：始点から近い順に確定させていく



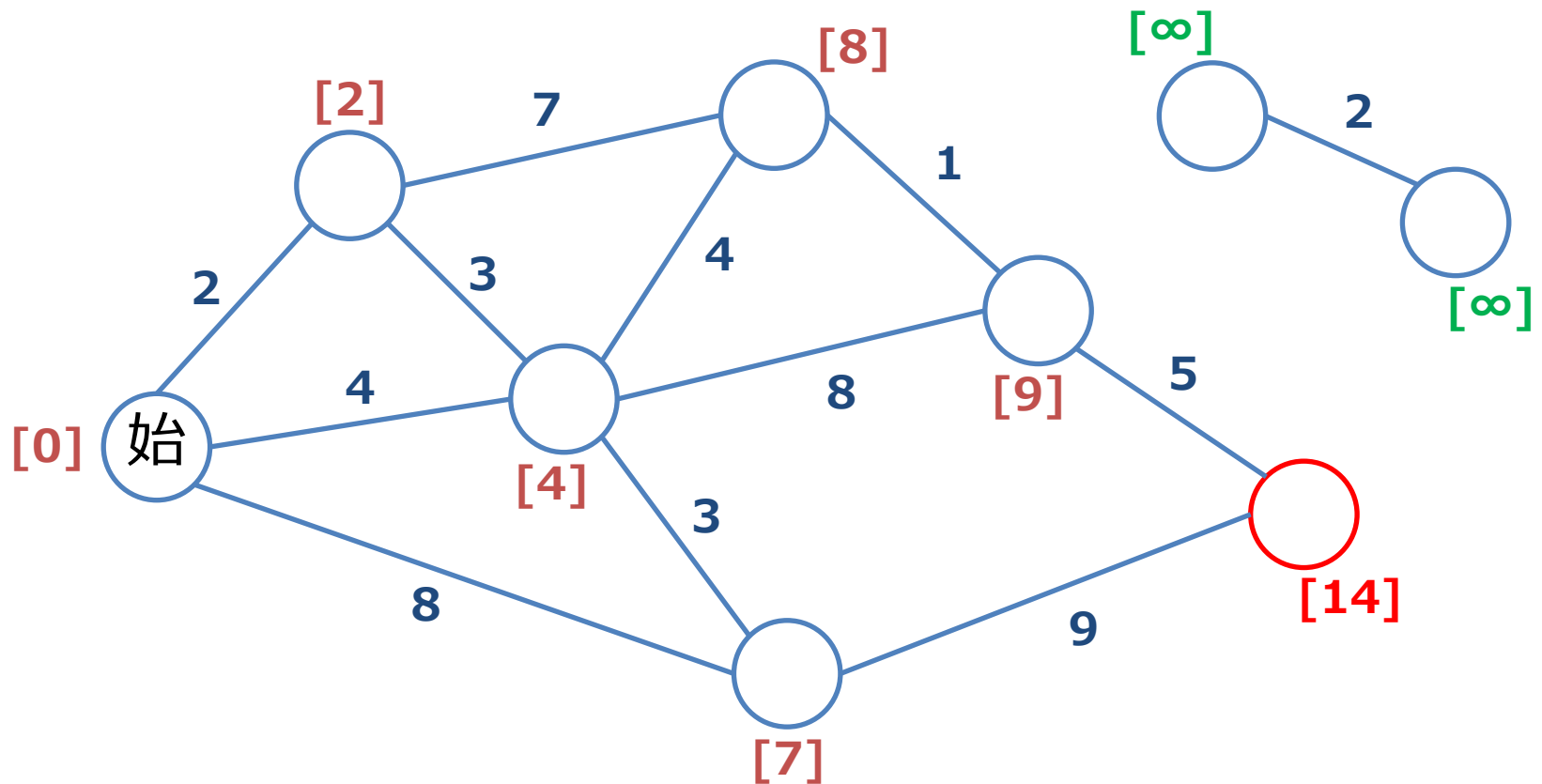
Dijkstra 法

- 方針：始点から近い順に確定させていく



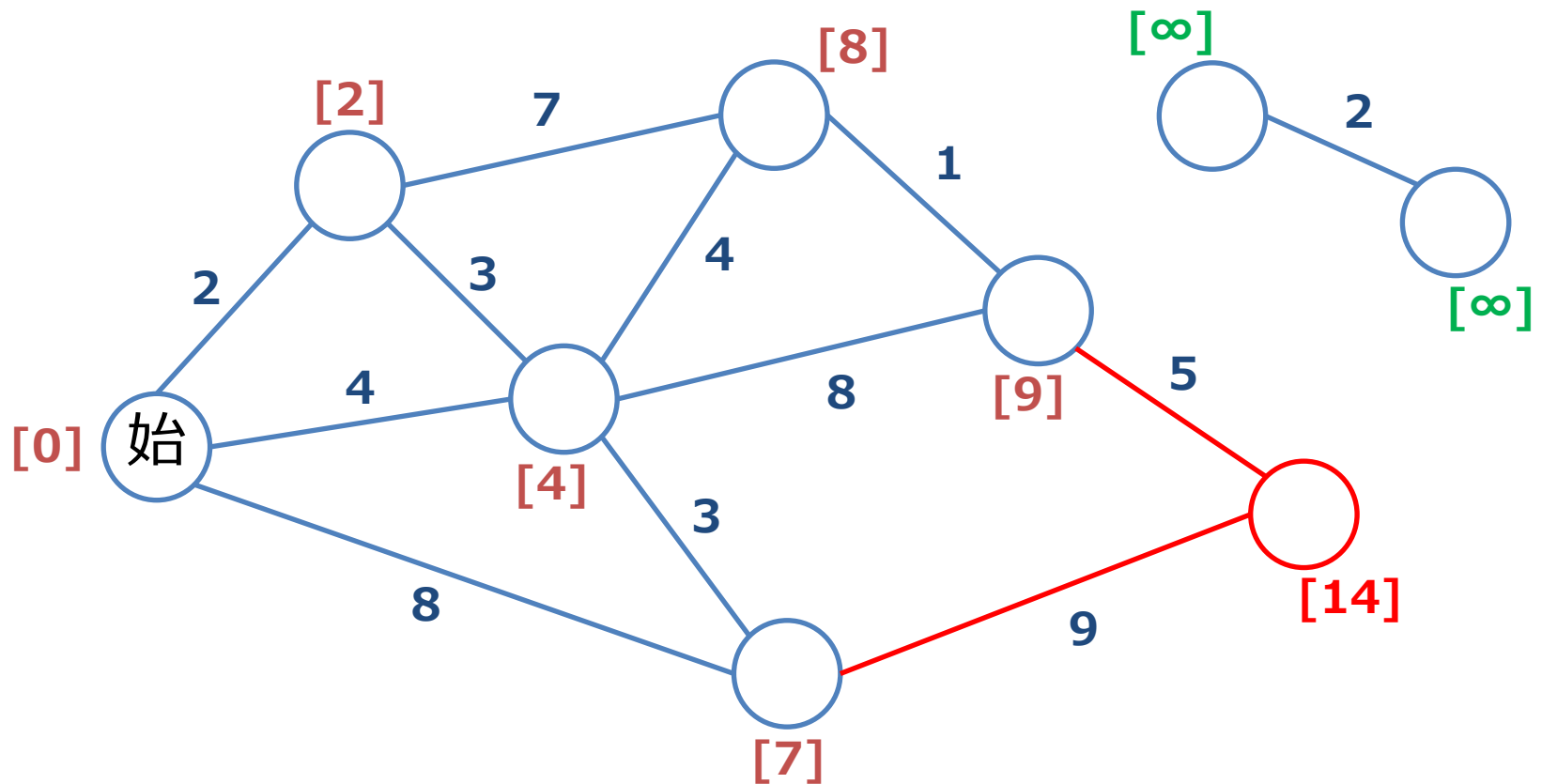
Dijkstra 法

- 方針：始点から近い順に確定させていく



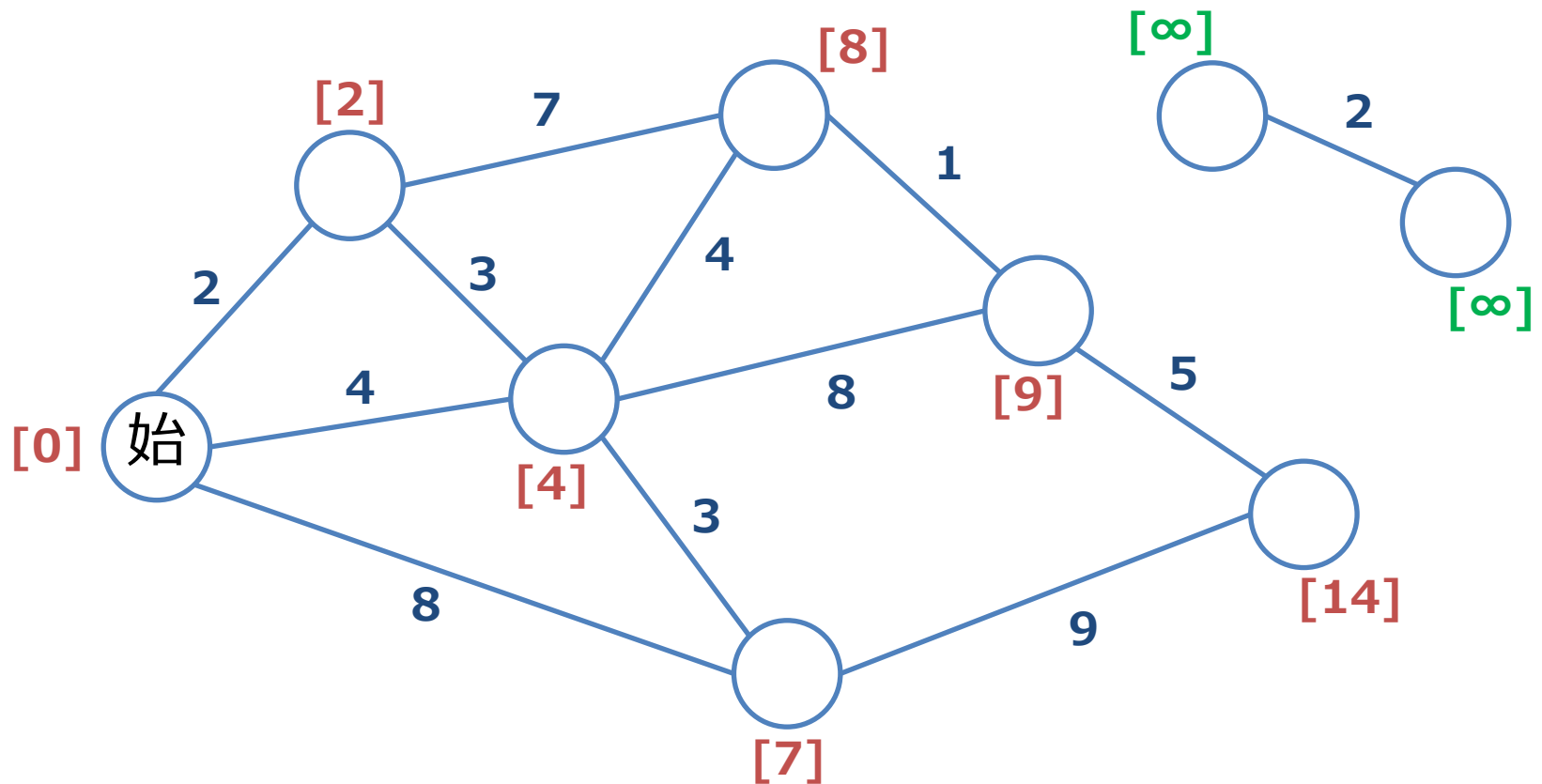
Dijkstra 法

- 方針：始点から近い順に確定させていく



Dijkstra 法

- 方針：始点から近い順に確定させていく



Dijkstra 法

各頂点 u に対し $dist[u] := \infty$, $flag[u] := false$.

$dist[start] := 0$.

以下を繰り返す:

$u := flag$ が $false$ の中で $dist$ が最小なもの.

$flag[u] := true$.

u に隣接している頂点 v それぞれに対し

if $dist[v] > dist[u] + (u \text{ から } v \text{ への距離})$:

$dist[v] = dist[u] + (u \text{ から } v \text{ への距離})$.

Dijkstra 法

- 計算量 (頂点数 V , 辺数 E のとき)
 - 時間 $O(V^2)$
 - 繰り返しが高々 V 回
 - 「最も近い頂点を選ぶ」のに $O(V)$
 - メモリ $O(V)$
- 本問では, 頂点数が $O(K)$ なので $O(K^2)$

Dijkstra 法

各頂点 u に対し $dist[u] := \infty$, $flag[u] := false$.

$dist[start] := 0$.

以下を繰り返す:

$u := flag$ が $false$ の中で $dist$ が最小なもの.

$flag[u] := true$.

u に隣接している頂点 v それぞれに対し

if $dist[v] > dist[u] + (u \text{ から } v \text{ への距離})$:

$dist[v] = dist[u] + (u \text{ から } v \text{ への距離})$.

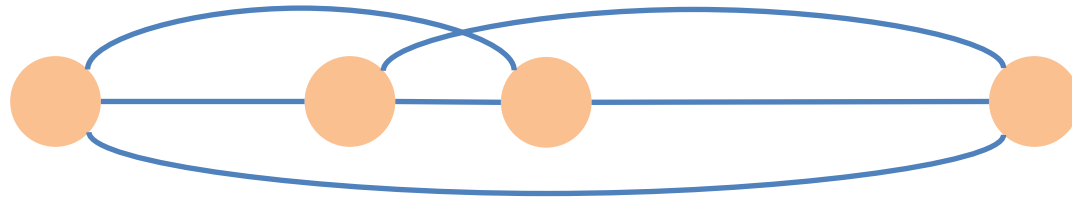
Dijkstra 法

- Dijkstra 法の高速化
 - 優先度付きキュー (priority queue)
 - 「要素の追加」「最小値の取得・削除」の操作を $O(\log(\text{要素数}))$ 時間で行うデータ構造
 - 二分ヒープで実装される
 - C++ の標準ライブラリ `std::priority_queue`
 - 時間 $O(E \log V)$
 - 注: $E = O(V^2)$ のときは $O(V^2)$ より遅い

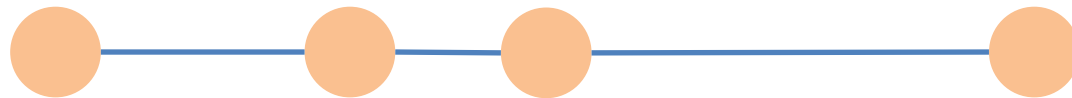
満点解法

- 辺数を $O(K^2)$ から $O(K)$ にしたい

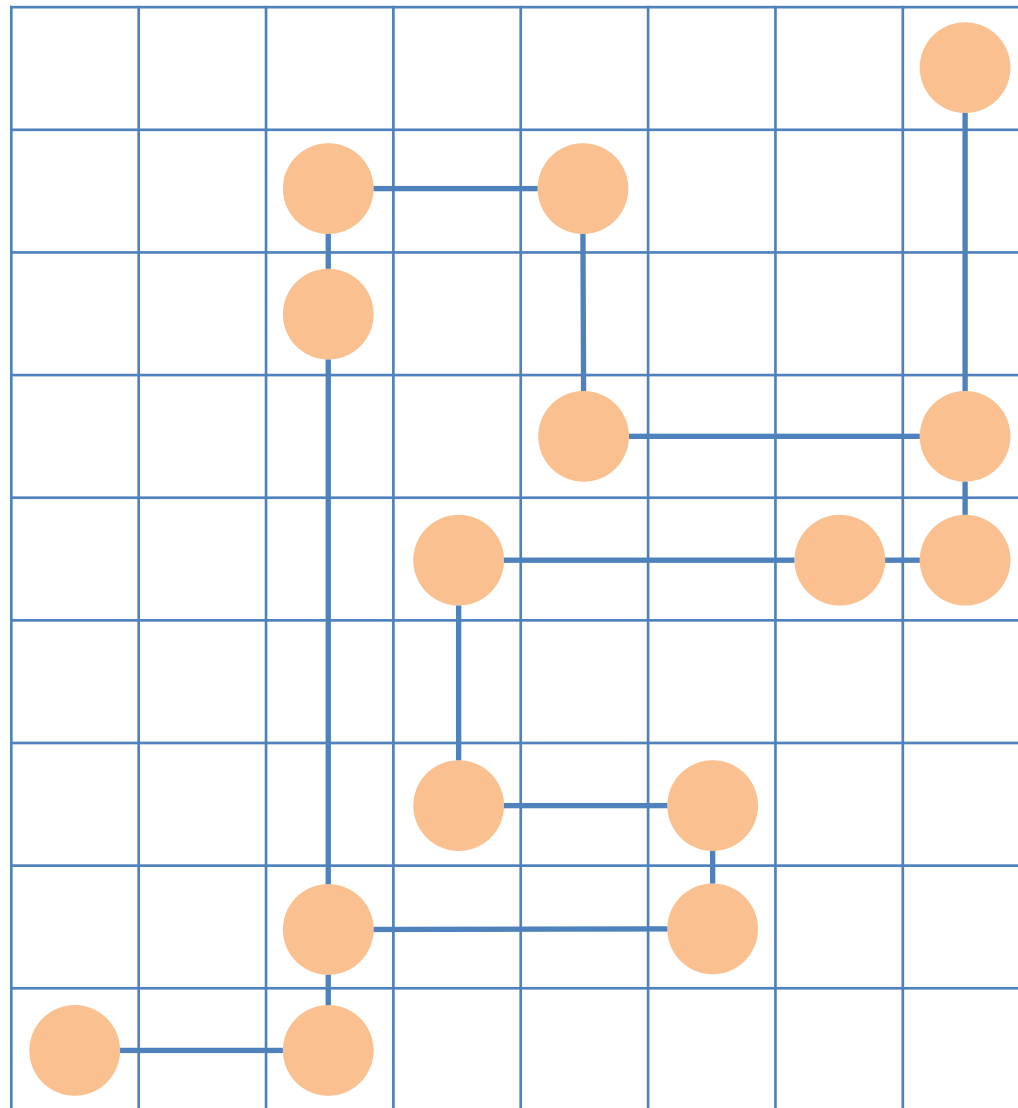
– 辺の張りすぎ



– 辺数を減らす



満点解法



満点解法

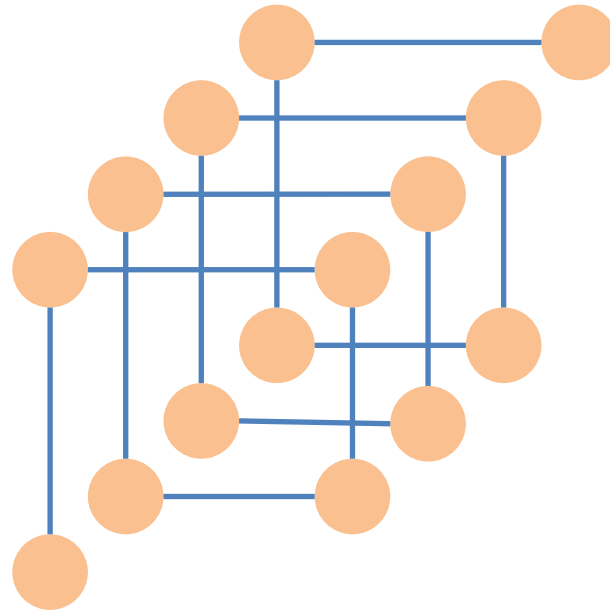
- 不要な辺を張らない
 - 必要なマスを x 座標の小さい順 $\rightarrow y$ 座標の小さい順でソートして, 隣り合ったものの以外は縦の辺で結ばない
 - 必要なマスを y 座標の小さい順 $\rightarrow x$ 座標の小さい順でソートして, 隣り合ったものの以外は横の辺で結ばない
- これで辺数が $O(K)$, 全体で $O(K \log K)$

注意点

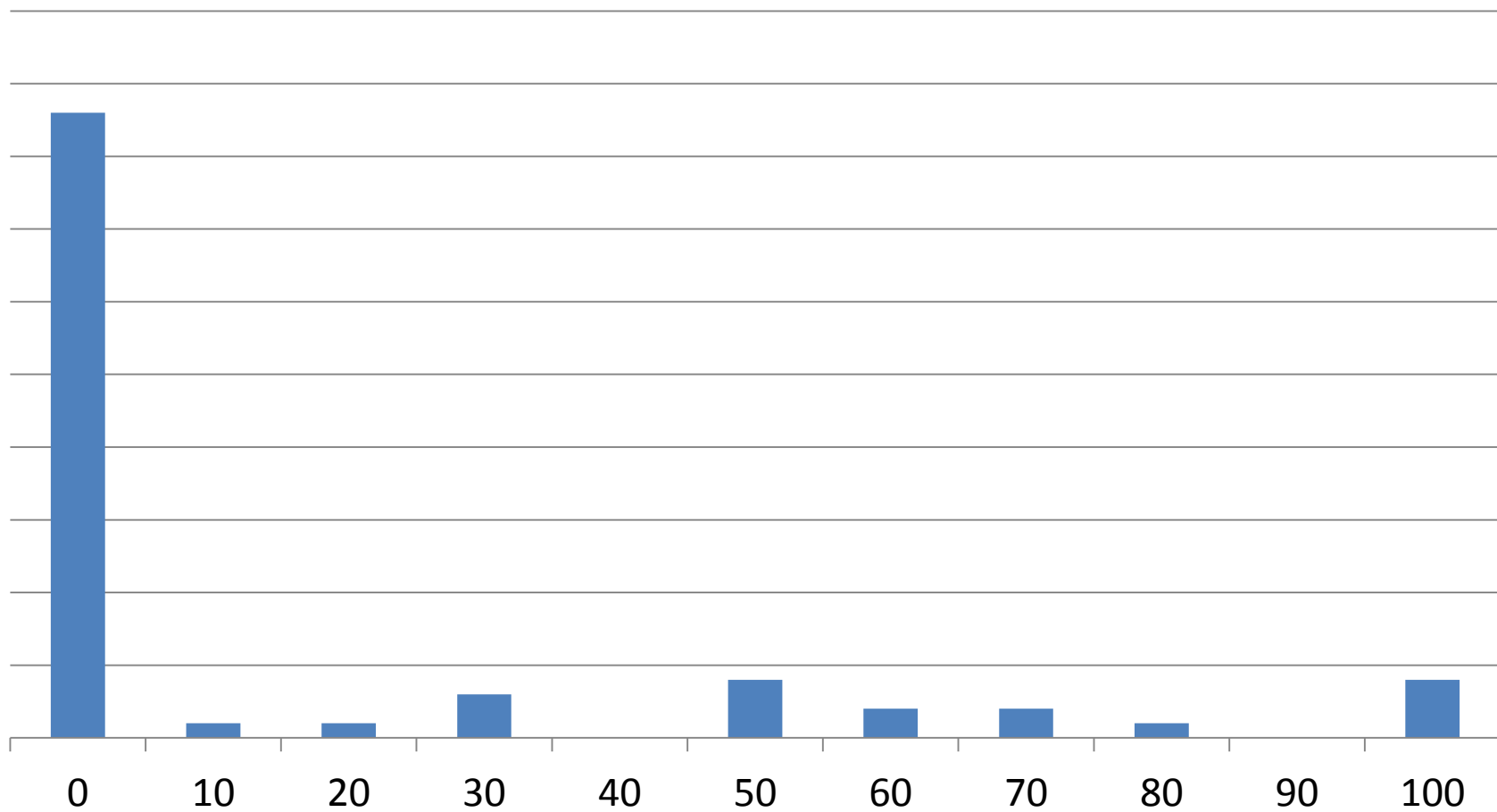
- $(1, 1)$ や (M, N) にはスイッチがあったりなかったりする
 - 頂点数いつもが $2K$ ではないので実装に注意
 - サンプルにもあり

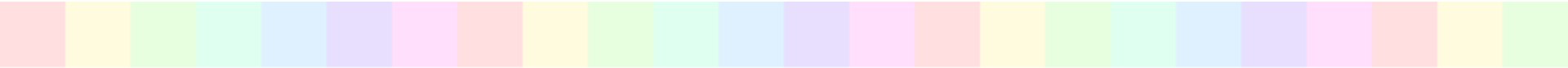
注意点

- 答えが非常に大きくなる可能性
 - 32-bit 整数型 (int) のオーバーフロー
 - 答えが 10^{10} くらいになる例：



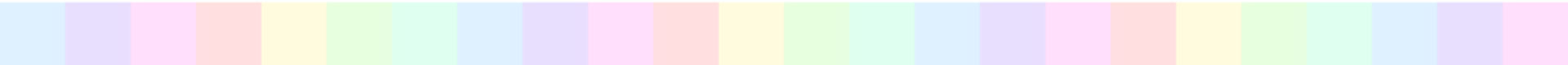
得点分布





おまけ

- priority queue についての参考資料
 - <http://hos.ac/blog/>



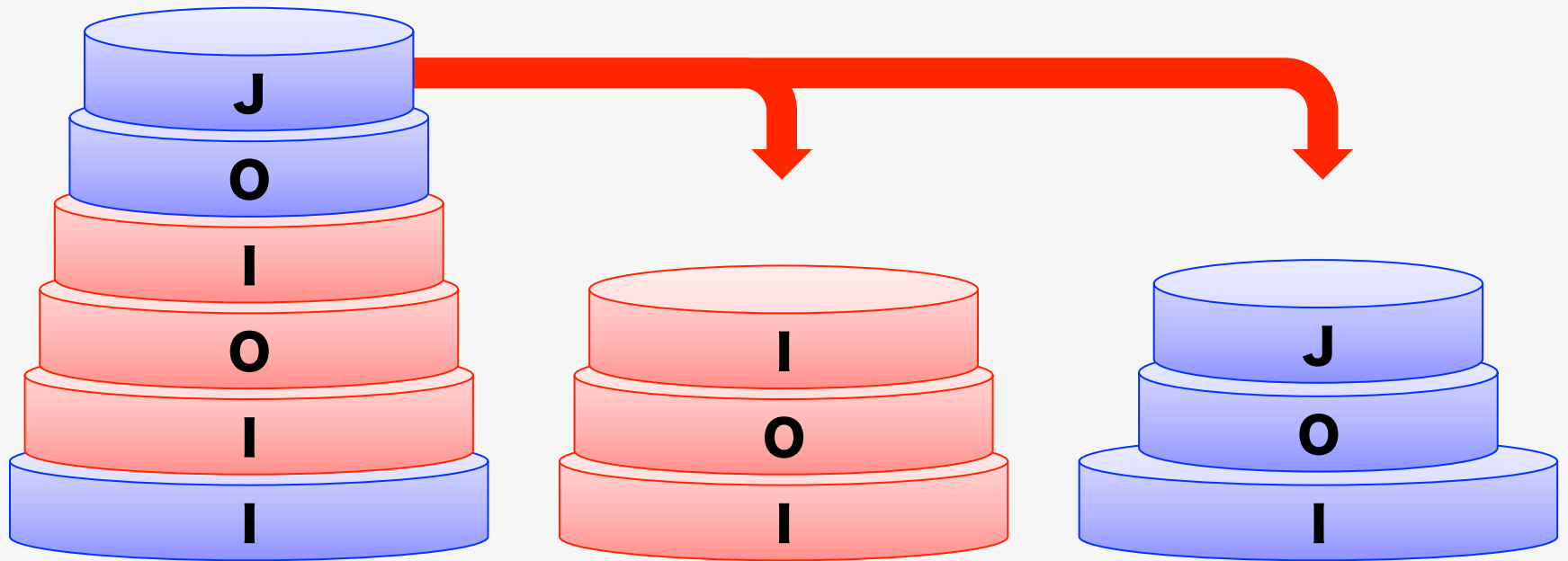
解説: JOIOIの塔

第12回情報オリンピック本選

今城 健太郎 (京都大学, IOI 2006 メキシコ大会)

問題の概要

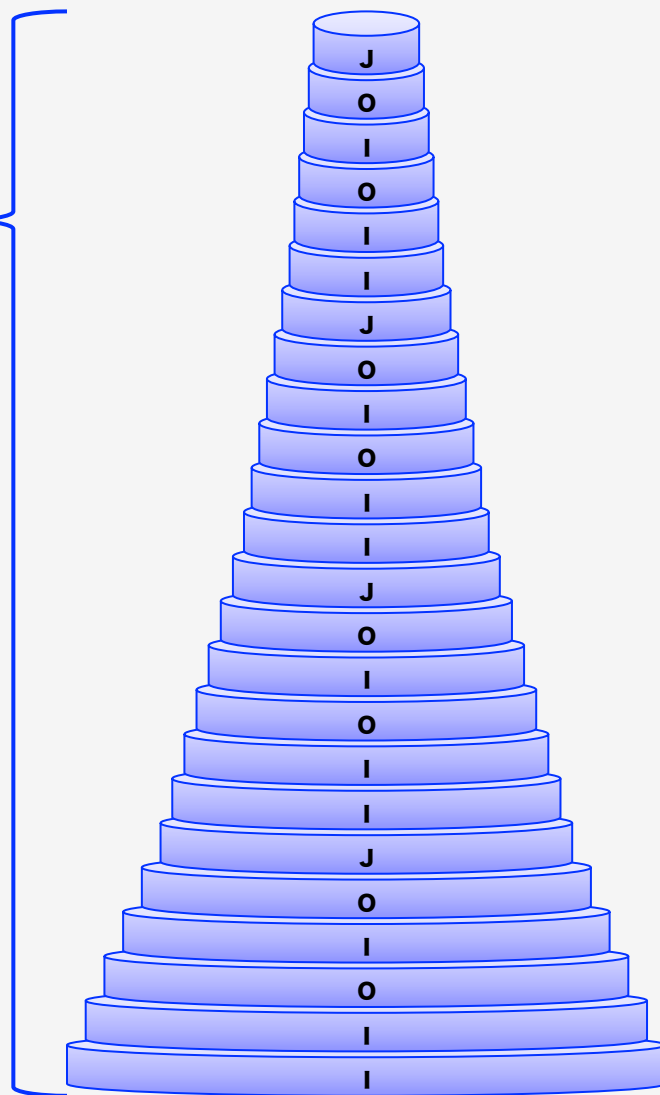
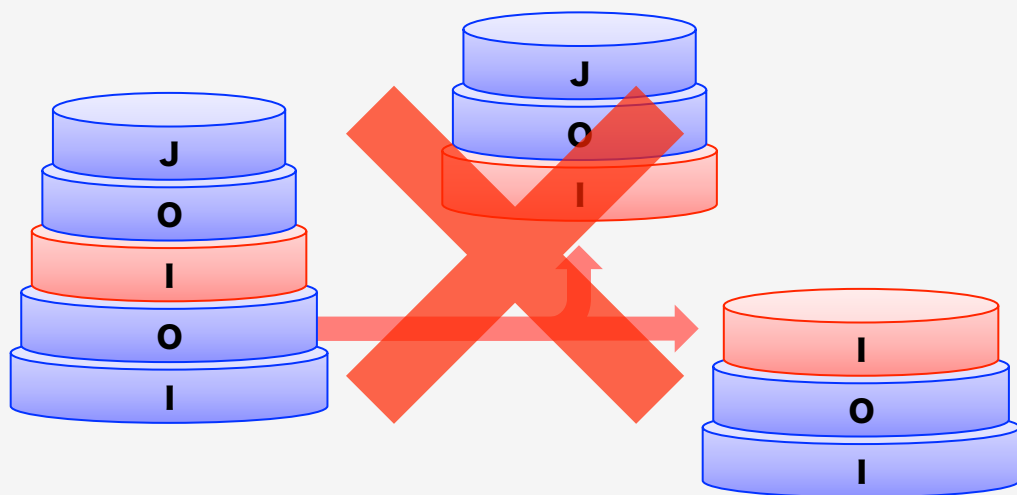
「JOIOIの塔」から「ミニJOIの塔
(IOIもしくはJOIのみからなる塔)」を
できるだけたくさん作りたい



問題の制約

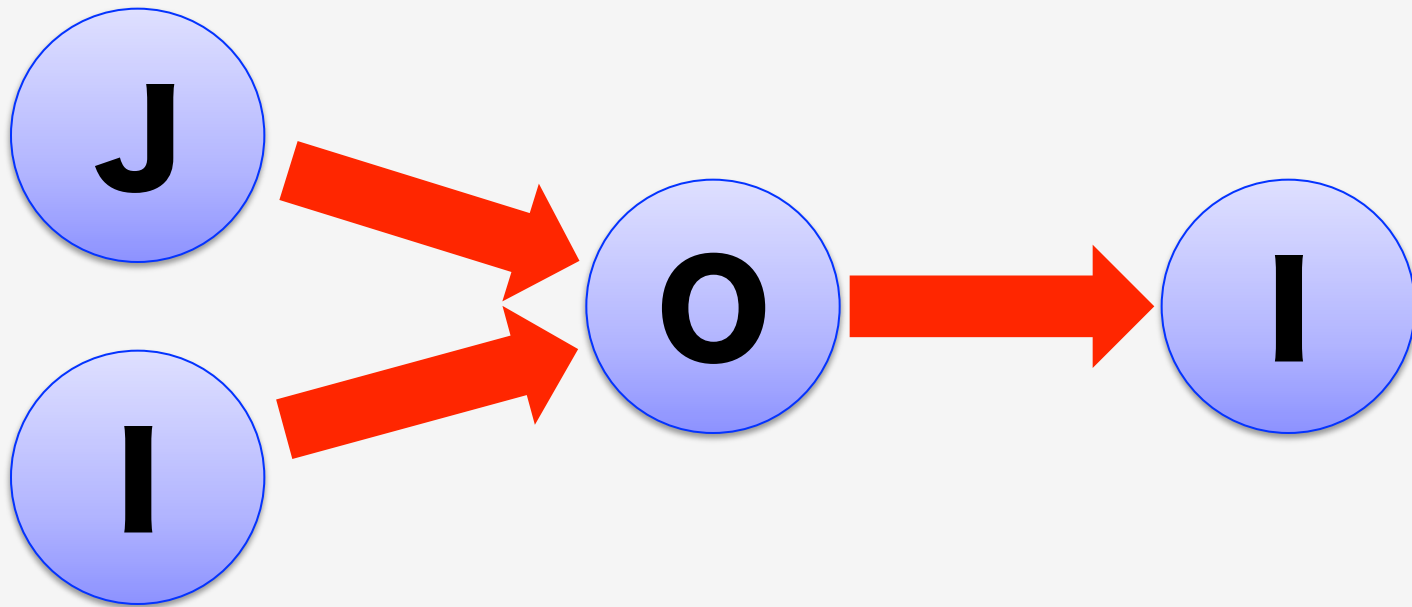
「JOIOIの塔」
の高さ N は100万以下

作る「ミニJOIの塔」は
同じ円盤を共有しない



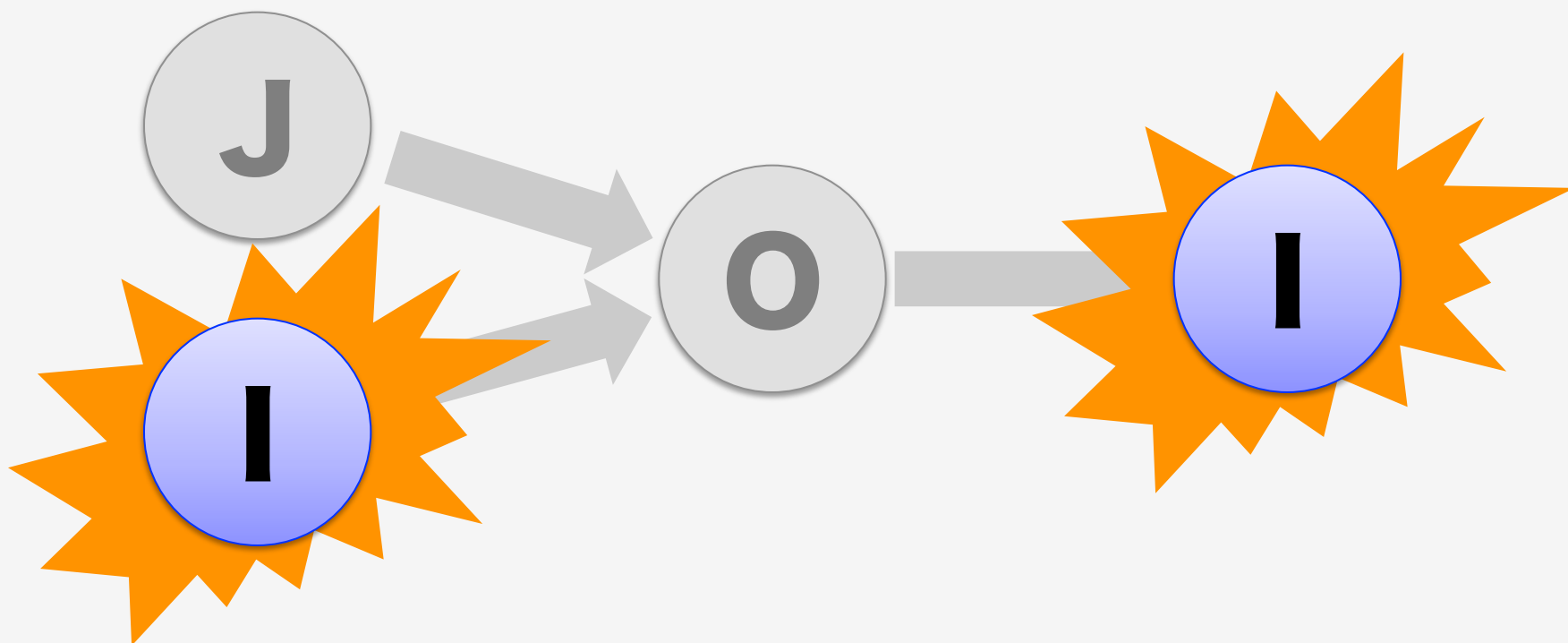
文字の特性の解析（その1）

「JOI」と「IOI」の文字列は
後ろの2文字が共通である



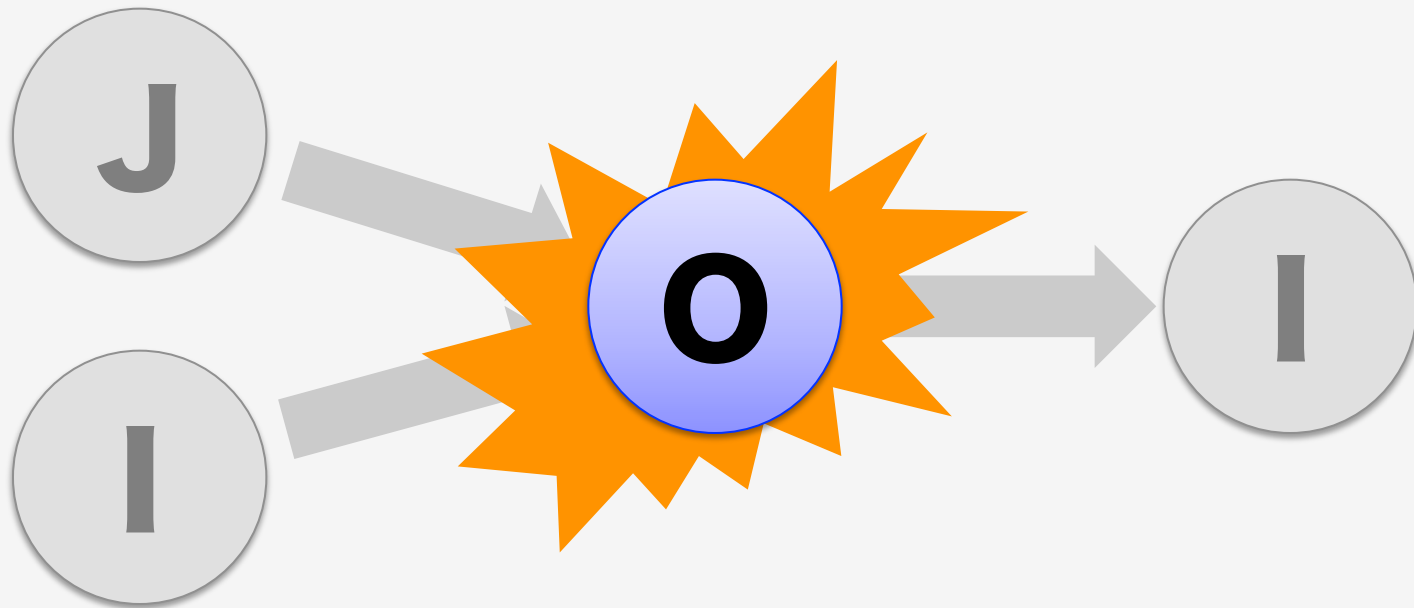
文字の特性の解析（その2）

「I」には使い方が2種類ある



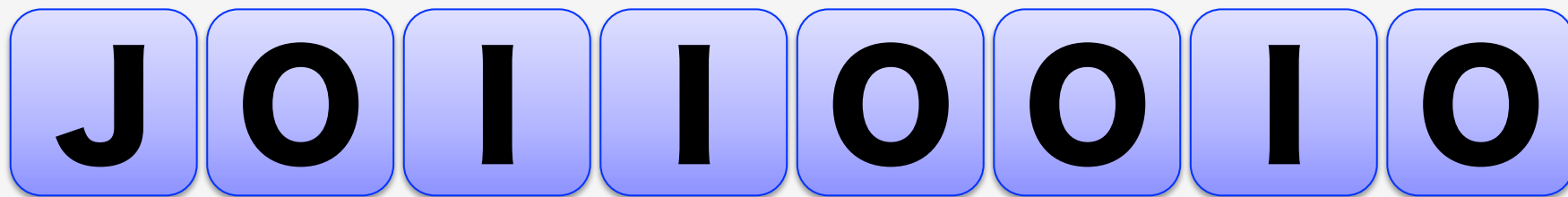
文字の特性の解析（その3）

「JOI」と「IOI」の文字列は
どちらも1度だけ「O」が出現する



個数を決めて作る

JOIOOIOから2個ミニJOIを作ってみよう

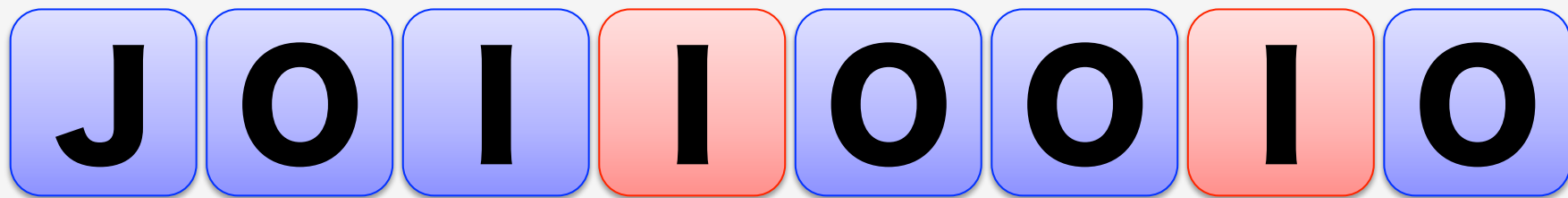


作り方は以下のいずれか



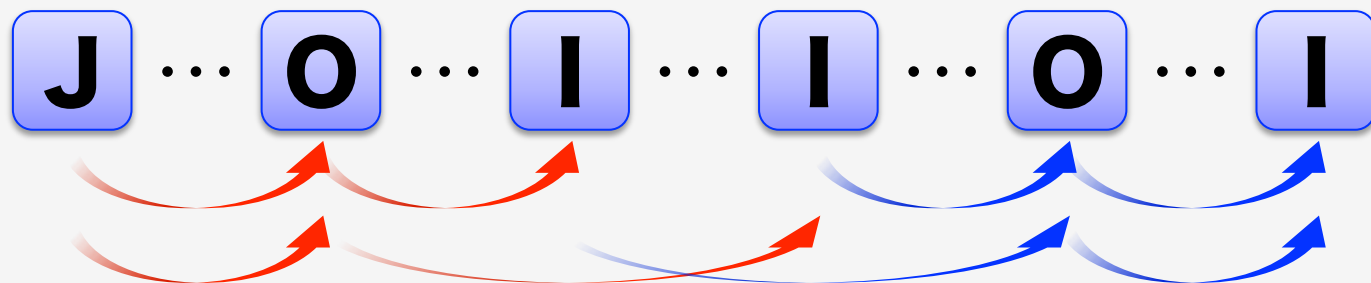
個数を決めて作る

ミニJOIOIの塔を2個作ることができるなら
最後の2個のIを右側のIに使うことができる



なぜなら

IOIの先頭は前のJOI/IOIの最後のIと交換可能



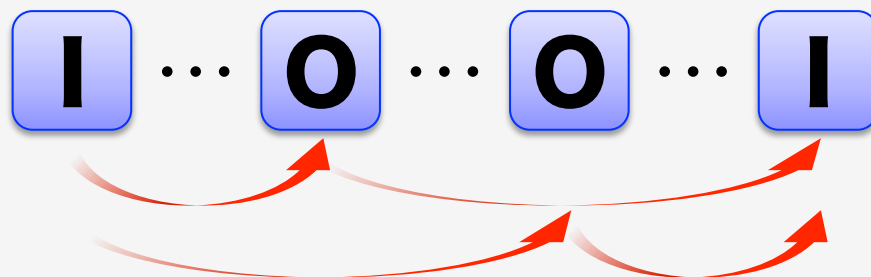
個数を決めて作る

右側のIが決まったので中央のOを決める

⇒ Oは右側のIと組ができるように右から選ぶ



選択肢があるなら右の方が次の選択肢が多い



個数を決めて作る

中央のOが決まったので
Jもしくは未使用のIを右から順に貪欲に選ぶ

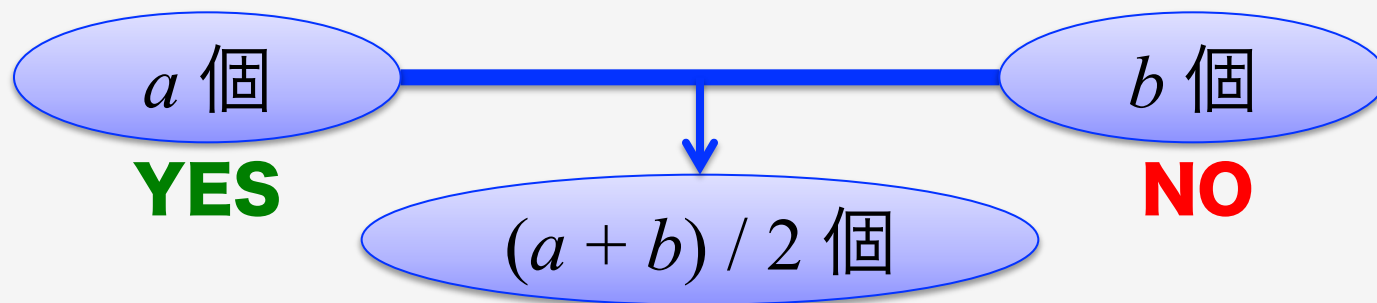


完成

個数の探索

k 個で作れるなら $(k - 1)$ 個で作れる

⇒ 個数で二分探索すれば答えが求まる

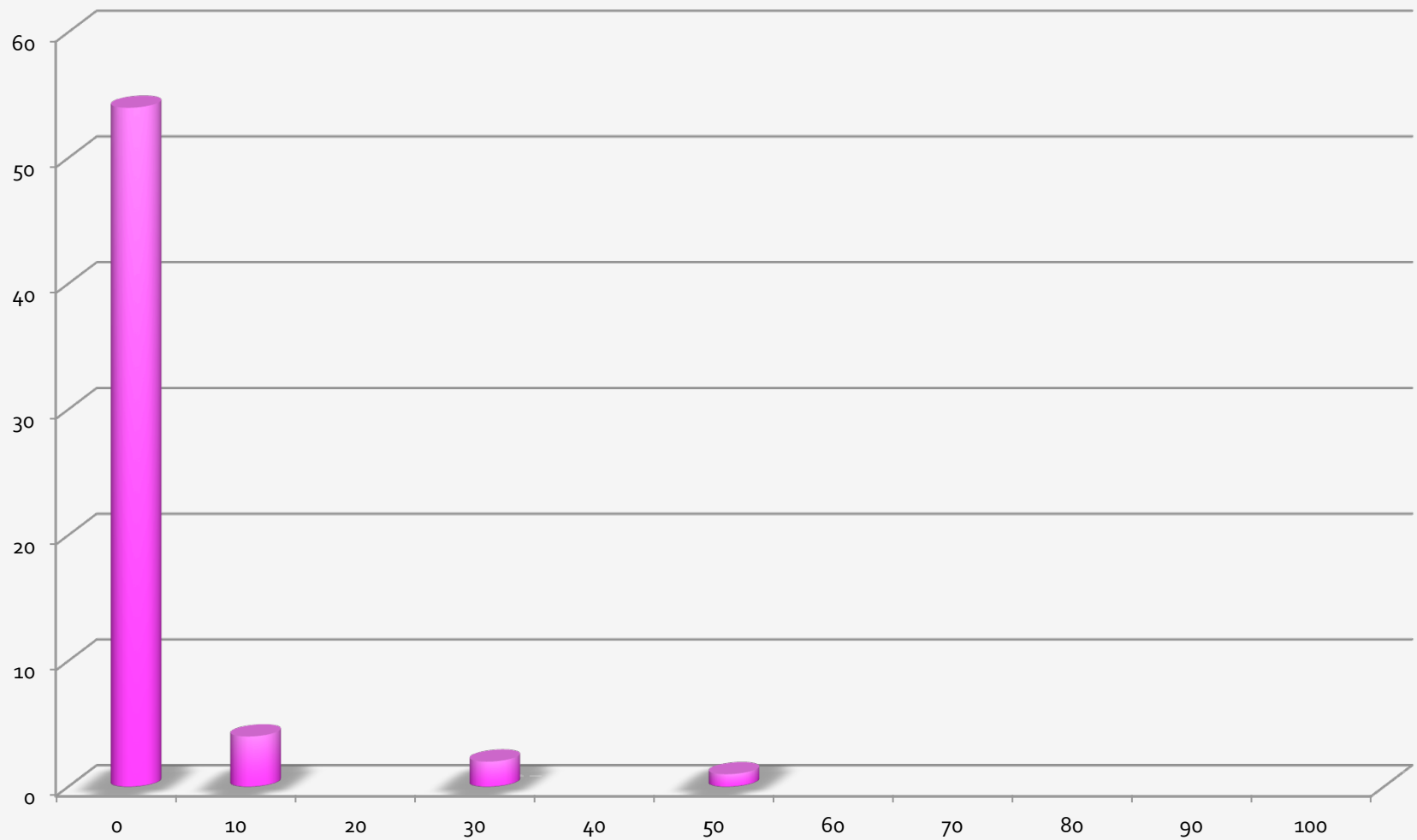


YES なら $(a + b) / 2$ 個以上

NO なら $(a + b) / 2$ 個未満

個数を決めての判定は $O(N)$ でできたので
全体での計算量は $O(N \log N)$

得点分布

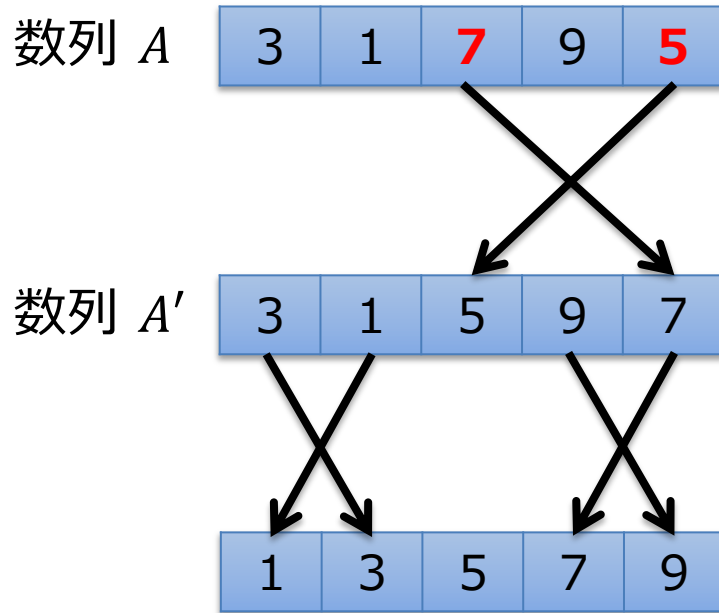


第 12 回 日本情報オリンピック本選 問題5 バブルソート

秋葉 拓哉 / @iwiwi

(東京大学 情報理工学系研究科)

問題



① 数列 A が与えられる

② 数を 2 つ選んで交換する

③ バブルソートする

バブルソートにおける数の交換回数の最小値は？
(②で出来るだけ良い選び方をしたい)

話すこと

1. バブルソートの交換回数とは？
(+ 10 点解法)
2. 交換回数はどのように変化するか？
(+ 30 点解法)
3. 満点解法

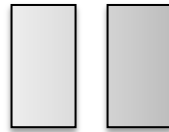
与えられる A に重複がないとして解説します.
重複ありの場合 (最後 20 点分) も本質的には同じ.

バブルソートの交換回数とは？

バブルソートの交換回数

数列 A を $a_1, a_2, \dots, a_n$ として,

数列 A をバブルソートした時の交換回数



$i < j$, $a_i > a_j$ となるような組 (i, j) の個数

大小と左右の関係が逆になっているペアの個数
(ちなみに, **反転数**という名前がついている)

バブルソートの交換回数

$i < j, a_i > a_j$ となるような組 (i, j) の個数

例

3	1	7	9	5
---	---	---	---	---



3	1	7	9	5
---	---	---	---	---

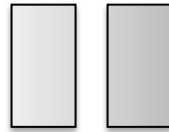
3	1	7	9	5
---	---	---	---	---

3	1	7	9	5
---	---	---	---	---

3 つ

バブルソートの交換回数

数列 A をバブルソートした時の交換回数



$i < j, a_i > a_j$ となるような組 (i, j) の個数

何故？ (略証)

- バブルソートの 1 回の交換で, 反転数は 1 減少
- 完了時すなわちソート済みの配列の反転数は 0

$O(n^3)$ 解法 (10 点)

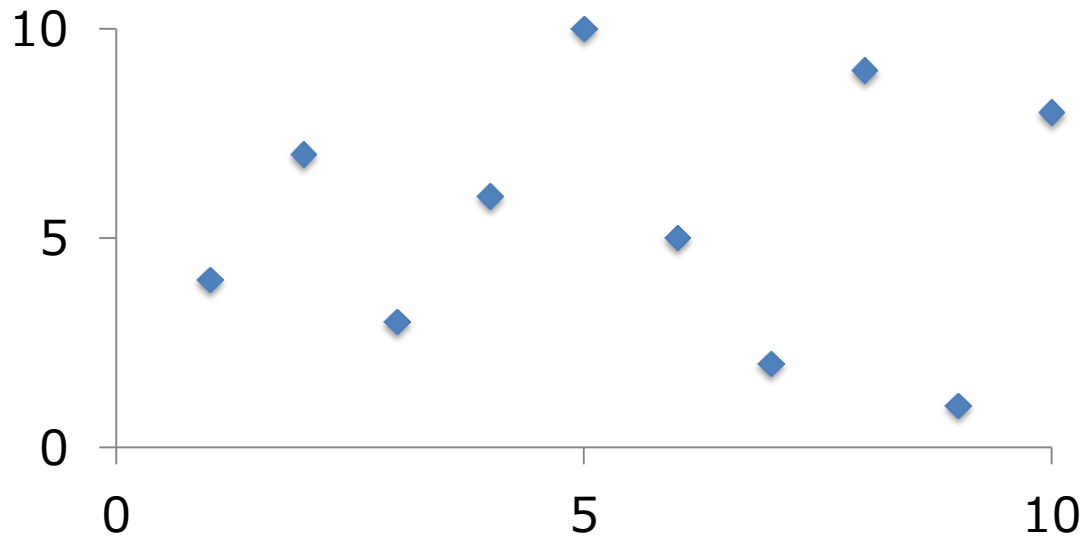
- 交換する 2 つの数を全通り試す
- 交換回数がどうなるか毎回 $O(n)$ で調べる
(その 2 数に関する部分だけ数えなおす)

交換回数はどのように変化するか？

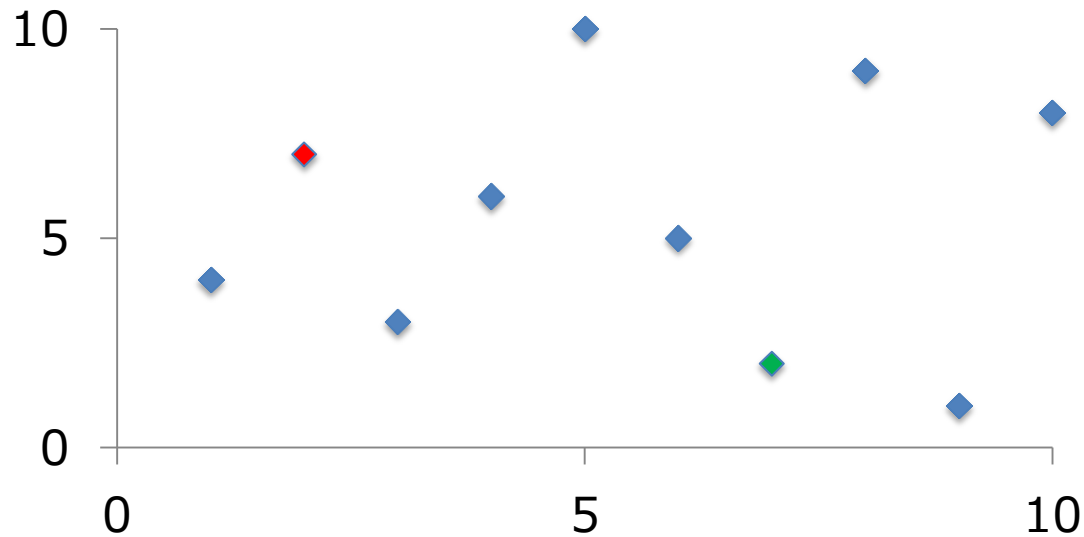
交換回数はどうように変化するか？

数列を平面にプロットして考えてみよう

4	7	3	6	10	5	2	9	1	8
---	---	---	---	----	---	---	---	---	---

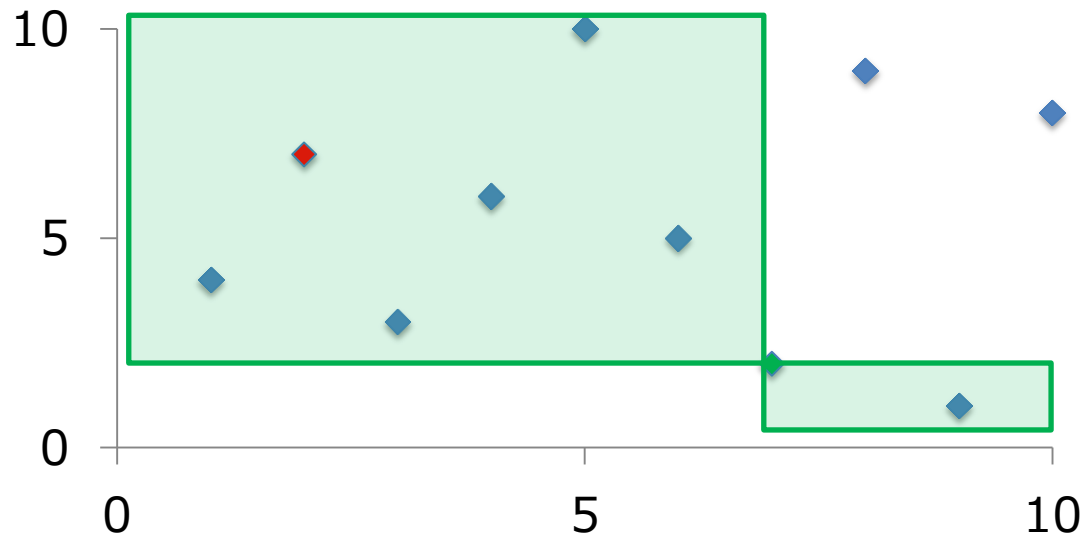


交換回数はどうように変化するか？



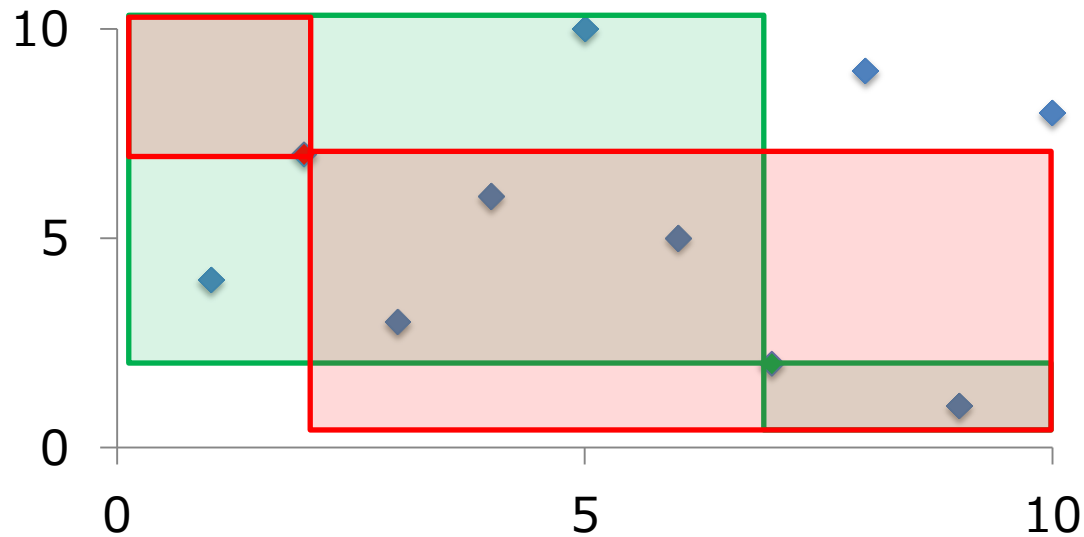
赤と緑を交換した時、
バブルソートの交換回数はいくつ減る？

交換回数はどうように変化するか？



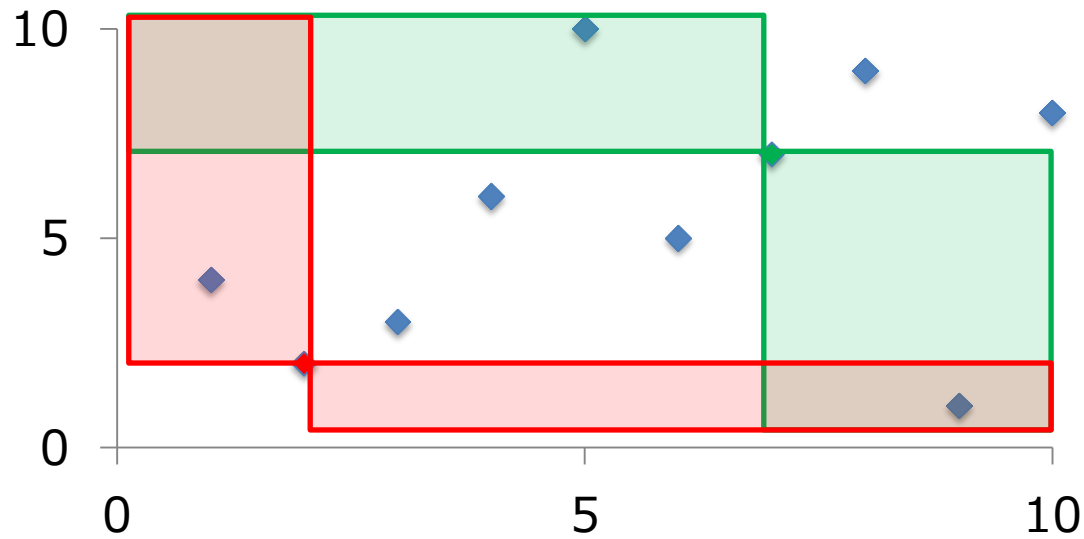
現在，**緑**が反転数に寄与している範囲

交換回数はどうように変化するか？



現在，赤と緑が反転数に寄与している範囲

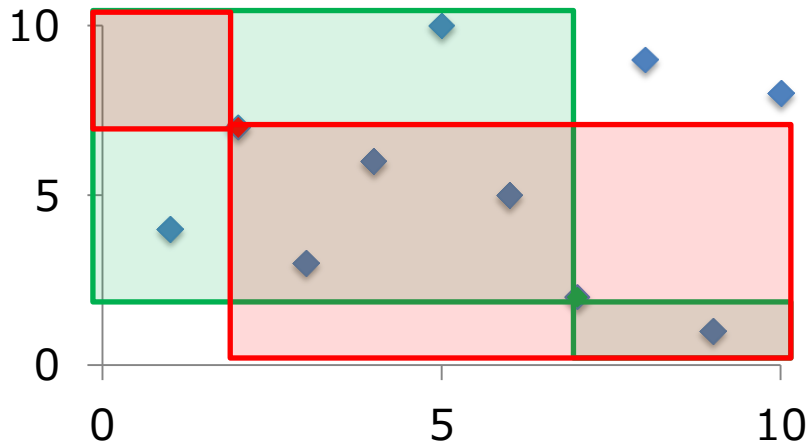
交換回数はどうように変化するか？



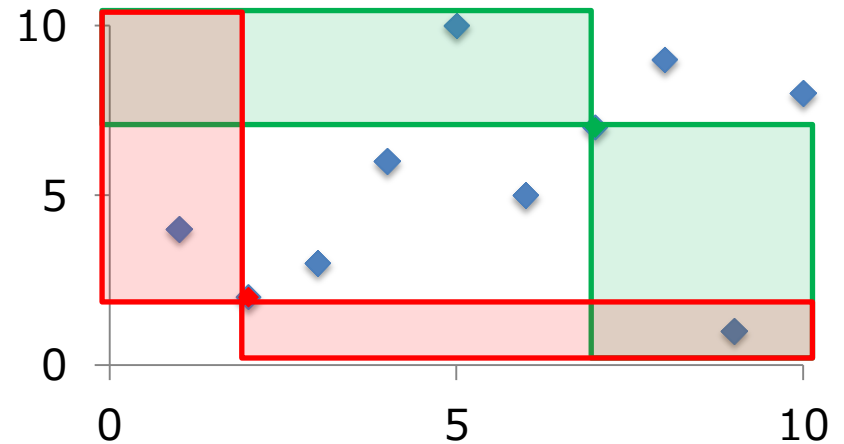
交換後

交換回数はどうように変化するか？

交換前



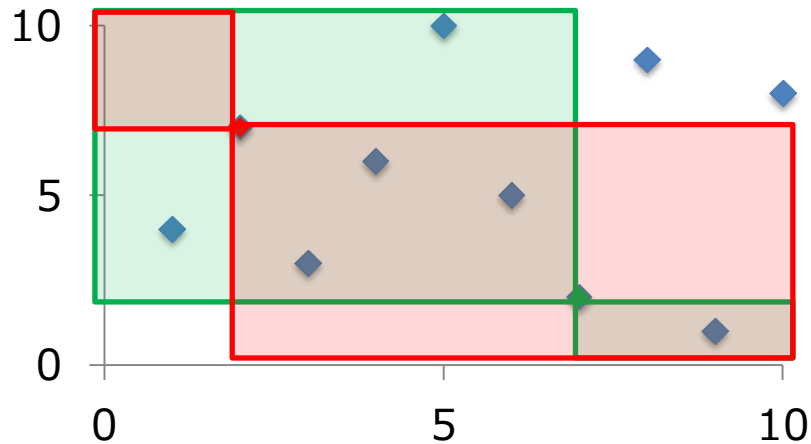
交換後



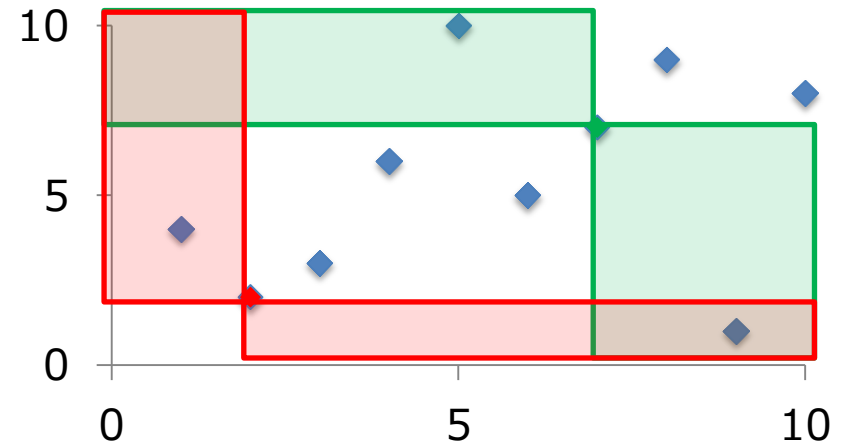
真ん中の長方形がすっぽり消える！
それ以外の部分は変わらない！

交換回数はどうように変化するか？

交換前



交換後



交換回数の減少：

(長方形に含まれる点の数) $\times 2 + 1$

- $\times 2$: 交換前は二重になってるので
- $+1$: 赤と緑それら自体の分

交換回数はどのように変化するか？

つまり,

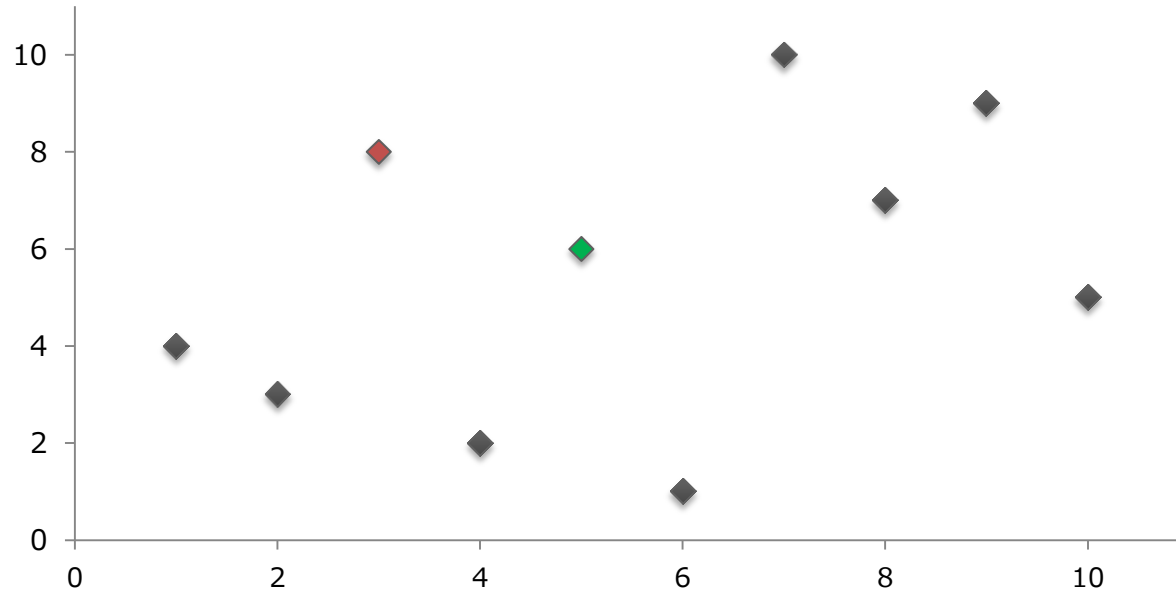
**それを左上・右下にして長方形を描いたら
できるだけ多く点を含むような 2 点を探す
という問題を考えれば良いことになった！**

$O(n^2)$ 解法 (30 点)

- 交換する 2 つの数を全通り試す
- 交換回数がどうなるか毎回 $O(1)$ で調べる
(累積和を前処理として計算しておけばよい)

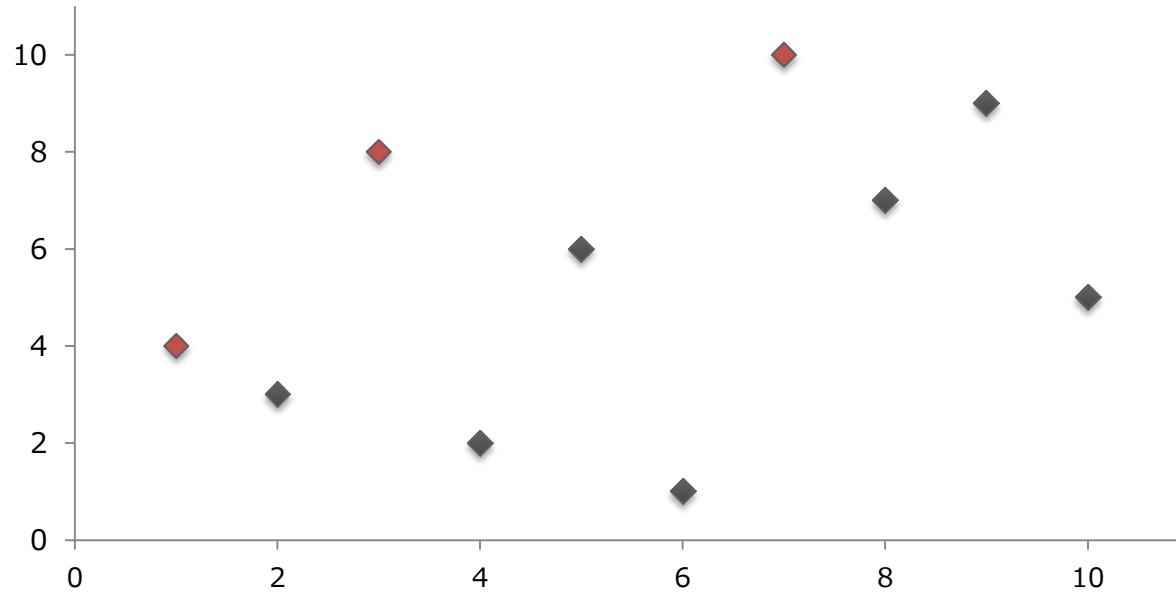
満点解法

考察



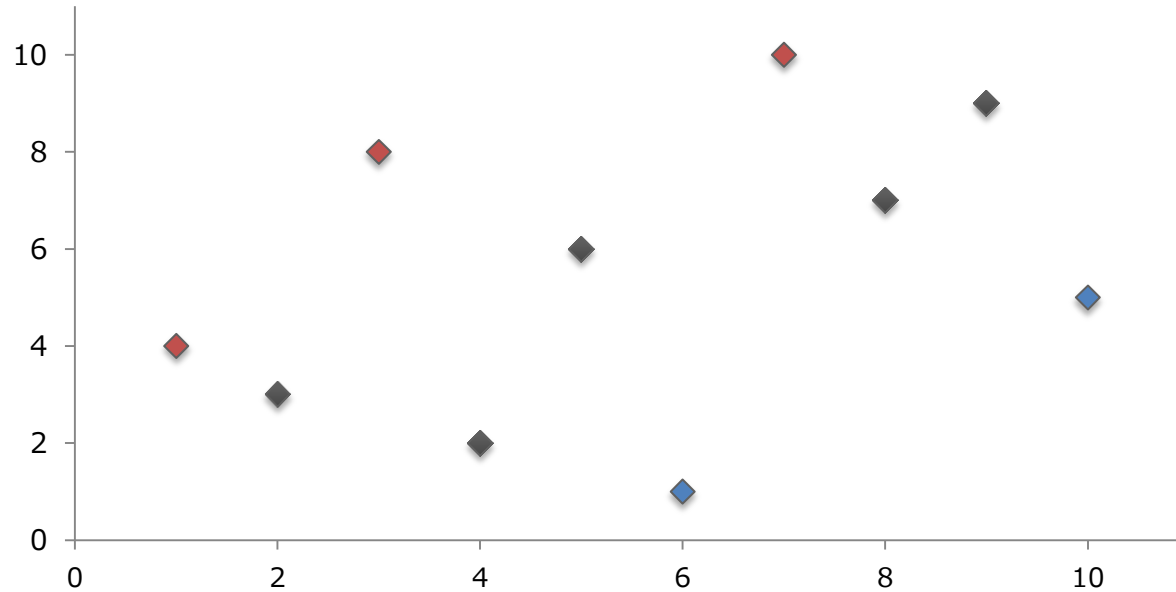
緑を長方形の左上とすることはあり得ない
(緑にするなら赤にしたほうが必ず得)

考察



左上の候補は，自分より左上に点がない点
(上に赤色で示した 3 つだけ)

考察



同様に，右下の候補も絞れる（青い点）
点が 3 種類に分類された！

満点を取りたい……！

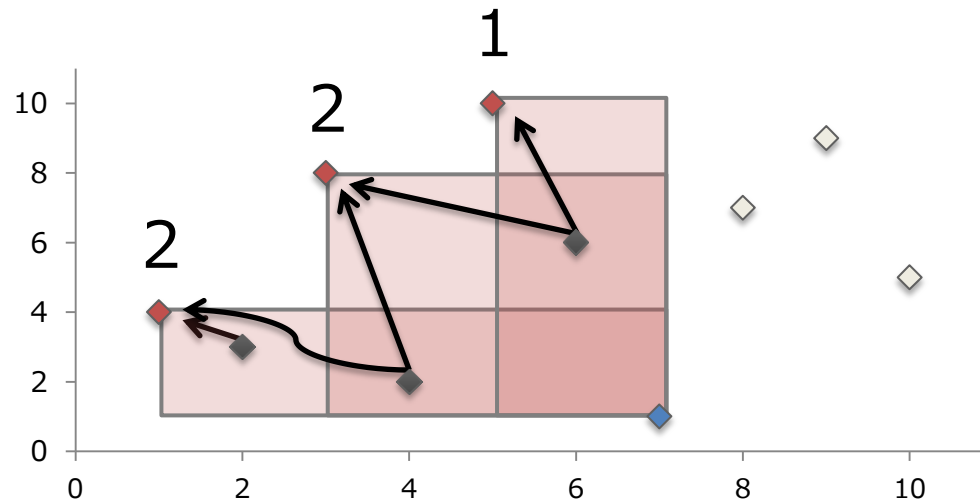
$n = 10^5$ を解くには $O(n^2)$ は無理

- つまり、交換を全通り試せない……
- うまく一度に大量の可能性を処理しなければならないはず……

どうする？

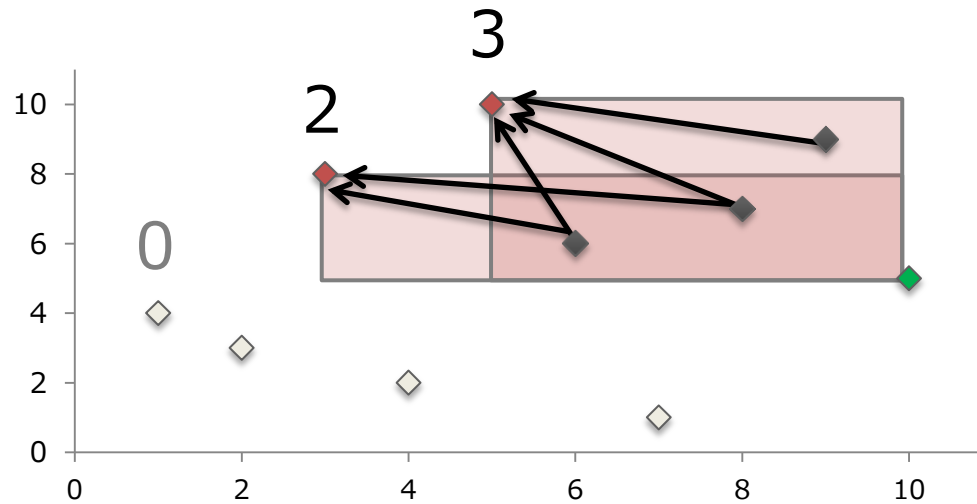
- 右下候補はさすがに全通り試すとする
- それぞれに対して、最良の左上候補を瞬時に知ることが出来れば良い……出来るか！？

解法を考える



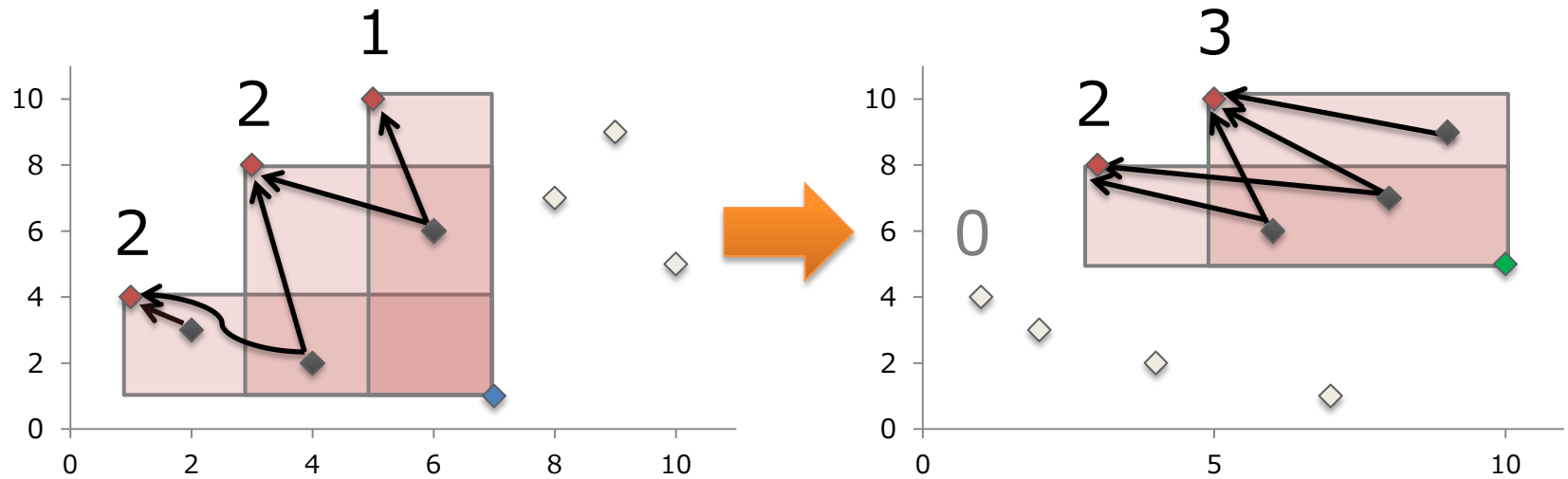
- 青の点を右下にするとしたときの図
- 赤の 3 点は, スコア 2, 2, 1
(スコア = それを左上にした時に囲える点の個数)

解法を考える



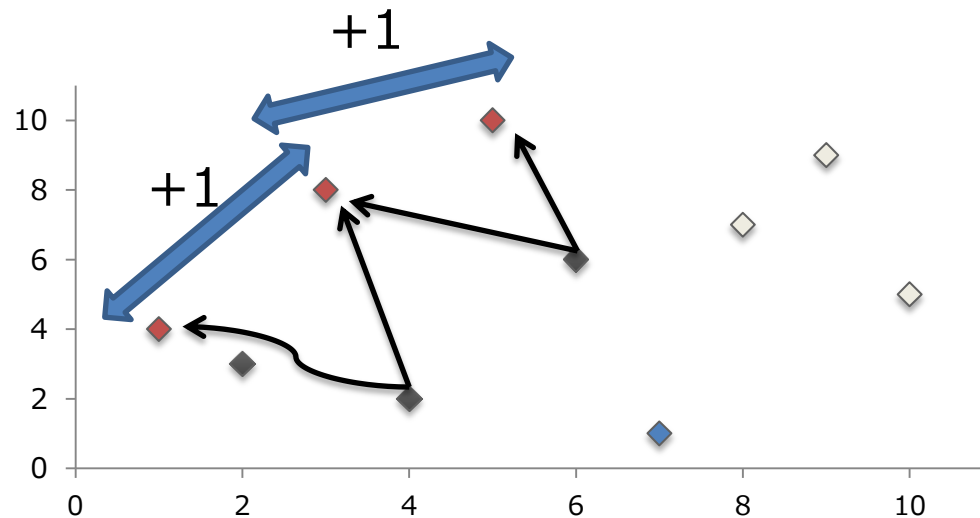
- 緑の点を右下にするとしたときの図
- 赤の 3 点は, スコア 0, 2, 3

解法を考える



左から右をスムーズに作れないか？
欲しいものはスコアのみ

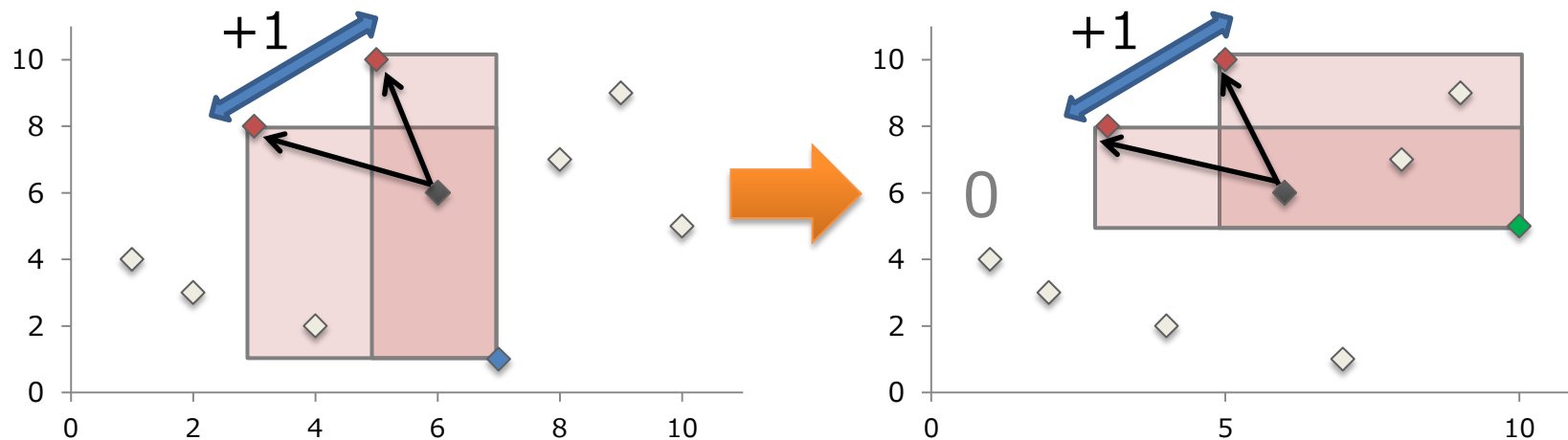
考察



考察①

赤い点の列に対して、
灰色の点が影響する部分は連続区間
(連続するある部分に +1 している)

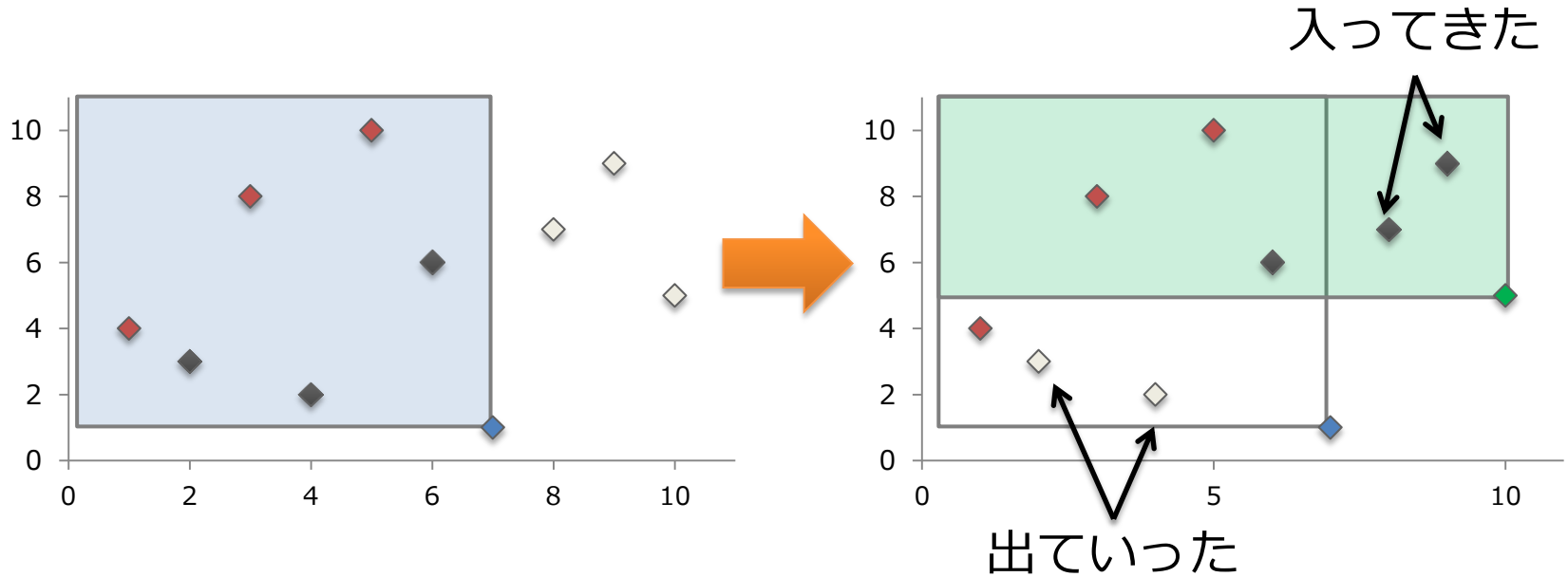
考察



考察②

灰色の点が影響する部分は
右下が違ってても変化しない

解法を考える



考察③

関係する灰色の点の変化は、
「右の点が入ってくる」
「下の点が出ていく」のみ

アルゴリズム

アルゴリズムが作れそうになってきた！

- 左上候補（赤の点）に関するスコアを管理
- 右下候補（青の点）を左から順に試す
 - スコアの最大値を見る
- 他の点（灰色の点）は
 - 青の点の x 座標が自分より大きくなったら「入る」（赤の点のある区間に $+1$ ）
 - 青の点の y 座標が自分より大きくなったら「出る」（赤の点のある区間に -1 ）

アルゴリズム

アルゴリズムが作れそうになってきた！

- 左上候補（赤の点）に関するスコアを管理
- 右下候補（青の点）を左から順に試す
 - スコアの**最大値を見る**
- 他の点（灰色の点）は
 - 青の点の x 座標が自分より大きくなったら「入る」
（赤の点の**ある区間に +1**）
 - 青の点の y 座標が自分より大きくなったら「出る」
（赤の点の**ある区間に -1**）

スコアの管理

列において、以下を効率的に行いたい

- 区間に加算
- 最大値の取得

セグメント木を使ってスコアを管理すれば、両方 $O(\log n)$ で実現可能

各点は、1 回入って、1 回出る。

よって、**全体では $O(n \log n)$!**

セグメント木については、以下などを見て下さい。

「プログラミングコンテストでのデータ構造」

(<http://www.slideshare.net/iwiwi/ss-3578491>)



↑ 載ってます

ところで

- 交換回数をどれぐらい減らせるかは分かった
- ところで、最初の列 A のバブルソートの交換回数はどうやって求めるのか？
 - $O(n^2)$ かけて実際にやるのは無理……

2 つの $O(n \log n)$ の方法

- Binary Indexed Tree を使って数える
- 分割統治法により数える



両方載ってます ↑

点数分布

